O'REILLY®

Training Data for Machine Learning

Human Supervision from Annotation to Data Science



Anthony Sarkis

to the grocery store, perhaps store A may have a different threshold for what's OK to sell than store B.

This leads us to considerations around who supervises the data—their backgrounds, biases, incentives and more. What *incentives* are the people who supervise the data given? How much time is being spent per sample? When is the person doing it? Are they doing a hundred samples at once?

As a supervisor, you may work for a firm that specializes in data supervision, or you may be hired by one company, or you may be a subject matter expert. Finally, the supervision may come directly from an end user, who is potentially unaware they are even doing supervision, e.g., because they are doing some other action in an application that just happens to also generate training data. An example comparison between being aware of the supervision being done versus unaware is shown in Figure 6-1.

Generally, if an end user is doing the supervision, the volume and depth of supervision will be lower. However, it may be more timely, and more contextually specific. Consider that both can be used together. For example, an end user suggesting that something was "bad" may be used as a flag to initiate further direct supervision.

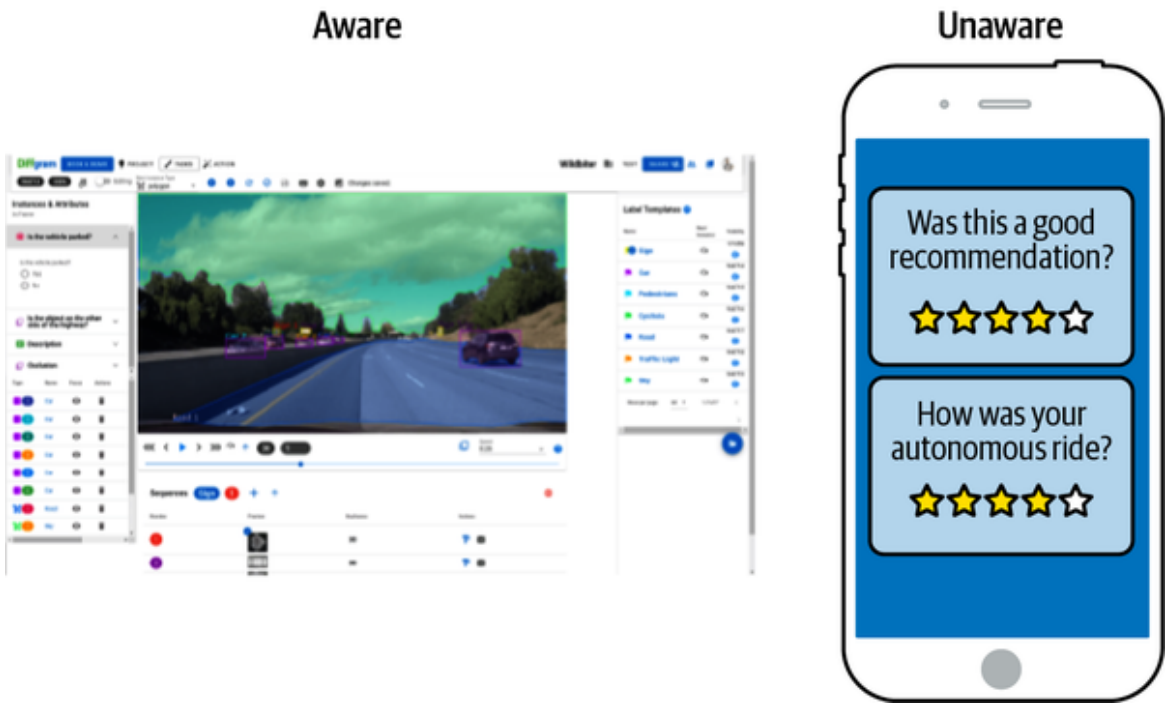


Figure 6-1. Visual difference between a scenario where the user is clearly aware they are training a system and one where the user may be unaware

This is all in the context of explicit (or direct) supervision of the data, someone is directly viewing the data and taking action. This contrasts with classic training data, where the data is implicitly observed "in the wild" and is not editable by humans.

Intentionally Chosen Data Is Best

Some companies scrape the internet to create big datasets from historical data. That's one approach; however, most supervised commercial datasets are novel. They consist of three main components:

- A newly created schema
- Newly collected raw data
- New annotations using the new schema and new raw data

Some examples of novel datasets are of privately recorded sports footage that is annotated, images of construction sites, and newly annotated sales calls. This is not random data from the internet. In fact, most supervised

datasets are private (even if some of the original data may be public), not in the public domain.

Intentionally chosen data creates the best alignment between schema, raw data (BLOBs), and annotations. As we know, those must be well aligned to get good results.

Working with Historical Data

The opposite of fresh data is data from historical contexts. If the schema, raw data, or even annotations come from historical sources, they will naturally be bound or related to that historical context.

Why is this a concern? Historical data usually includes many problems, spanning from the technical to the political. For example, the historical data may have used a sensor different from modern sensors, or political concepts may have shifted such that the categories don't make sense anymore, etc. Historical data also may not have a well-defined "lineage," meaning that problems may not be obvious at first glance. Often, the very reason historical data is used is in an effort to save time or money. However, the larger the dataset, the more historical artifacts are encountered, and the more difficult they are to validate for modern contexts.

It's worth repeating here that in many classical machine learning problems, the historical data was the *primary* form of data available. For example, if you are training an email spam-detecting model, existing spam emails are a great source of data. However, in the supervised context, the domain space is often more complex, e.g., images, videos, and data derived from other sensors. The schema and prediction goals are then more complex. At the moment, the best way to achieve good results in this complex environment is to get great alignment between the schema, raw data, and annotations. Usually, overly leaning on historical sources makes this difficult.

You know you are creating "new" data and not bound to history by doing the following:

- Creating the schema yourself, or validating each record in an existing "must use" taxonomy, or reframing an existing taxonomy inside a new schema.
- Collecting novel data. It may be acceptable to also source data from existing datastores, as long as it's treated as raw data, and the age of the data is well understood.
- Annotating fresh data. This will always be required if either the schema or raw data are new, and ideally both are new.
- Any "must use" historical data is validated within the concept of the current situation.

It can be very tempting to lean into some historical concepts, especially for things like schema, where there may be well-defined existing taxonomies. If you must use an existing taxonomy, then validate as many records as you can for your new context. Ideally, for real-world cases, review as much of the data as possible, and it's especially important that the schema is created novelly, by you, for your specific needs. This means any framing concepts that could influence the analysis (bias, historical concerns, etc.) are reflections of your current view, not some generic historical concept.

To wrap up this section, I'll make a brief comment on costs. In Chapter 2, I already discussed the many cost trade-offs around creating new data. The bottom line when it comes to historical data is that it's often not really a true cost comparison of "use existing" versus "create new." Rather, the historical data is usually either technically or politically untenable at any realistic cost. When historical data is used, it's often only in part, and only when the parts that can be validated and brought forward are done so out of necessity or obvious usefulness.

Training Data Is Like Code

It's useful to think of training data like code because that helps frame the importance of it. The moment that someone says coding, I think "that's serious," and there should be the same reaction for the creation of training

data. It's also useful in that it helps convey the importance of subject matter experts, and that they are exercising a very material level of control in the overall process. This helps frame the fact that data science is consuming the data produced by training data, not coming up with the "program" (model) exclusively by themselves.

Here, I will compare training data as a higher- level abstraction to already high-level programming languages like Python. Similarly to how we use languages like Python to be more expressive than we can be in assembly code,[1] we can use training data to be more expressive than we can using Python.

For example, instead of defining a program to detect strawberries using `if` statements, we can show an image containing a strawberry, then label a strawberry as the "desired result." This difference is illustrated in Figure 6-2. The ML process then uses the "desired result" to create a "program" (the model).

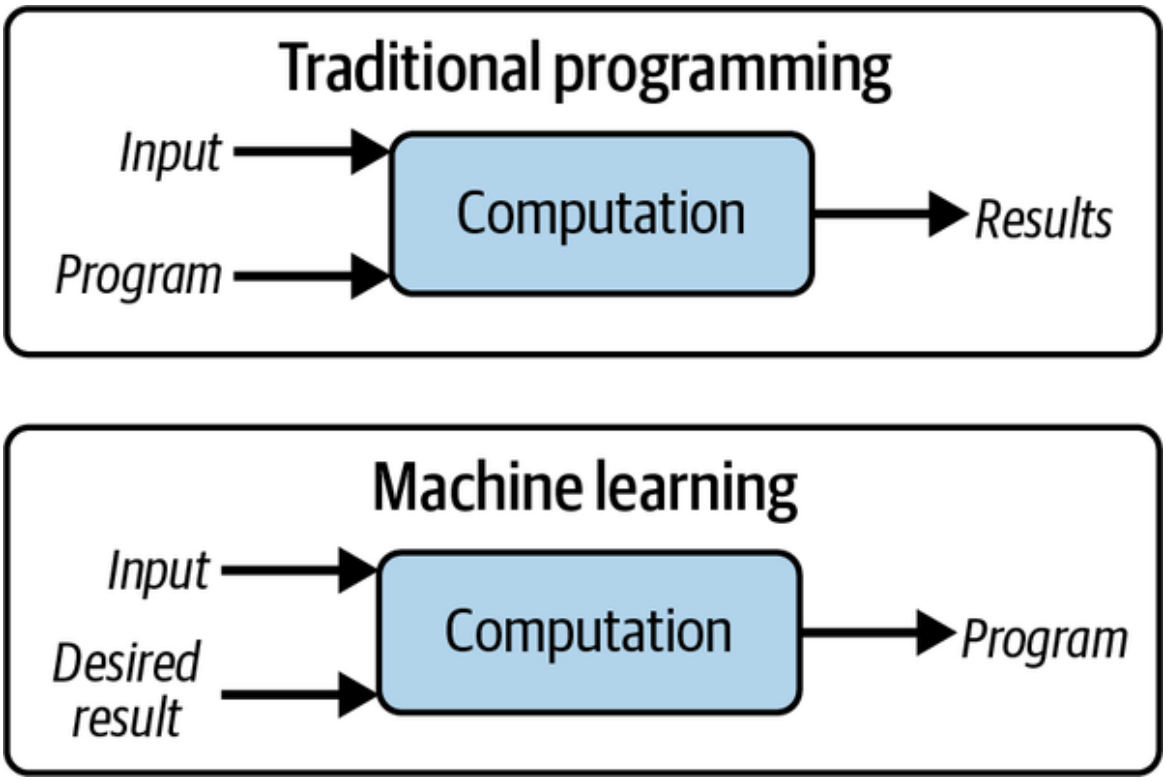


Figure 6-2. Difference between traditional programming and machine learning

While conceptually this may be similar to how we just tell a child “that’s a strawberry,” in reality we are programming a specific technical label, “strawberry,” and technical spatial coordinates, onto defined samples of data. While we may hope those samples are generalizable, the actual work done is the literal encoding of our knowledge into a dataset.

Software programs often have business logic, or logic that only makes sense in the business context exterior to the program. Similarly, training data relates to an external context, and is mapped to that exterior context of raw data and related assumptions. It’s human meaning encoded in a form ready for consumption by a machine learning algorithm. Put simply, training data is another way to write programs via specifying desired results, rather than lines of code.

Shifting to a slightly more technical stance, training data controls a system by defining the ground truth goals. These goals eventually result in the creation of machine learning models. We may not know what model will be run on the data, but as long as the model outputs data similar to the ground truth, then the system is working as intended. This is similar to how we don’t usually care how high-level languages like Python are “compiled/just-in-time compiled,” as long as the code runs.

Of course, there are limits to this analogy, and there are many practical and philosophical differences. Code explicitly defines logic and operations, whereas training data provides examples that the model must generalize from. The intermediary step of model creation is less direct than that from code to program output. My intent is not to convince you there’s a one-to-one analogy between coding and training data, but merely that training data is *like* coding in many forms.

Surface Assumptions Around Usage of Your Training Data

With training data, we define a set of assumptions, such as the schema itself, what the raw data will look like, and how the data will be used, and expect the training data to be useful in that context (and only in that context).

As an analogy, if you are an Olympic runner training to run the 100 meters, you are training to run based on the assumptions for the 100 meters, not the 400 meters, 800 meters, or the high jump. Because while similar, those events are outside of the scope of what you expect to encounter—the 100 meters.

Unlike the Olympics, where the conditions around the 100 meters are already well defined, with training data the assumptions are often not defined well, or even at all.

So, the first step is to start defining those assumptions. For example, is our strawberry picker assumed to be on a commercial strawberry field? Then, we get more specific. For example, perhaps we assume the camera will be mounted to a drone, or a ground-based robot, or the system will be used during the day, etc.

Assumptions occur in any system; however, they take on a different context with training data because of the inherent randomness involved.

The runner training for the 100 meters intuitively knows that the training for the 100 meters is different than for the 400 meters, 800 meters, or high jump. Getting a similar intuition around AI training is part of the challenge. This is especially hard because we typically think of computer systems as being deterministic, meaning that if I run the same operation twice, I will get the same result. The way AI models get trained, however, is not completely deterministic, and the world in which AIs operate is not deterministic. The processes around the creation of training data involve humans, and are not deterministic either.

Most training procedures have a variety of effects designed for randomness. For example, the initial model weights are usually random. Even if those weights are saved such that they can be reproduced, initially they were random. If all of the training parameters, such as data, model architecture, random seed value, training procedure, etc., are exactly recorded, it is generally possible to reproduce a similar model; however, that original model was still random.

Therefore, at the heart of training data is an *inherent randomness*. Much of the work with training data, especially around the abstractions, is defining what is and is not possible in the system. Essentially, we are trying to rein in and control the randomness into more reasonable bounds. We create training data to cover the expected cases, what we expect to happen. And as we will cover in more depth later, we primarily use rapid retraining of the data to handle the expected randomness of the world.

To end this section on a practical note, I'll list some common assumptions that need to be surfaced and reexamined when working with training data:

- Where, when, and how fresh data will be collected
- What the availability of retraining or updates to the schema will be
- What the "real" performance expectations will be, e.g., what is nice to have versus mission critical in the schema
- All of the annotation context, including guides, offline training sessions, annotator backgrounds, etc., that went into the literal annotation

Use definitions and processes to protect against assumptions

Defining a process is one of the most fundamental ways to set up guardrails around randomness. Even the most basic supervision programs require some form of process. This defines where the data is, who is responsible for what, the status of tasks, etc.

Quality assurance in this context is highly fragmented, with many competing approaches and opinions. We will discuss multiple opinions in the book. Generally, we will go from manual techniques toward more automated, "self-healing," multi-stage pipelines. An example process is depicted in Figure 6-3.

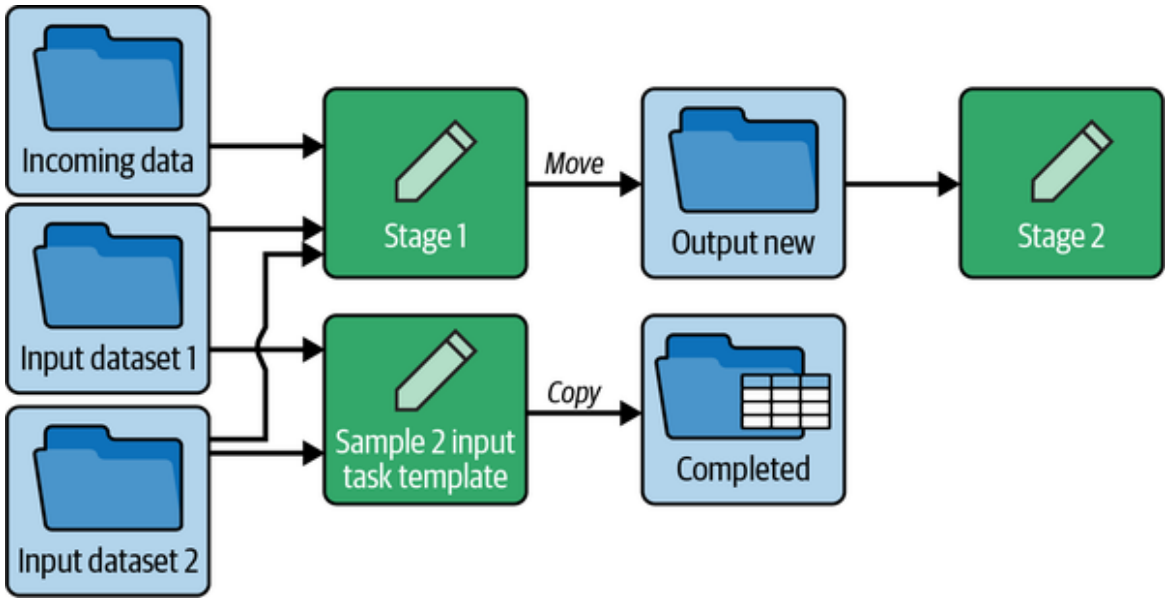


Figure 6-3. Visual graph of workflow in annotation process

As soon as you understand the basics of training data, you will quickly realize there is an obvious bottleneck: doing the literal annotation. There have been many approaches to speed up this core part of the process. We will explore the processes and trade-offs. Choices here can quickly become very complex. It's one of the most often misunderstood, but also one of the most important, parts of training data.

Human Supervision Is Different from Classic Datasets

"Supervised, semi-supervised, unsupervised"—these terms often come up in data science work. Any human intervention is some form of supervision, be it structuring the data, choosing features, or handcrafting a loss function. When I refer to "human supervised," I mean humans supervising the data, regardless of the technical approach used to model the data afterwards.[2] The general focus of this book is on the current understanding of supervised training data: the automation of what we already know, not discovery.[3]

Human-supervised training data is different from classic datasets. It's new. It has different goals, involves different skill sets, and uses different algorithmic approaches.

Discovery versus automation

In typical classical machine learning cases, we usually don't know what the answer is and we wish to *discover* it. In the new human-supervised case, we already know what's correct, and we wish to structure this understanding (produce training data) in a data science–consumable way. Of course, this is painting with a wide brush, and is not *always* true, but I use it to frame a contrast for an average case.

For example, in the classical case, we don't know what movie preferences someone has, so we wish to discover them, or we don't know what is causing a weather pattern and we want to discover it.

In a similar line of thinking, deep learning concepts are often treated as distinct concepts within machine learning. Often, human–supervised training data goes hand in hand with deep learning approaches because with deep learning there is an expectation of automatic feature extraction.[4] However, there is no strict requirement to use a deep learning approach.

As an example, a common relationship between the three concepts is shown in Table 6-1.

Table 6-1. Relation between goal, training data method, and algorithmic choice

Goal	Algorithmic approach example	Training data
Discovery	Machine learning	Classical
Automation	Deep learning	New—human-supervised

Discovery and automation goals are the two major complementary, and sometimes intersecting, camps within training data.

Consider the diagram in Figure 6-4. On the left is a spreadsheet, and on the right, a view of the world from a car. In the classic case, which is the spreadsheet, the data is already fixed. There is little to label. In contrast, in

the new case, the raw data doesn’t mean anything by itself. Humans must add labels to control the meaning. This means there are more degrees of freedom and more capabilities. In other words, in the classic context there is only indirect human control, whereas in the new context there is direct human control.

Owner	Country	File_date	IPC_class
Company A	US	6/18/2008	H05H13
Company A	EP	1/30/1998	A61N5
Company A	EP	1/30/1998	A61N5
Company A	EP	1/30/1998	A61N5
Company A	JP	8/28/1997	A61N5
Company A	JP	10/4/2002	A61N5

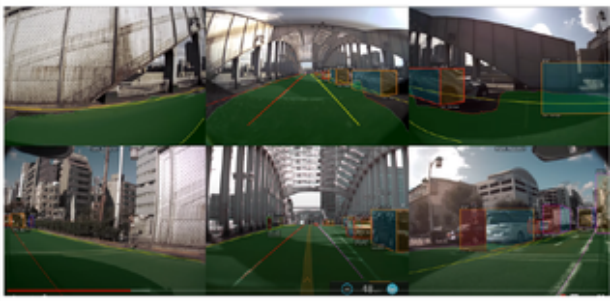


Figure 6-4. Comparison of tabular data taken (discovery) as is versus complex human-supervised data

This is not to diminish the continued importance of classic datasets or discovery-focused use cases. Also, to be clear, image analysis is not new in and of itself, and conceptually many theories are not new either, but the way that the data is being structured and the assumptions around the mainstream industry approaches are substantially different now.

Human-supervised training data and classic datasets are different approaches covering different problems. We shouldn’t oversimplify them as being all the same—a spreadsheet is materially different from the assumptions and structures of data discussed in this book. We also shouldn’t contrast them as competing; supervised training data is unlikely to help in discovery-focused use cases, so both approaches are still needed.

Discovery

Datasets classically have been about the discovery of new insights and useful heuristics. The starting point is often text, tabular, and time series data. This data is usually used as a form of discovery, such as for recommender systems (Netflix’s suggested movies), anomaly detection, or car reconditioning costs.

In classic dataset techniques, there is no form of human “supervision.” Humans may be involved in defining preprocessing steps, but they are not actively creating new data to the same degree as in annotation. Humans may also still be involved in feature engineering, or in a deep learning context (but still classic discovery focused) there may not even be feature engineering. Overall, the data is fixed, a mathematical algorithmic goal is defined, and the algorithm goes to work.

Feature engineering is a practice of selecting specific columns (or slices) that are more relevant, for example the odometer column for a vehicle reconditioning cost predictor.

With the new training data techniques, humans are supervising the data. We already know what the correct answer is, and the goal is to essentially copy that understanding, “Monkey See, Monkey Do.”

This direct control makes the new techniques applicable to a variety of new use cases, especially in the realm of “dense” or “unstructured” data, such as video, images, and audio. Instead of the data being fixed, we can even control the generation of new data, such as taking new images.

General Concepts

When you are creating, updating, and maintaining training data, there are a number of concepts that come into play. While we cover many specifics earlier in the book, I want to take a moment to reflect on more general concepts that carry throughout multiple areas of training data:

- Data relevancy
- The need for both qualitative and quantitative evaluations
- Iteration
- Transfer learning
- Bias

- Metadata

Data Relevancy

Continuing with the theme of validation of existing data, how do we know if our data is actually relevant to the real world? For example, we could get a perfect 100% score on a test set, but if the test set itself isn't relevant to the real-world data, then we are in trouble!

There is currently no known direct test that can define if the data is guaranteed to be relevant to the production data—only time will truly tell. This is similar to the way a traditional system can be loaded and tested for *x* number of users, but the true usage pattern will always be different. Of course, we can do our best to plan for the expected use and design the system accordingly. Like in classic systems, tests and other tools can help, but you still need a good design in the first place.

Overall system design

There are usually many choices for how to design a data collection system in machine learning. For example, for a grocery store, a system could be positioned over an existing checkout register, such as to prevent theft, or aid in faster checkout, as shown in Figure 6-5.

Or, a system could be designed to replace the checkout entirely, such as placing many cameras throughout and tracking shopper actions. There is no right or wrong answer here—the primary thing to be aware of is that the training data is tied to the context it's created in. In later sections we will cover system design in much more depth.

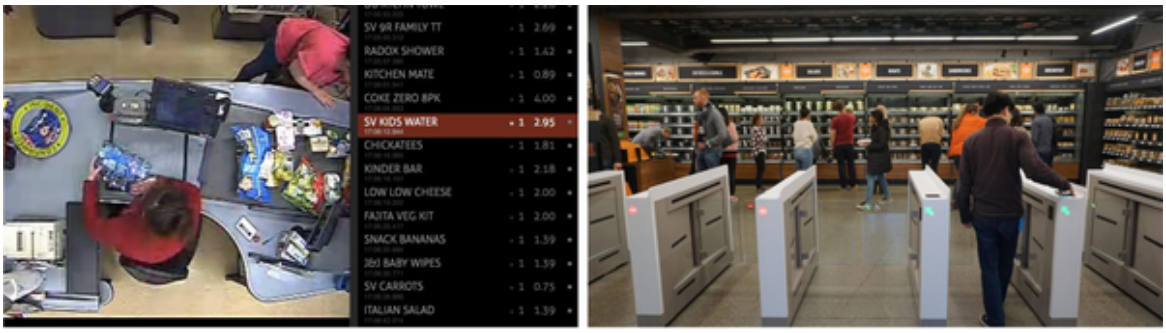


Figure 6-5. Left, example of integration with an existing system; right, example of new process

Raw data collection

Beyond the high-level system design perspective, the collection and storage of raw data is generally beyond the scope of this book. This is, in part, because of the vast array of options available for raw data. Raw data can come from real-life sensors (video, audio, images), or it can be screenshots, PDF scans, etc. Virtually anything that can be represented in a human-parsable form can be used for raw data.

Need for Both Qualitative and Quantitative Evaluations

Both qualitative and quantitative types of evaluation are important. You'll want to apply the appropriate weight to quantitative methods. Quantitative metrics are tools, not overall solutions, and can't provide the comfort of knowing if the system is actually going to work. Often, these tools are "overweighted," and looked at as the oracle of system success, so you may need to decrease the weight of how significant these metrics are when viewed by you and your team.

Metrics-oriented tools also place more emphasis back on the data science team and further away from the experts who can actually look at the results one sample at a time—qualitatively, not just statistically. Most real-world systems don't work as well as is often believed by those who are overly guided by metrics. Metrics are needed and important. Just keep separate the also important, and always needed, human qualitative ability.

Iterations

In traditional programming, we iterate on the design of functions, features, and systems.

In training data, there is a similar form of iteration practiced using all of the concepts we are discussing herein. The models themselves are also iterative; for example, they may be retrained at a predetermined frequency, such as daily.

Two of the biggest competing approaches here are the “fire and forget” and the “continual retrain.” In some cases, it may be impractical to retrain, and so a single, final model is created.

Prioritization: What to Label

As part of dataset construction, we know we need to create a smaller set from a larger set of raw data—but how? This is the concern of “What to Label.” Generally, the goal of these approaches is to find “interesting” data. For example, if I have thousands of images that are similar, but three are obviously different, I may get the most bang for my buck if I start with the three most different ones. This is because those outliers will benefit the most from human judgments.

Transfer Learning’s Relation to Datasets (Fine-Tuning)

As a brief primer, the idea of transfer learning is to start off from an existing knowledge base before training a new model. Transfer learning is used to speed up training new models. While the specifics of the implementation details of transfer learning are outside the scope of this book, I wish to draw attention to how some of the training data and dataset choices relate to and impact fine-tuning.

You may have heard that popular datasets used in transfer learning or pretrained models contain biases that are hard to remove. There are theories about the effects of those datasets on the researcher, including that the biases leak into the design of the models. This is often true to a degree, but is usually not particularly relevant to real-world supervised AI applications.

To get a better intuition, let’s consider the context of pre-training for image detection tasks. A large portion of the pre-trained knowledge is things like detecting edges, basic shapes, or sometimes common objects. In this context these base level features are more an artifact of the training process than true “training data biases.” In other words, they are features whose inclusion is either routine or their presence is not negative in a meaningful way.

A critical distinction here, relative to the earlier discussed concerns around historical datasets, is the original dataset is just a starting point, from which the real dataset will be fitted. In other words, if there’s a problem in the real dataset, then yes the model will still be trying to learn that. But if there’s a problem in the original set not present in the real set, the chances of it remaining during the fine-tuning or transfer learning processes are much lower.

To test this for your specific case, train both with transfer learning and without, and compare. Usually, these comparisons show that transfer learning is faster, but the actual end result, the end state of the model after the training process, is similar. You can do this as a check, and then for ongoing iterations you will know if you can likely use transfer learning “safely” with less bias concern.

The concern can be stated like this: “Because we are indirectly using training data from that prior model training, if there was undesirable bias in that model, it may carry over to our new case. There is a dependency on that prior training data set.” However, we must remember the intent of transfer learning: a way to reach an optimal point faster. In practice, this means it’s usually about getting general “lower-level” concepts to exist in the model (think edges, basic shapes, basic parts of speech, etc.). Moreover, most of the things we could consider to be human-level bias are overwritten on the first few training passes when the new labels are used as the source of error “loss.”

As an example, consider that you can train on images with meanings totally different from the original set. For example, a model pre-trained on

Common Objects in Context (COCO) is a large-scale object detection, segmentation, and captioning dataset) has no concept of a product "SKU 6124 Cereal box." A commonly mistaken assumption is that using transfer learning is automatically better than not using transfer learning. Actually, you can achieve just as good results without transfer learning, as with it. For real-world applications, transfer learning is usually more of a compute optimization than a model performance or bias concern.

Technically, a model should be able to converge to the same point regardless of the starting weights from transfer learning. For context, the pre-trained weights represent the learned representation (higher dimensionality) of the original dataset. I used the terms "weights" and "data" interchangeably here to focus on the training data comparison, but in data science practice they are naturally different. There may be cases where there is not enough data to converge or overwrite the original weights completely, and data from the original set remains, e.g., in the form of unchanged pre-trained weights. If the original dataset contains useful data to the new task (data not in the new set) this remaining data may even be desired. The joint dataset (original plus starting weights) may now still get better performance than either set individually. Having a true joint dataset is less likely with larger new datasets, in which case more or all of the transferred set's weights get "overwritten." It also can depend on the context of the pre-trained set. If the intent was to make use of semantic information, that is a different use case than using it for "base" concept detection like edges, as mentioned earlier.

While more depth in that topic is beyond the scope of this book, essentially, as long as you are using transfer learning as an optimization (e.g., to save time training) and still have your own novel data over top of it, then it is unlikely that bias from the original set will be a material issue. Potential issues become even less noticeable as the ratio of the new dataset grows relative to the pre-trained set. On the other side of the coin, if for your case, the original dataset (through transfer learning) is remaining or affecting the final distribution of the data dramatically, then you should treat the data of

the pre-trained model as part of your overall current dataset and inspect it closely for undesirable bias.

So to recap, there are two things to remember about transfer learning:

- It is not a replacement for having your own domain-specific (new) datasets.
- It is usually a compute or algorithmic optimization more than bias concern. However, if substantial parts of original dataset or weights are kept, and the weights are beyond basic internal concepts like edges and shapes, then it is a bias concern.[5]

Again, to be clear, if there is unwanted bias in the training set, and that set is directly used, that is bad. If a transfer learning set is used just to speed up the initial weighting setting of your dataset, then the risk is a lot lower. For any cases where you suspect bias to be an issue, train with and without the transfer learning set to compare and inspect the original set for undesirable bias.

This is an evolving and controversial area, and the above is meant only to provide a high-level overview as relevant to training data. Please note this is all in the supervised (human reviewing the data) context.

Per-Sample Judgment Calls

Ultimately, a human will supervise each sample, generally one sample at a time. In all of this, we must not forget that the decisions each person makes have a real impact on the final result. There are no easy solutions here when it comes to difficult judgment calls. There are tools available, however, such as taking averages of multiple opinions, or requiring examinations.

Often people, including experts, simply have different opinions. To some extent, these unique judgments can be thought of as a new form of intellectual property. Imagine an oven with a camera. A chef who has a signature dish could supervise a training dataset that, in a sense, reflects that chef’s unique taste. This is a light introduction to the concept that the line

between system and user content becomes blurred with training data in a way that's still developing.

Ethical and Privacy Considerations

First, it's worth considering that some forms of supervised data are actually relatively free of bias. For example, it's hard for me to imagine any immediate ethical or privacy concerns from our strawberry picking dataset. There are, however, very real and very serious ethical concerns in certain contexts. This is not an ethics book, though, and there are already some relatively extensive books on the effects of bias in automation more generally. However, I will touch on some of the most immediate and practical concerns.

Bias

The term "bias" is overloaded; it has many many meanings. Here, I will disambiguate just a few; there is technical bias, imbalanced classes bias, desired human bias, and undesirable human bias.

First, even within technical bias, there are multiple meanings for the term. In ML modeling, bias can mean the fixed value that is added to the variable part of a calculation. For example, if you wanted the model to return 3 when the sum of the weights is 0, you can add a bias of 3.[6] It can also mean the estimator bias as measured by distance from the mean of the real distribution (which, for real-world applications, is never fully known). These measures may be useful to data scientists and researchers, but it's not the bias that we are concerned about, which is human bias.

Imbalanced classes are when one label has more samples than another. This is easy to state, but how to fix it, in practice, is undefined. To understand why, consider the following example: imagine that we are designing a threat detection system for an airport scanner, as shown in Figure 6-6.

We may have classes like "forearm" and "threat." How many forearm examples do we need? How many threat examples do we need? Well, the

forearm, in this context, has a very low variability, meaning that we likely only need a small sample set to build a great model. However, the possible variations in threat placement, and the purposeful efforts to obscure it, mean we likely need many more examples.

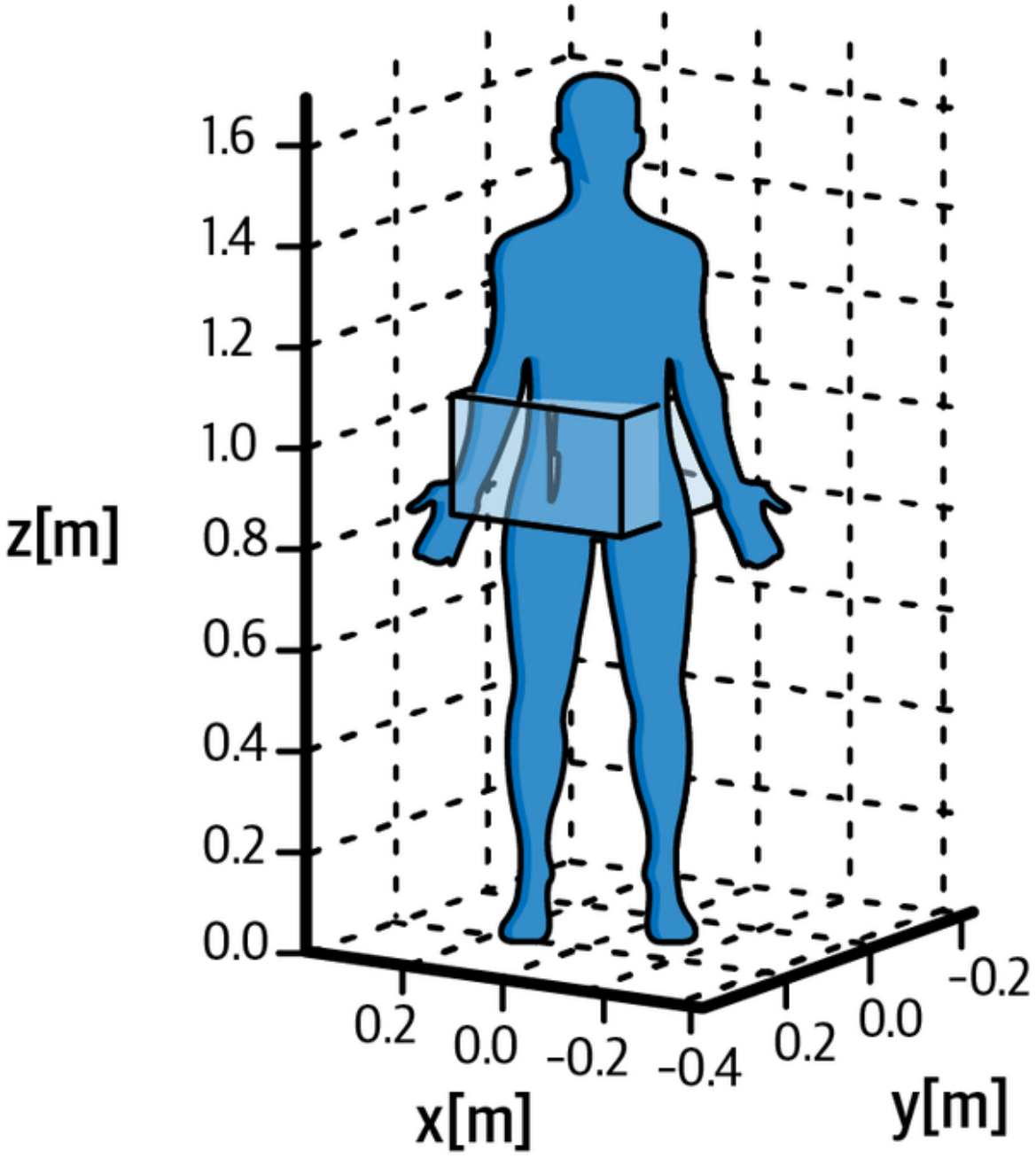


Figure 6-6. Example of raw scan from 3D millimeter wave scanner

At first glance, in that context, then, “forearm” may be out of balance to “threat,” but it’s actually desirable, because “threat” as a concept is a harder

problem than “forearm.” Maybe it needs 10× as much data, or maybe 100×—the absolute count of samples doesn’t matter as much as how much data is needed to be performant *on that class.* This is the imbalanced classes problem. It may take 10× as much data for label A to be “balanced” with label B. Another way to approach this is to subdivide “threat” into smaller categories.

This leads us to a slightly more subtle problem. We have been assuming, in our example, that all instances of a “threat” are equal. But what if the training data isn’t representative of the real-life data? This is technically a different concept, the data distribution.

The thing is that correcting “obvious” bias has a variety of technical solutions, such as hard negative mining, but correcting the relationship of the data to the real world does not. This is similar to how, for example, a classic program can pass hundreds of “unit tests,” but still utterly fail to meet the end user’s needs.

The model needs to be “biased” toward detecting what you want it to detect. So, keep in mind that from that perspective, you are trying to make the model “biased” toward understanding your worldview of the data.

Bias Is Hard to Escape

Imagine this scenario:

- A dataset is created in a given month
- To maintain freshness, only “new” data from the last six months is used
- To optimize the model, a sampling of the output and errors is reviewed and corrected by humans

What this really means though, is that every “new” example is recycled into the model. In other words, say that a prediction and subsequent correction happens on day one. How long can we use this? We presume this “fresh” correction is valid for six months. But is it really?

Well, the reality is that even if it's "correct," after six months, its basis is a model that is now old. This means that even if we retrain the model using only data corrected within the last six months, there is still bias from the "old" model creeping in.

This is an incredibly hard problem to model. I'm not aware of a scientific solution here. This is more of a process thing to be aware of—that decisions made today can be difficult to completely roll back tomorrow.

An analogy in coding may be system architecture. A function may be relatively easy to "correct," but it's harder to realize if that function should exist at all. As a practical matter, an engineer working to correct an existing function will likely start with that function, so even the corrected function will contain the "spirit" of the old one, even if literally every character is changed.

Besides the literal data and instances, a further example here is the label templates. If the assumption is to always use the existing predictions, it may be hard to recognize if the templates are actually relevant any more.

Metadata

Imagine you spent thousands of hours (and potentially hundreds of thousands of dollars) to create multiple datasets, only to realize that it wasn't clear what assumptions were present when it was created. The implication of this is the data could be less useful than previously expected, entirely useless, or even worse, it could have excess negative effects due to unexpected results when used outside the expected context.

There are many reasons, some described following, that a dataset that is technically complete can become largely unusable. A surprisingly common problem is losing information on how a set was constructed. Imagine, for example, that a data supervisor has a question about a project and sends it over a channel like email or chat. The problem, such as "how do we handle a case when such and such happens?" is resolved and life goes on. However, as time progresses that knowledge is often lost. If that issue arises again, the process of querying the colleague and waiting for a response has

to happen again or in some cases the raw records or colleague may not be available. Therefore it's best to maintain as much metadata as reasonably possible for your context, e.g., by including it in definitions in your system of record.

METADATA OF A DATASET

Data about the dataset that is not directly used by the model. For example, when the set was created, who created it. Don't call annotations "metadata," because annotations are a primary component of the overall training data structure. Consider that if we called annotations metadata, then metadata becomes meta-metadata which doesn't make sense.

A small note is that some people refer to any type of annotation as metadata. If we take an old textbook definition, this is reasonable. In practice, annotations and schemas are thought of as separate things from generic metadata.

Preventing Lost Metadata

There are many examples of metadata that is commonly "lost" from sets:

- What was the original purpose of the set?
- Who were the humans who created it?
- What was the context in which the humans created it?
- When was the data captured? When was it supervised?
- What were the sensor types, and other data specifications?
- Was the element made by machine originally, or by a human? What assist methods, if any, were used?
- Was the element reviewed by multiple humans?

- What other options (e.g., per attribute group) were presented?
- Was the templating schema changed during set construction? If so, when?
- How representative is the set versus the “original” data?
- When was the set constructed? For example, the raw data may have a timestamp, but that is likely to be different from when a human looked at it (and this is likely to be different *per sample*).
- What guides were shown to supervisors? Were those guides modified, and if so, when?
- What labels are in this set and what are the distributions of the labels?
- What is the schema of relating the raw data to the annotations? For example, sometimes annotations are stored at rest as a filename like *00001.png*. But that assumes that it’s in the folder “xyz.” So if for some reason that changes, or it isn’t recorded somewhere, it can be unclear which annotations belong to which samples.
- Is this only “completed” data? Does the absence of an annotation here mean the concept doesn’t exist?

Metadata loss can be avoided by capturing as much information as reasonably possible during the creation process. In general, using a professional annotation software will help with this.

Train/Val/Test Is the Cherry on Top

When creating a model, a common practice is to split an original set into three subsets; this is called *Train/Val/Test*. The idea is to have a set that the model is trained on, a second set that’s withheld from training to validate on, and a third set that’s held in reserve until work is complete, to do a one-time final test on. It’s relatively trivial to sample an existing set and split it.

However, where did the original “set” come from?! This is the concern of the training data context: constructing the original, updated, ongoing sets

themselves.

There are a variety of considerations around splitting these subsets, such as randomly selecting samples to avoid bias, but that's more the concern of data science than training data.

Usually, there is more raw data than can be annotated. So, part of the basic selection process is choosing which of the raw samples will be annotated. There are normally multiple original sets. In fact, many projects quickly grow to have hundreds of these original sets. More generally, this is also a concern of the overall organization and structure of the datasets, including selecting which samples are to be included in which sets.

Sample Creation

Let's explore, from the ground up, how to create a single sample of training data. This will help build understanding of the core mechanics of what the literal supervision looks like.

The system will have a deep learning model that has been trained on a set of training data. This training data consists of two main components:

- Raw images (or video)
- Labels

Here, we will discuss a few different approaches.

Simple Schema for a Strawberry Picking System

Let's imagine we are working on a strawberry picking system. We need to know what a strawberry is, where it is, and how ripe it is. Let's introduce a few new terms to help us work more efficiently:

Label

> A label, also called a *class*,[7] represents the highest level of meaning. For example, a label can be "strawberry" or "leaf."

For those technical folks, you can think of it as a table in a database. This is usually attached to a specific *annotation* (instance).[8]

Annotation (instance)

A single example is shown in Figure 6-7. It is connected to a label to define *what* it is. Labels also usually contain positional or spatial information defining *where* something is. Continuing with the technical example, this is like a row in a database.

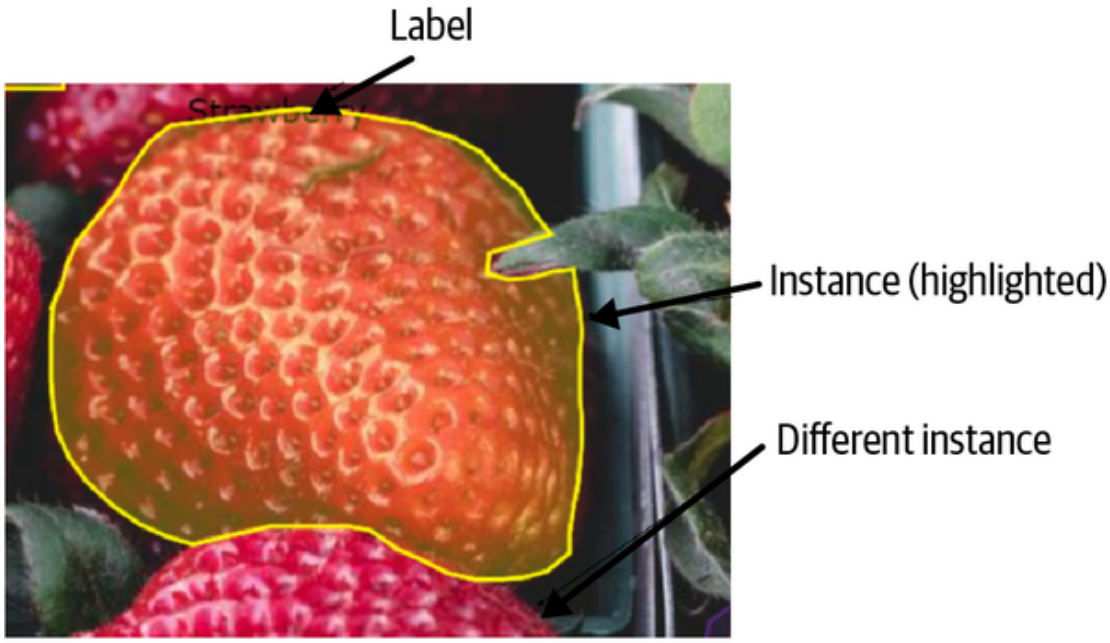


Figure 6-7. Labeled and not labeled instances

Attributes

An instance may have many attributes. Attributes are things that are unique to a specific instance. Usually, attributes represent selectable properties of the object itself, not its spatial location.

This choice will affect the speed of supervision. A single instance may have many unique attributes; for example, in addition to ripeness, there may be disease identification, produce quality grading, etc. Figure 6-8 depicts an example.

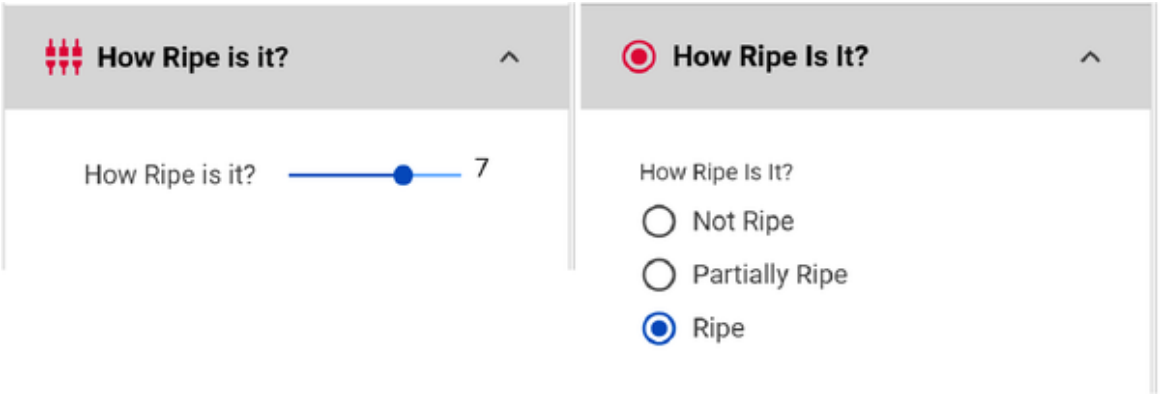


Figure 6-8. Example of UI showing choices for attributes

Imagine you only want the system to pick strawberries of a certain ripeness. You may represent the ripeness as a slider, or you may also have a multiple choice option about ripeness, as shown in Figure 6-8. From the database perspective, this is somewhat akin to a column.

Geometric Representations

You'll also make choices about what type of geometric representation to use. Your choices here are as much part of training data as doing the literal supervision.

Geometric shapes can be used to represent objects. For example, we can represent the strawberries shown in Figure 6-9 as boxes, polygons, and other options. In other chapters, I discuss these trade-offs.

From a system design perspective, there can be choices around what type of raw data to use, such as image, audio, text, or video. Sometimes, you may even combine multiple modalities together.



Figure 6-9. Annotation example with box and polygon geometric shapes

Angle, size, and other attributes may also apply here.

Spatial locations are often singular in nature. Now, while an object is usually only in one place at a given moment in time,[9] in some cases, a single frame may have multiple spatial annotations. Imagine a beam obstructing the view of a car: to accurately label the area of the car, we may need to use two or more closed polygons related to the same annotation.

Binary Classification

A basic approach is to detect the difference between something existing and not existing. One way to do that is called binary classification. This is akin to asking “Does the traffic light exist in the photo or not?”

As an example, two of the images in the set may look like Figure 6-10:



Figure 6-10. Left, an image in a dataset showing a traffic light with trees in the background; right, an image in a dataset showing trees

To supervise Example 1, we need only two things:

- To capture the relation to the file itself: e.g., that the filename is “*sensor_front_2020_10_10_01_000*.” This is the “link” to the raw pixels. Eventually, this file will be read, and the values converted into tensors, e.g., position 0,0, and RGB values will be assigned.
- To declare what it is in a meaningful way to us, for example: “Traffic_light” or “1.” This is similar to saying filename “*sensor_front_2020_10_10_01_000*” has a “Traffic_light” existing in it.

And for Example 2, we could declare it as:

- “sensor_front_2020_10_10_01_001”
- “No” or “0”

In practice, there will usually be more sets of images, not just two.

Let’s Manually Create Our First Set

You can do this with a pen and paper or white board. First, draw a big box and give it a title, such as “My First Set.” Then draw a smaller box, and put a sketch of a traffic light and the number 1 inside it. Repeat that two more times, drawing an image without a traffic light and a 0, as shown in Figure 6-11.



Figure 6-11. Visual example of a simple training data set

This is the core mapping idea. It can be done using pen and paper and can also, of course, be done in code. Realistically, we will need proper tools to create production training data, but from a conceptual standpoint, each methodology is equally correct.

For example, consider this Python code. Here, we create a list and populate it with lists where the 0th index is the file path and the 1st index is the ID. The completed result (assuming the literal bytes of the .jpgs are available in the same folder) is a set of training data:[10]

```
Training_Data  = [
 ['tmp/sensor_front_2020_10_10_01_000.jpg',     1],
 ['tmp/sensor...001.jpg',                                0],
 ['tmp/sensor...002.jpg',                                0]]
```

This is missing a label map (what does 0 mean?). We can represent this simple dictionary as:

```
Label_map = {
          1 :  "Traffic light",
          0 :  "No Traffic light"
}
```

Congrats! You have just created a training dataset, from scratch, with no tooling, and minimal effort! With a little wrangling, this would be a completely valid starting point for a basic classification algorithm.

You can also see how, by simply adding more items to the list, you can increase the training dataset.

While “real” sets are typically larger, and typically have more complex annotations, this is a great start!

Let’s unpack the human algorithm we are using. In sequence, we would do the following:

1. Look at picture.
2. Draw on knowledge of traffic lights.

3. Map our knowledge of traffic lights to sparse value, e.g., “Traffic light is present.”

While it’s obvious to us, it’s not so obvious to a computer. For example, prior approaches in this space, such as Histograms of Oriented Gradients (HOGs) and other edge detection mechanisms, have no real understanding of “traffic light” and “not traffic light.”

For the sake of this example, I made a few assumptions. For instance, I imagined the traffic lights were pre-cropped by some process, the angle of the traffic lights was presumed to be correct, etc. In practice, it’s common to use a spatial shape, such as a box or a cuboid so that the location can be determined by the model (e.g., object detection). Also, there may be many other states or pre-processing pipelines.

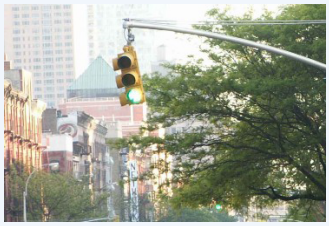
Of course, in production, the processes become more complex, but this example demonstrates the aspect of this process is classification and mapping our understanding to the computer. There are multitudes of more complex forms of doing this, but this is the heart of the idea. If you understand this, you are already well ahead of the game.

Ultimately, other approaches typically build on classification, or add “spatial” properties to samples. At the end of the day, even if some prior process runs, there’s still a classification process being done.[11]

Upgraded Classification

To expand from “it exists” to “what is it” we will need multiple classes. Table 6-2 shows an example of how this is laid out.

Table 6-2. Visual comparison of raw data and corresponding label

Sample #	Raw media	Label name	Integer ID
Sample 1		Red	1
Sample 2		Green	2
Sample 3		None	0

With regard to whether to use strings or integers, generally speaking, most actual training will use integer values. However, those integer values typically are only meaningful to us if attached to some kind of string label:

```
{ 0 : "None",
  1 : "Red",
  2 : "Green"}
```

Here, I will introduce the concept of the label map. While mapping of this type is common to all systems, these label maps can take on additional complexity. It's also worth remembering that in general, the 'label' means nothing to the system, but it's mapping the ID to the raw data. If these

values are wrong, it could cause a critical failure. Worse—imagine if the testing relied on the same map!

That's why, when possible, it's good to "print output"; that way you can visually inspect whether the label matches the desired ID. As a small example, a test case may `assert` on a known ID matching `string`.

Where Is the Traffic Light?

Continuing with the traffic light example, the problem with the preceding approach is that we don't know *where* the traffic light is. There is a common concept within ML modeling called an *objectness score* that we touched on earlier in the book. And there are other, more complex ways to identify location. From a training data perspective, as long as we have a bounding box present we have identified the *where* (the spatial location), the implementation of the algorithm to learn that is up to data science.

Maintenance

Now that we have covered the basics of how a single sample is created, and introduced some key terms, let's zoom back out to a high-level look at the process. In Chapter 2, we covered basics like setting up your training data software, schema, and tasks. But what do ongoing maintenance actions really look like?

Actions

These are actions that can be taken, in general, after some form of information has been returned from the model training process.

Increase schema depth to improve performance

One of the most common approaches to improve performance is to increase the depth of the schema. One example is to divide and conquer the label classes, especially poorly performing classes. Essentially, this is to both identify and improve the specific classes that are the weakest. To borrow

from our earlier example, let's use the label "Traffic Light." When there is mixed performance, it may be unclear which examples are needed to improve performance.

When reviewing the results, we notice that "Green" seems to show up more often in the failure cases. One option is to try to add more green ones to the general traffic light set. A better option is to "split" the class "Traffic Light" into "Red" and "Green." That way we can very clearly see which is performing better. We can repeat this until the desired performance is reached. We do this by again, splitting between large and small as shown in Figure 6-12. There are a few intricacies and approaches to the implementation of this, but they generally revolve around this idea. The key idea is that at each "split," it becomes clear which way to go next in terms of performance.

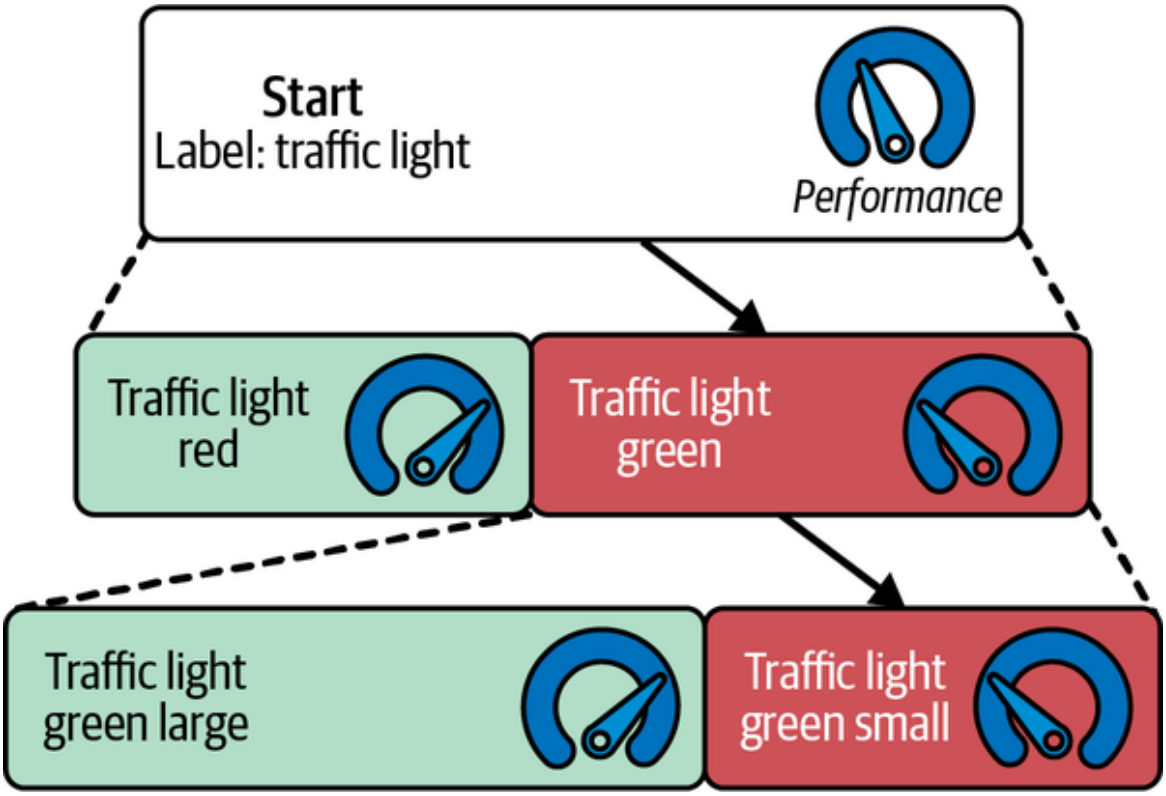


Figure 6-12. Example improvement path of single label expanding into multiple, more specific, attributes based on performance needs

Better align the spatial type to the raw data

Imagine you started with choosing segmentation. Then, you realize the model is not training as desired. You may be able to simply switch to an "easier" task like object detection, or even full image classification.

Alternatively, perhaps object detection is yielding a bunch of overlapping boxes which aren't helpful, and you need to switch to segmentation to accurately capture the meaning, as shown in Figure 6-13.

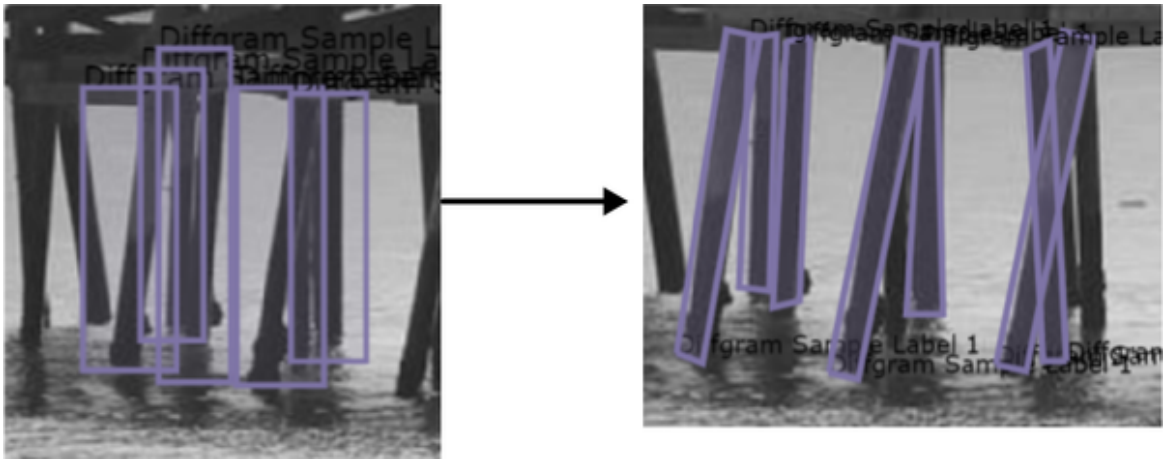


Figure 6-13. Boxes failing to provide useful info; example of moving to segmentation to get better spatial results

In the left side of the diagram, we start with boxes, which results in the overlapping detections, as shown on the left. By moving to segmentation, we can get more clear segments, as shown on the right. While it may appear clear which methods are less ideal for certain cases, the optimal method is often less clear.

Create more tasks

Annotating more data for better performance has almost become a cliche already. This orientation is often combined with the other approaches, for example, dividing the labels or changing the spatial type, and then supervising more. The primary consideration here is whether more annotation will provide net lift.

Change the raw data

In some cases, it’s possible to change the raw data sourcing or collection mechanisms. Examples include the following:

- Change the sensor type or angle.
- Change what part of the screen is being captured.

Net Lift

As we annotate a dataset, we usually want to do two things:

- Identify previously unknown cases or problem cases.
- Maximize the value of each net new annotation.

To illustrate the need for net lift, consider a raw, unlabeled dataset consisting of hearts, circles, and triangles, as shown in Figure 6-14.

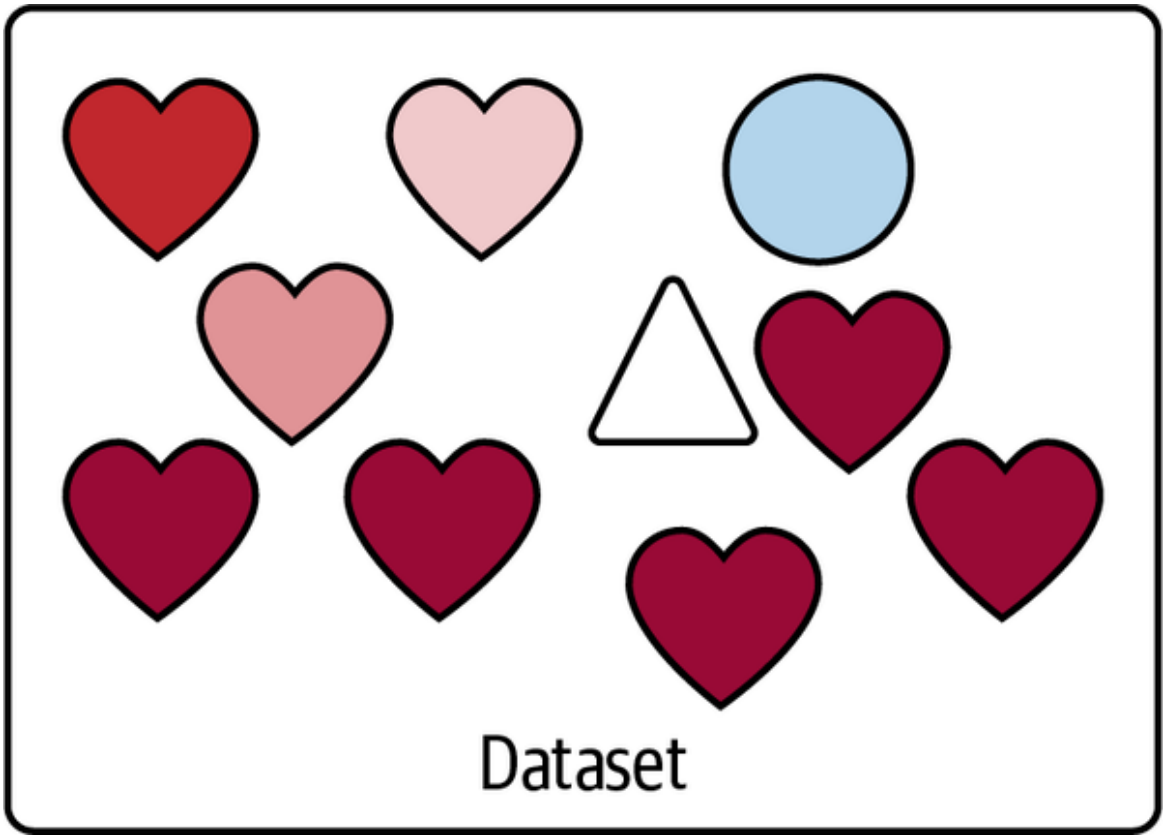


Figure 6-14. Example dataset with abstract shapes to represent data elements

If we annotate only hearts, then each heart we supervise provides minimal value. It's also a problem that we don't understand the complete picture because we did not annotate any circles or triangles. Instead, we should always try to annotate to get true *net lift*, which often means annotating samples different enough to provide value to the model (see Figure 6-15).

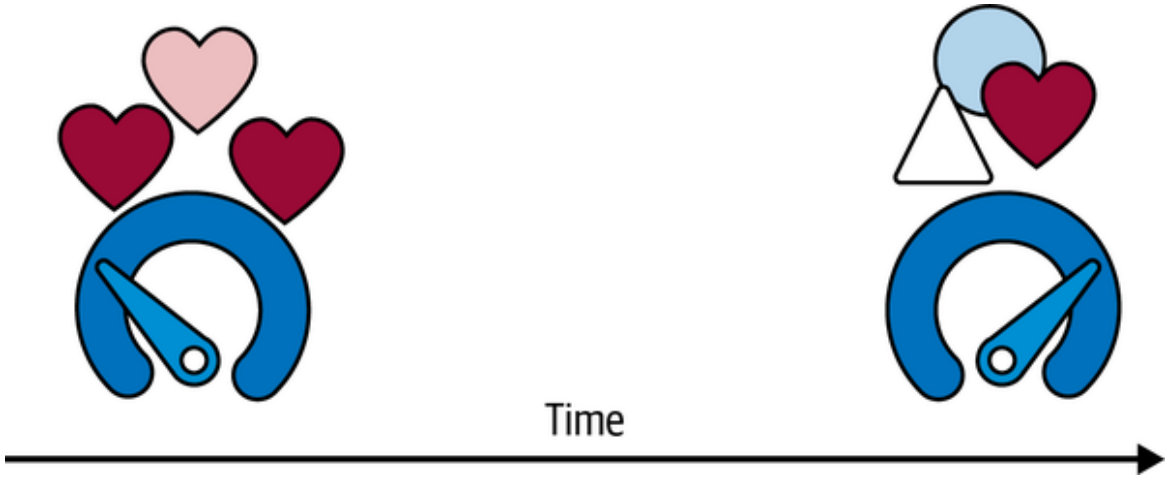


Figure 6-15. Example of better performance over time by annotating different shapes, not just slightly different hearts

There are a few ways we can test if we have seen enough of one sample type:

- Relative performance of that label type is high (as shown in "Labels and Attributes—What Is It?").
- A process like similarity search has identified overly similar data elements (or missing elements).
- Humans have looked at the data and feel it's very similar or know that elements are missing.

Note that a model may need less data for one class than another—sometimes by multiple orders of magnitude. This is completely OK, and achieved in data science via methods like using multiple models or multiple "heads" of the same base model. It is not about having 1:1:1 ratios for every label, but rather labels that will well represent the difficulty of the problem. To extend and reframe the preceding example, if it really matters that we

detect slightly different colors of the hearts, then we may need dramatically more heart examples than triangles or circles.

Levels of System Maturity of Training Data Operations

From early exploring, to proof of concept, through to production, common ways to identify the stage you are in are defined in Figure 6-16.

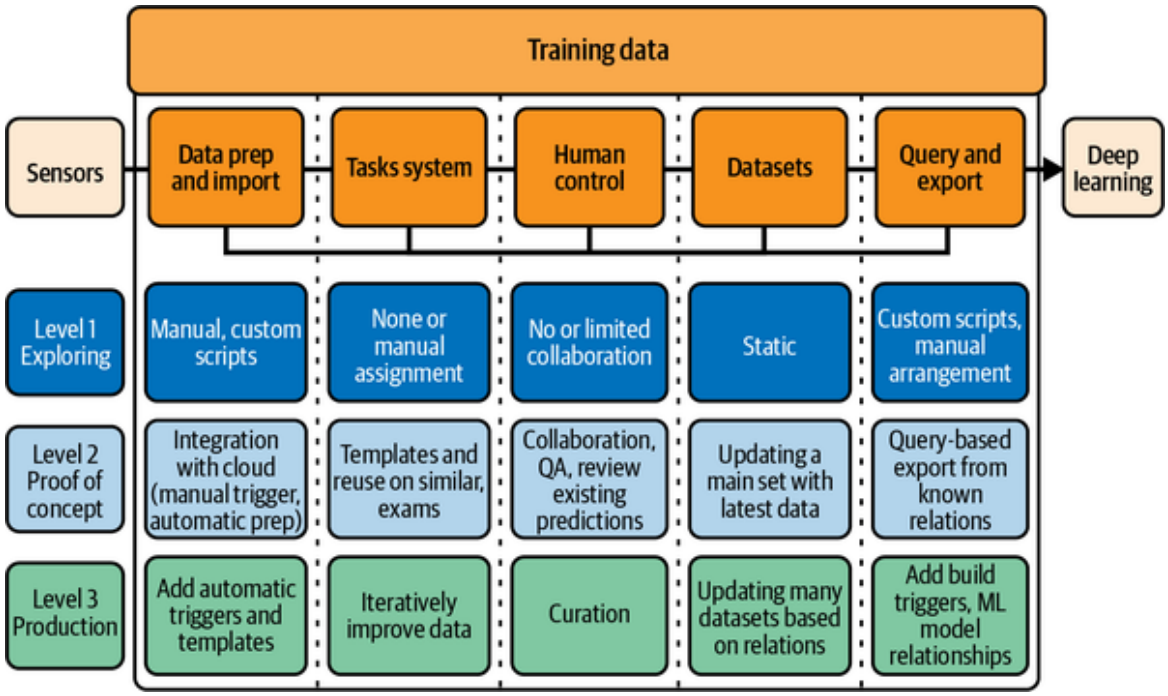


Figure 6-16. Levels of system maturity of training data operations

Applied Versus Research Sets

The needs and form of training data continue to rapidly evolve. When people think of datasets, popular research datasets, like MS Common Objects in Context (COCO) 2014, ImageNet 2009, (both vision) and the General Language Understanding Evaluation (GLUE) 2018[12] come to mind. These sets are designed for research purposes, and by design, they evolve relatively slowly.

In general, it's the research that's designed to change around these sets, not the sets themselves. The sets are used as benchmarks for performance. For example, Figure 6-17 shows how the *same* set is used in five different

years. This is useful for research comparisons. The core assumption is the sets are static. The key point here is the dataset is not changing, in order to compare algorithms; however, this is opposite what often happens in industry, where the algorithms change less and the data changes more.

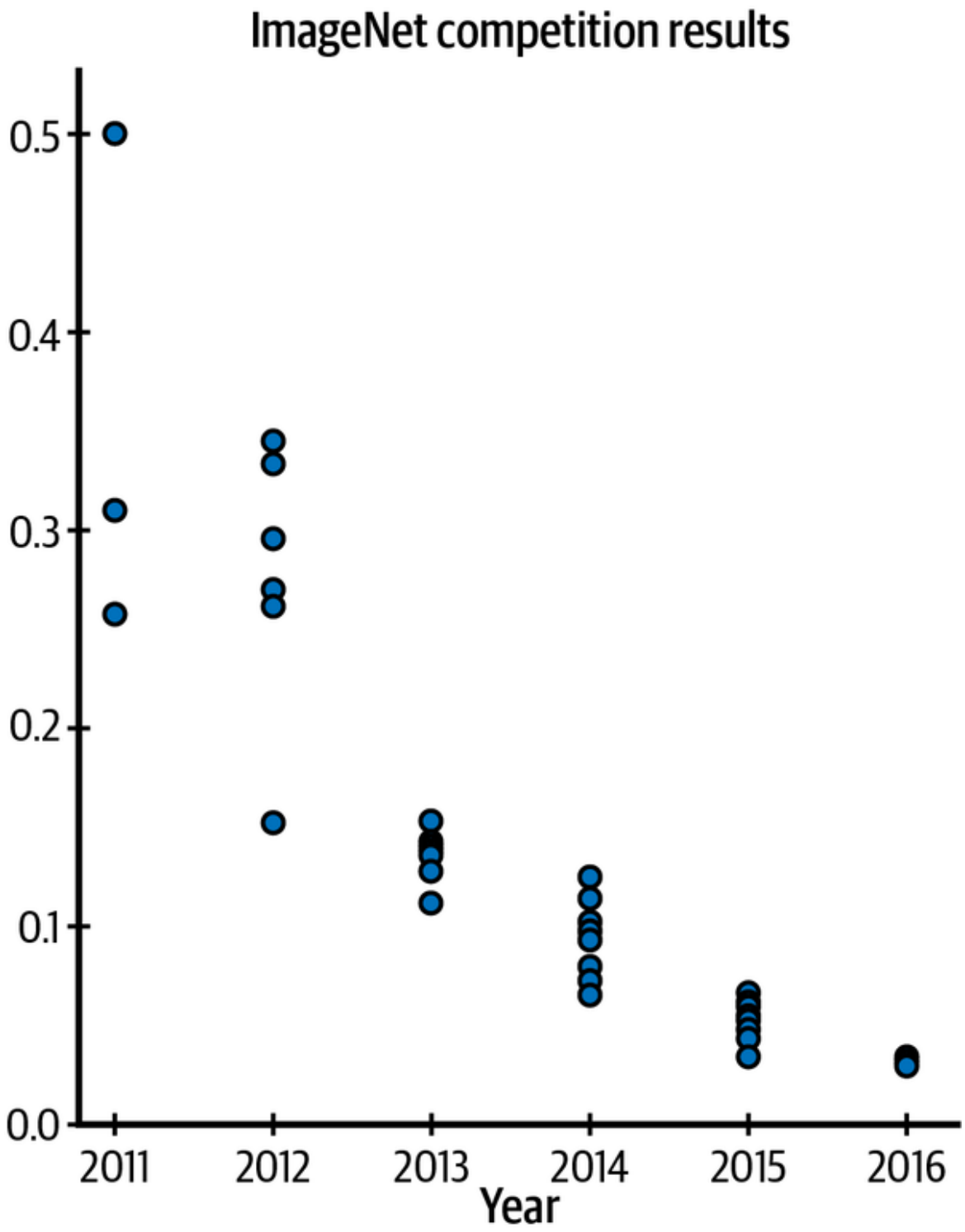


Figure 6-17. Showing the same dataset getting a lower error rate over time due to improvements in studied algorithms (Source: Wikipedia)

In the context of a practical commercial product, things change. In general, the rule of thumb is that data more than 12 months old is unlikely to

accurately represent the current state. Generally, the assumption for practical sets is that they are only static for the moment of literal training, but are otherwise always changing. Further, commercial needs are very different from research. Time available to create the sets, time to update, costs to create the sets, implications of mistakes, etc., are very different in the two contexts.

Training Data Management

We have touched on some of the maintenance concerns of the process. Training data management is concerned with the organization of humans doing the literal tasks and the organization of data throughout the entire operations cycle.

One of the central ideas behind training data management is maintaining the "meta" information around the training data, such as what assumptions were present during the creation of the data. In the context of continually improving and reusing data, for a large organization this is especially important. Imagine spending a significant budget on setting up pipelines, training people, and creating literal sets, only to "lose" that information because of improper handling.

Quality

Quality discussions can escalate quickly into religious-type arguments, with people mystically conjuring statistics, and others rallying hordes of superstitions to their side.

There are a few things to remember during this excitement:

- All datasets contain errors, and yet many ML programs still obtain acceptable and useful results. This shouldn't actually be a surprise, because it's similar to how all programs have some degree of bugs.
- Neither all lines of a code program, or a data program, exist in a vacuum. Each has a variety of cues, visible at various points, that act

as checks and balances. If I see a loading icon stuck, I know something is wrong. As an annotator, if I see that none of the schema matches what I am seeing, I know something is wrong.

- If the margin of error is such that many people need to look at it to get the right answer, the schema is probably wrong.

The annotation quality is itself just one aspect of the overall quality, and it isn't improved just by "adding more" annotations. Sizing the dataset appropriately also impacts the overall quality of the model. This is another reason why using a custom dataset that is custom created to suit your needs is important. If the sample size is too large (or too small), the output will be negatively impacted. Too much extraneous data can skew the results and produce more errors from a statistical perspective, and too little data won't accurately depict the model.

Annotation quality is never fully solved solely by just doing more of it, duplicating samples to more people, etc. This is similar to how "more" education, especially repeating similar lessons, doesn't automatically make you a more effective worker.

Completed Tasks

Knowing which samples are completed at any given moment in time is surprisingly challenging. At a high level, this is partly because we are trusting other humans to identify what is completed and what is not. Second, because the schema often changes, the definition of complete also changes. For example, a file may have been completed relative to a schema that is no longer relevant.

The most minimal management need is to separate "complete" samples from "incomplete" samples. It matters because any sample trained on is considered to be valid to the network. So, for example, if a sample is included with no further labels, it will be considered background.

Why "complete" matters:

This can cause severe problems:

- It makes it hard to create a performant network/debug it.
- If this class of error makes it to a validation test set, the test set will be equally affected.

While this may seem trivial in small set sizes, in most significant sets:

- The data scientist(s) will never observe 100% of the samples, and sometimes will never observe a significant set of the samples, or even *any*! In the case of using transfer learning, e.g., pre-train on ImageNet, anyone who has ever used ImageNet has done this!
- In large set sizes, it is unreasonable to expect any single person to be able to review all of it.

To make an analogy to web design, this is the "test case passed, but important checkout button is invisible to the user and our sales stopped" case for training data.

In general, one should use software to manage this process and ensure that completed/not completed is well tracked. It's also worth considering that in many externally created datasets/exported datasets, this "completed" tracking gets lost, and all of the samples are assumed to be completed. Three common cases showing "completed" status changes are listed here in Table 6-3.

Table 6-3. Tracking of “completed” statuses

Tracking “completed” status context	Reasonable way to track “completed” work status
Human review	Use the “review loop” concept. This means the file status is not complete until the review loop is completed.
Schema changes	Tasks have a schema related to them. If the schema changes, then the “completed” status should be considered questionable.
Multimodal	Use compound files (or tags, groups of tasks) to group multimodal elements together, such that when a compound file is completed, it’s completed as one thing.

Freshness

Certain aspects of training data “age well.” What does age well:

- Transfer learning–type concepts/“lower-level” representations, likely on the order of decades (yet unproven)
- Tightly controlled scenarios (e.g., some types of document reading, medical scanning), likely on the order of years

What doesn’t age as well:

- Novel scenarios /unconstrained real-world scenarios (e.g., self-driving cars)
- Anything that involves a sensor that’s not in tightly controlled space., e.g., anything outside

- Specific “heads/endpoints” labels
- Step function changes in the distribution, e.g., sensor type changes, scenario changes (day/night), etc.

Different applications naturally have different freshness requirements. However, some models that appear to “last a long time” are doing so because they were relatively overbuilt to begin with.

Part of the “trick” with the whole freshness aspect is determining which samples to keep. If possible, it’s good to test and compare a rolling window versus continuous aggregation.

Maintaining Set Metadata

In the rush to create training data, we often lose sight of key contextual information. This can make it very difficult to go back, especially as one develops many different sets. A training data set without context to a real distribution, i.e., the real world, is of little value.

Task Management

The reality is that with humans involved, there is inevitably some form of organization or “task” management system needed—even if it’s “hidden” behind the scenes. Often, this ends up involving some form of people-management skills, which although important, is beyond the scope of this book. When we talk about tasks, we will generally remain focused on training data–specific concerns, such as tooling and common performance metrics.

Summary

Creating useful training data for machine learning systems requires careful planning, iterative refinement, and thoughtful management. We explored theories around how training data controls system behavior by encoding

human knowledge as the ground truth goals for machine learning algorithms:

Theories

> A system is only as useful as its schema. Who supervises the data matters. Intentionally chosen data works best. Training data is like code. Focus on automation over discovery. Human supervision differs from classic datasets.

Concepts

> Data relevancy, iterations, transfer learning, bias, metadata.

Sample creation

> Defining schemas, labels, annotations, attributes, and spatial types. Binary and multi-class classification examples.

Maintenance

> Changing labels, attributes, spatial types, raw data. Considering net lift from new annotations. Planning system maturity levels.

Management

> Tracking quality, task completion, freshness, metadata, and evolving schema.

Key takeaways:

- Alignment between schema, raw data, and annotations is critical.
- Managing the training data lifecycle complexity is key, especially for large, real-world systems.

- Training data controls system behavior by defining ground truth goals. Like code, it encodes human knowledge for machines.

You learned about how to increase schema depth to improve performance, and other actions like better aligning spatial types. We went over maintenance concepts like focusing on the net lift of each new annotation.

I introduced the major training data parts of MLOps and a path for planning system maturity. I introduced the concept of training data management: the organization of training data and people.

---

[1] Assembly: "Any low-level programming language with a very strong correspondence between the instructions in the language and the architecture's machine code instructions." "Assembly Language", *Wikipedia*, accessed on September 14, 2023.

[2] For the technical reader, supervised learning is often thought of as the algorithm having access to the answer key (e.g., { input : output } pairs). While this is a valid definition, the focus here is on the human supervision aspect.

[3] Training data can be used in combination with other approaches to discover new concepts in certain circumstances, but in general the bulk of the focus is on repeating existing known work.

[4] For the technical reader, as we know, deep learning is usually viewed as a subset of machine learning. I will sometimes refer to machine learning (or even AI) as a catch-all term to encompass things relevant to either approach. The intent of this is to maintain the focus on the training data art—how the data is modeled is up to data science.

[5] Note that theoretically, it can always carry over unwanted bias. But in practice it's often not that relevant, since the most "important" weights are often overwritten by the new dataset.

[6] Example inspired by article by Farhad Malik, "Neural Networks Bias and Weights: Understanding the Two Most Important Components", *FinTechExplained—Medium*, May 18, 2019.

[7] Other names include: label template, annotation name, class.

[8] Usually by reference ID only.

[9] That is, in one reference frame. Multimodal annotations can be represented as a set of related instances, or spatial locations for a given reference frame, such as camera ID x.

[10] Sharp-eyed readers may notice this becomes a matrix once completed. The matrix shape has no significance here, and it's generally best to think of these as a set. However, Python sets introduce quirks also not relevant here—so I use a list.

[11] Algorithms usually predict a continuous range and then do a function (e.g., softmax) to convert to these category values.

[12] These sets all have taken an incredible amount of work, and some have been substantially updated over time. Nothing in here is meant to take away from their contributions.

OceanofPDF.com

Chapter 7. AI Transformation and Use Cases

Introduction

This is the start of the AI transformation era. The most successful companies will be the ones who embed AI most deeply, who infuse AI broadly throughout the entire org, and push responsibility to use, train, and manage AI down to the lowest capable levels of the firm.

In this chapter I will cover two major topics:

- AI transformation and leadership
- Use case discovery, rubrics, and unlocking your own experts

This chapter also has extensive guidance on how to identify high-impact areas to apply AI. Examples include evaluation rubrics you can use to test your ideas, example questions to ask, and common missteps to avoid. If you want to dive straight into implementation details, the use case section may be most valuable to you. Moving to implementation, it's important to set a talent vision, which is something we'll go over. "The New "Crowd Sourcing": Your Own Experts" will empower you to achieve your use cases in a cost-effective manner by repurposing existing work.

After reading this chapter, you will have a strong grasp of how to:

- Start your AI transformation at your work today.
- Evaluate AI use cases for your business.
- Define training data knowledge needs for yourself and your organization.

- Understand how to adopt modern tools and start aligning your team, peers, and staff with the goals of working with training data.

When you have the mindset, the leader, the use case, and the talent vision, you can use modern training data tools to make it a reality. I'll conclude this chapter by grounding these concepts into tooling. Training data tools have advanced considerably in the past few years, and there's a lot to learn.

The age of AI is here. This chapter will provide you with the mindset, leadership skills, use cases, and talent strategy needed to transform your company and emerge as an AI leader.

AI Transformation

AI transformation is the answer for business leaders who want to transform their company to take advantage of AI first. It can start today with you. So far, we have covered the basics of training data and taken deep dives into specific areas like automation. Now, we zoom out to see the forest. How do you actually get started with training data at your company?

In this chapter, I will share five key steps, depicted in Figure 7-1. We'll start with mindset and leadership, then move on to the concrete problem definition, and conclude with the two key steps to succeeding: annotation talent and training data tools. Please consider the plan merely a starting point to be adapted to your needs.

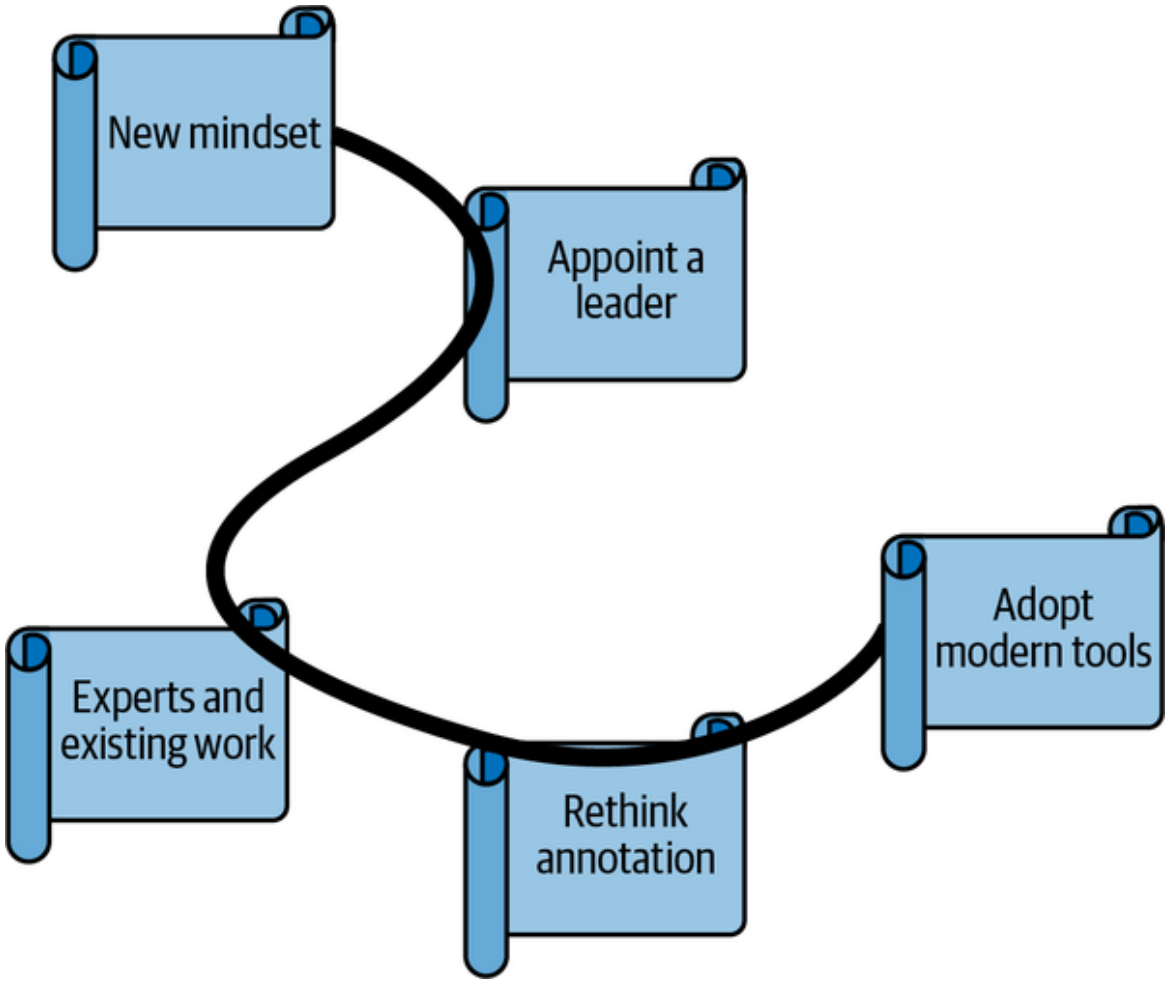


Figure 7-1. AI transformation map

To get started fast at adopting modern AI in your company, here are five key things to act on:

- Create new meaning instead of analyzing history—the creative alignment revolution.
- Appoint an AI data leader to lead the charge.
- Focus on use cases involving experts, using existing high-volume work.
- Rethink annotation talent—your best aligners are already working for you!

- Adopt new training data tools, like AI datastores, UI/UXs, and workflows.

For the last decade or so, the big wins with AI have been with classical cases, as discussed in earlier chapters.

Now the biggest commercial opportunities are with supervised learning, including the human alignment of generative AI systems like large language models. This supervision is about unstructured data that requires annotation (alignment).

Early work was often a case of putting together an "annotation project." This meant throwing data over a fence to some team and hoping for the best. This is like ordering fast food. Sure, it will help with immediate hunger, but it's not a healthy long-term solution.

True AI transformation, like eating healthy, takes work. It's a mindset shift, from seeing annotation as a one-off project, to seeing your day-to-day work as annotation.

Seeing Your Day-to-Day Work as Annotation

To frame this, consider this statement with regard to your company:

> *All of your company's day-to-day work can be thought of as annotation.*

That's right. Every action that the majority of your employees take, every day, is, both literally and figuratively, annotation. The real question here is, how can we shift those actions from being *lost*, every day, to being captured in a way that can be repeated, to the same quality degree, automatically? This mindset shift is reflected in Figure 7-2.

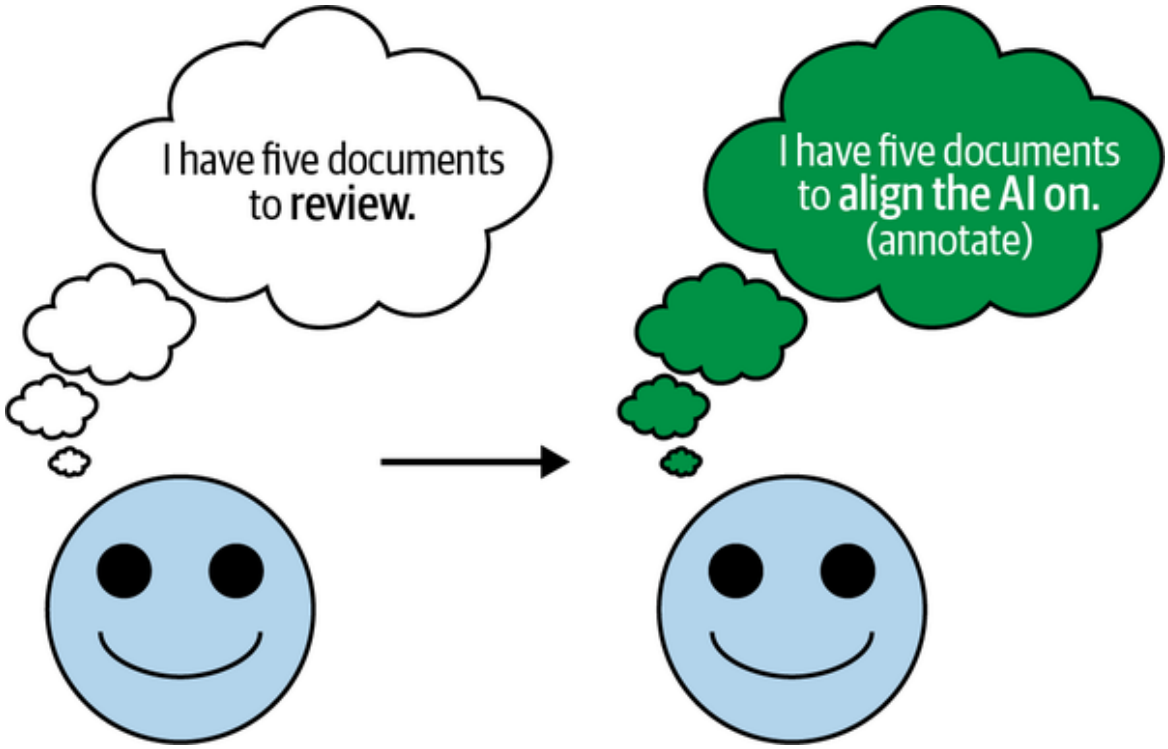


Figure 7-2. Thought process shift from review to annotate (alignment)

To invert this, every moment an employee does something that's not being captured as an annotation, that's productive work lost. The greater the percentage of that work that can be captured through annotation, the greater your productivity is. A further comparison is shown in Table 7-1.

Table 7-1. Comparison of classic working style versus AI training–centric data capture

Before	After
All day-to-day work is “one-offs.”	Day-to-day work is handling the exceptions not yet captured in annotations.
Training is for humans only.	Training is for machines too.
If it’s not in the computer, it doesn’t exist.	If it’s not in training data, it doesn’t exist.

A rough analogy here is that in the movement from analog to digital, we realized that if it didn’t exist in a digital form, then it didn’t “exist” (whether that was true in reality or not). Now in the same way, if it doesn’t exist in data, in this case AI training data, it might as well not exist either, since it won’t help your company improve its productivity.

Let’s put some structure to this theory.

There are two major types of AI transformation:

- Inspiring all relevant aspects of operations to give consideration to AI and establishing new reporting units
- Inspiring a training data–first mindset and reorganizing reporting relations

At a classic company, it’s usually about building large-scale systems and adopting AI tools.

At an AI product company, it’s about keeping up with the latest trends, updates, and AI tooling. Usually, an AI product company is more about

technical infrastructure–level tools, whereas a classic company may use tools produced by said AI companies.

The Creative Revolution of Data-centric AI

Data-centric AI can be thought of as focusing on training data in addition to, and as being even more important than, data science modeling. But this definition does not really do it justice. Instead, consider that data-centric AI is more about creating new data to solve problems.

The creative revolution is a mindset of using human guidance to create new data points to solve problems. Opening up to the huge magnitude of this potential will help prime you and your team to define the best use cases. Next is appointing someone to lead the charge—a Director of Training Data. This position will be new in most companies. Given that a major portion of AI costs is made up by human labor in producing training data, this position naturally needs someone to account for it.

You Can Create New Data

In the data-centric mindset, the critical realization is that you can do one of two things:

- Use or add new data
    - New connections to existing files and data
    - New sensors, new cameras, new ways to capture data, etc.
- Add new human knowledge
    - New alignment and annotation

For example, for a self-driving car case, if you want to detect people getting cut off, you can create the meaning of “Getting Cut Off” as shown in Figure 7-3.

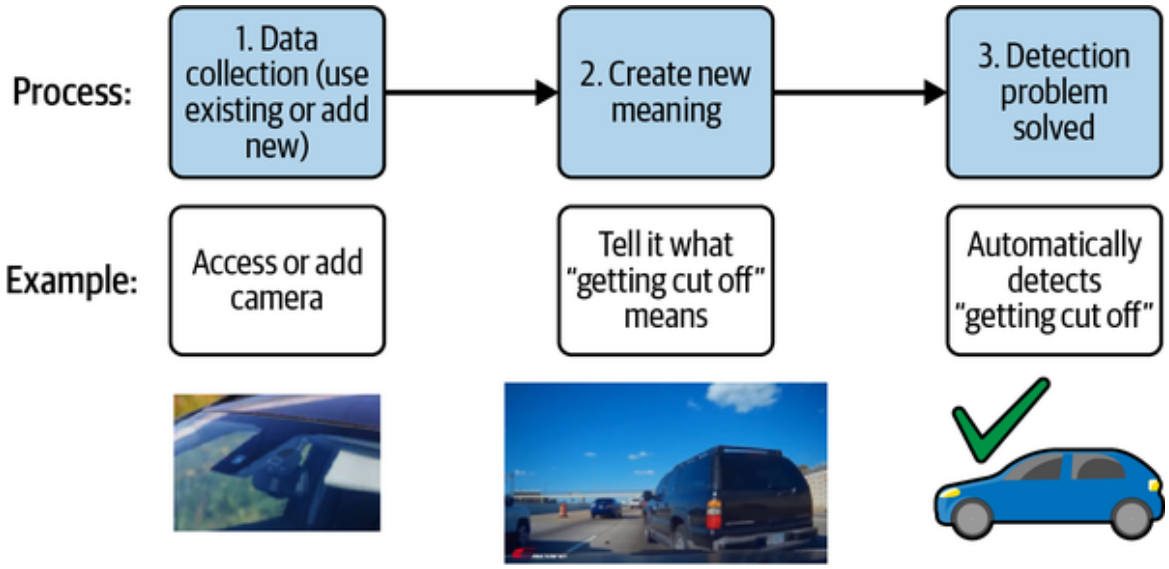


Figure 7-3. First example of creating new data for Data-centric AI

Or if you want to automatically detect what a “foul” means, you can create that too, as shown in Figure 7-4.



Figure 7-4. Second example of creating new data

You Can Change What Data You Collect

This may be obvious, but let’s consider how different the following two scenarios are. Classically, in data science, you could *not* change the data you collected. For example, if you were collecting sales data, the sales history was just that—history. Small specifics aside, the sales were whatever they were. You couldn’t invent new sales, or really change the data.

With training data, you can literally collect new data. You can place new cameras. Launch more satellites. Install more medical imaging devices. Add more microphones. Change the frequency of collection. Increase the quality, e.g., the resolution of the camera.

By focusing on the data that you can control, you can directly improve the performance. Better camera angle? Better AI. More cameras? Better AI. More…? I think you get the picture.

You Can Change the Meaning of the Data

Back to our sales example. A sale is a sale. There's little value in trying to expand a row in a spreadsheet to mean something more than it is.

With training data, you literally create meaning that was in no way, shape, or form there before. You look at a piece of media, like an image, for which the computer had no meaningful structure of interpretation before—and now you can literally say, "this is a human," "this is a diet soda," "this is a lane line." This act of annotation maps your knowledge into the computer.

The limit is only your creativity. You can say "this human is sad," or "this diet soda has a dent in it." You craft and mold and modify it to your needs. This quality of having infinite degrees of freedom is what makes it so powerful.

You Can Create!

So next time someone says to you that "data-centric AI is a way to get better model performance," you will know it's so much more than that! It means you can change the data you collect, and the meaning of that data. It means you can encode your understanding of a problem in a whole new way. It means you can define the solution even if no solution existed prior. You can *create*!

Think Step Function Improvement for Major Projects

Some of the efforts described next are major projects requiring many people, resources, and years of effort to complete. Think of them as conceptual thought starters.

We'll start by considering retail shopping. In that context, with these new methods, we can supervise machines to tell them what people look like and what groceries look like.

This unlocks all-new use cases, like entirely replacing the cashier. This is *not* 5% better pricing. This is a fundamental shift in how we shop for groceries and design stores. Further comparisons are shown in Table 7-2.

Table 7-2. Conceptual before and after capturing supervision for major projects

Concept	Before	After
Shopping	Every time I shop, every time a cashier checks me out, that work is lost.	Annotation of people shopping.
Driving	Every time I drive, the effort put in, the work, is "lost."	Annotation of driving. Professional annotators annotate common scenes. My driving is captured to aid in this effort.
Document review (imagine loans, requests, etc.)	Every document I review, the work is lost.	Annotation of document. Work is captured to reduce similar work in the future.

The key insight, which bears repeating: anything you can annotate can be repeated.

Build Your AI Data to Secure Your AI Present and Future

In the ideal case, every dollar invested in aligning AI has the potential to add value to every similar employee's role, or even generate all-new revenue channels. Keep in mind that while this is true, and possible, there are many challenges and risks with AI. Most AI projects require human engagement and oversight, and it's possible for projects to fail completely and never reach a minimum level of performance required to deploy and get value back. The more your team is able to learn the concepts in this book, the more they will be able to contribute to production success and a real return on your AI investment. By building your AI data transformation, you are securing your AI present and future. It will completely transform your existing projects and help protect you from competitors in the future.

Appoint a Leader: The Director of AI Data

All revolutions need leaders, someone to preach the new message. To rally the troops. To reassure doubts. And the leader must have a team. In this section, I'll lay out best practices and common job roles and discuss how they all come together to form an optimal team structure to support your training data revolution.

Team organizational concepts are key to training data success. From the company's viewpoint, what is changing? Are the differences between training data and data science reflected in the organization? What new organizational structures are needed? Even if you are already in an AI-centric organization, there are training data–specific nuances that can help accelerate your progress.

New Expectations People Have for the Future of AI

Right now, the de facto standard process of getting a bunch of people together to annotate seems akin to an old typing pool, an army of people, doing relatively similar, predetermined work, in order to translate from one medium to another. A "typing pool" of annotation, outsourced or otherwise, can work for some contexts, but is clearly inefficient and limited. What happens when the strategic direction, local priorities, or individual work

goals change? Where is the room for autonomy within business units, teams, and at the individual level?

Instead, what if people think of AI as part of their daily work? What if every new or updated process gave, first, a thought to AI transformation? What if every new application thought first about how the work could be captured as annotation? What if every line of business thought first about how annotation factored into their work? Explore before and after examples in Table 7-3 to consider the effects of the AI transformation.

Table 7-3. Before and after comparison of new AI expectations for staff

Before	After
Hire a new separate pool of workers (usually outsourced)	Your existing experts and data entry folks (primarily)
One-off projects, separate, one-time efforts	Part of daily work, like using email or word processing
“Bolt-on” mindset	“AI First” assumes AI will be present or demands it, integrated systems, aiding in the flow of existing work
AI being “pushed” onto people	People “pulling” AI into the org

Yes, people who are training AI instead of just doing their normal job may demand a higher wage. However, the return on capital is still a great deal better, even when paying a higher wage for one person, if that person, when combined with AI, is as productive as two or three people.

During the transition, it's natural that you still may need additional help. Depending on your business needs, there may be valid use cases that require outsourcing. As with any labor, there is a need for a spectrum of expertise in annotation. But the key difference here is that the annotation is seen as normal work, and not a separate project for "those people" to do.

The need for a range of talent requires a different approach when it comes to where you get that talent. A pool of workers exclusively hired "to annotate" without any other context at your business is distinctly different from any other level of worker hired into your business directly.

Another way to conceptualize this is to imagine a company of 250 people. Hiring 50 people overnight would be a massive deal. Yet that same company may think it's OK to hire 50 annotators. Try to see it more as truly hiring 50 people into your company.

You may already be thinking, there are a few areas that would be good targets, and/or "Well this sounds great, but I just can't see a way to annotate the such and such process." The reason I brought up this typing pool concept is that, when AI is a bolt-on, after-the-fact process, there will always be such hurdles. The more the organization is, instead, pulling AI in, seeing annotation as a new part of their day-to-day work, being directly involved in annotation, the more opportunities will come.

Sometimes Proposals and Corrections, Sometimes Replacement

A simple example of integrated proposal and correction that you may have perhaps already used is email. For example, in Gmail, it will prompt you with a suggested phrase as you're typing. That phrase can be accepted or rejected. Additionally, the suggestion can be marked as "bad" to help correct future recommendations, proposals, predictions, etc., as shown in Figure 7-5.

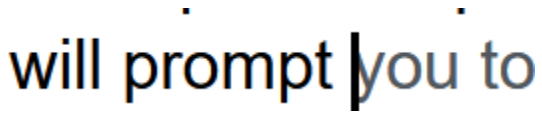


Figure 7-5. Example of AI proposal to user

This highlights an important consideration for all of the products you buy as you go forward. It also circles back to the theme of using AI to make someone more productive, rather than directly replacing them.

Upstream Producers and Downstream Consumers

Training data work is upstream to data science. Failures in the training data flow down to data science (as shown in Figure 7-6). Therefore, it's important to get training data right.

Why am I making a distinction between training data and data science?

Because there are clear differences between the day-to-day responsibilities of people *producing* training data and data science people *consuming* it.

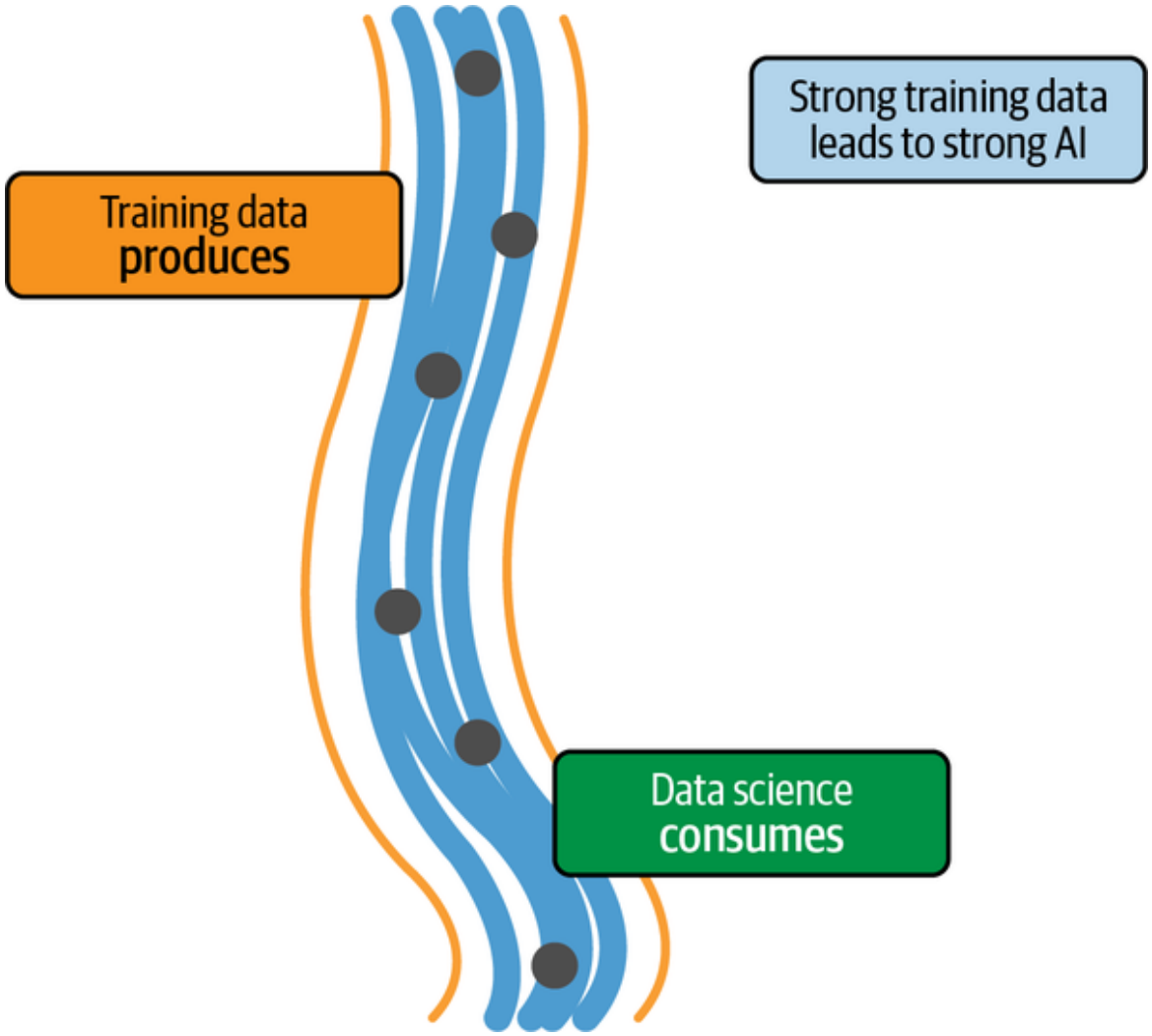


Figure 7-6. Relationship between production of training data and its downstream use by data science

Producer and consumer comparison

I think of the relationship between training data and data science as a producer and consumer relationship. See Table 7-4 for a comparison.

Table 7-4. Comparison of producing and consuming training data

Training data—producer	Data science—consumer
Captures business understanding and needs in a form usable by data science Converts unstructured data to structured data	Creates models that map fresh data back to business needs
Responsible for annotation workflow	Uses annotation output
Manages dataset creation, curation, maintenance	Uses datasets, mild curation
Supervises validity of outputs from data science to the business problem	Prediction outputs validity to training data
Example: KPIs (Key Performance Indicators): % of business need covered by data % of annotation rework required volume, V\variety, and velocity of annotation Depth of annotation	Example KPIs: Model performance, e.g., recall or accuracy Inference runtime efficiency GPU/hardware resources efficiency

Producer and consumer mindset

To a data scientist, the thoughts are often along the lines of "What datasets do we already have for *x*?" or "If I just had *x* dataset then we could do *y*." There is almost a short circuit, so to speak, where a project starts, and the moment there is an idea for something, the question is "how quickly can we get a dataset for this?"

An analogy is almost like, say I'm hungry and I want to eat something. I want to eat it now. I don't want to worry about the farm crop, or the harvester, or anything like that. There's nothing wrong with that—we all

need to eat, but we must realize the distinction. That is, even if farms and farmers might not come to mind when I'm hungry, of course, they are relevant to my situation. In our case, the farmers are like the producers of the training data, and they are equally important to training data.

The more one learns about training data, and the more emphasis is placed on production, the further one gets from concerns about how to actually use the data.

As an illustration of this, I had a conversation with a leading training data production director who was trying to figure out how to get a specific type of rotated box. I suggested annotating as a 4-point polygon, and that the box can be provided based on the bounds of the polygon. This was a surprise to him—he had thought of box and polygon as two totally distinct forms of annotation.

The deeper you get into the data science world, the more the focus becomes on the literal consumption of the data, rather than the production of it. That data science consumption step is usually relatively clear-cut, e.g., a one-time mapping. It's the human interactions that produce the data, the ongoing challenges around the production of AI data at scale—that production is the important thing to focus on for a training data expert.

Why is new structure needed?

Bad data equals bad AI. AI that is dangerous, misfires in commercial settings, and becomes wasted, or even a negative investment.

Second, as part of the goal of AI transformation, there must be a principally responsible individual to lead the charge. While a VP or CEO can also play this role at the strategic level, the Director of AI is responsible to execute this strategy. This Director of AI role is naturally critical to the success of AI projects. The new role will help the Director build out the needed mindsets and tools around data. To do this effectively, they must have the appropriate power within the company.

Third, as the volume of people involved balloons, the simple reality is that this is a team of teams, and with people of many distinct characteristics.

Even a small team (by large company standards) may likely have at least one or two production managers, and twenty to fifty annotation producers. In some cases, everyone using an AI tool is producing some form of human alignment. In a large organization, there may be hundreds or even thousands of annotation producers, all working to align multiple AIs and AI tools to commercial interests.[1]

It's an army of people to be managed. Strong leadership is required, and the budget to go along with it.

The budget

One of the most perplexing things about data science and training data organization is how to manage budgeting. Often, only a very small team of data science professionals are needed relative to a much larger volume of people doing data production. From a cost perspective, the data production cost may be an order of magnitude greater than data science. Yet somehow, the data science line item is often the top-level item.

If your overall objective is to maximize your AI results, an improved setup is:

AI/ML Structure:

- *Data production*: Everything to do with the production of data, including inserting and updating datastores, training data, human alignment, data engineering, and annotation
- *Data consumption*: Systems, tools, and people that consume data, querying datastores, including data science
- Supporting and other roles

This concept may seem straightforward, so why aren't teams already set up with this structure?

Historically, one reason for this has been in part the large hardware cost that data science is responsible for. While it's expected that this AI training and running cost will decrease with time, the cost needs to be allocated as the

team is built and scales. Additionally, from a divide-and-conquer perspective of resources, the data science team is already burdened with managing the hardware cost, so it makes little sense to further cloud their mind with training data concerns. Rather, dedicating budget and team headcount to a robust data production and data consumption model will maximize this skill set on the team and ultimately deliver better AI.

The AI Director's background

There are a few skill sets that are essential to consider when looking for someone to fill the important role of AI Director:

- This is a people-leadership role.
- This is a change agent role.
- The person must be in tune with the business needs.
- Ideally, the person is able to traverse multiple departments of the company; perhaps is already a corporate-level analyst.
- There must be some level of technical comprehension to facilitate discussion with engineering.

What the background need not be:

- Formal education requirements. This is more a school-of-hard-knocks role. In practice, though, this person may hold an MBA, under or graduate scientific degree, etc. Most likely, they will also be up-to-date on the latest refresher and online courses in machine learning areas.
- A "Data Scientist." In fact, the more data science background the person has, the more risk that they'll focus on the algorithmic side over this new creative, human-focused side.

The Director's budget will need to be divided into two main components, people and tools.

Director of Training Data role

Given the aforementioned team structure, it is most ideal to have a *Director of Training Data* position established early.

This person can, for example, report to the VP of AI, the VP of Engineering, or the CTO. Even if this role is baked into some type of Director of AI role, the level of responsibility stands.

Ideally, this is a full-time, dedicated person in this Director role. In case it is not possible to have a dedicated person, then the role can be taken on by someone else.

Figure 7-7 illustrates both the Director of Training Data's responsibilities and sample descriptions of the key team member roles. These are not meant to be complete job descriptions, just highlighting some of the key structural elements of the role. The roles within the AI Center may be combinations of part-time or full-time positions. It's assumed that the engagement with the rest of the org will be mostly on an as-needed basis.

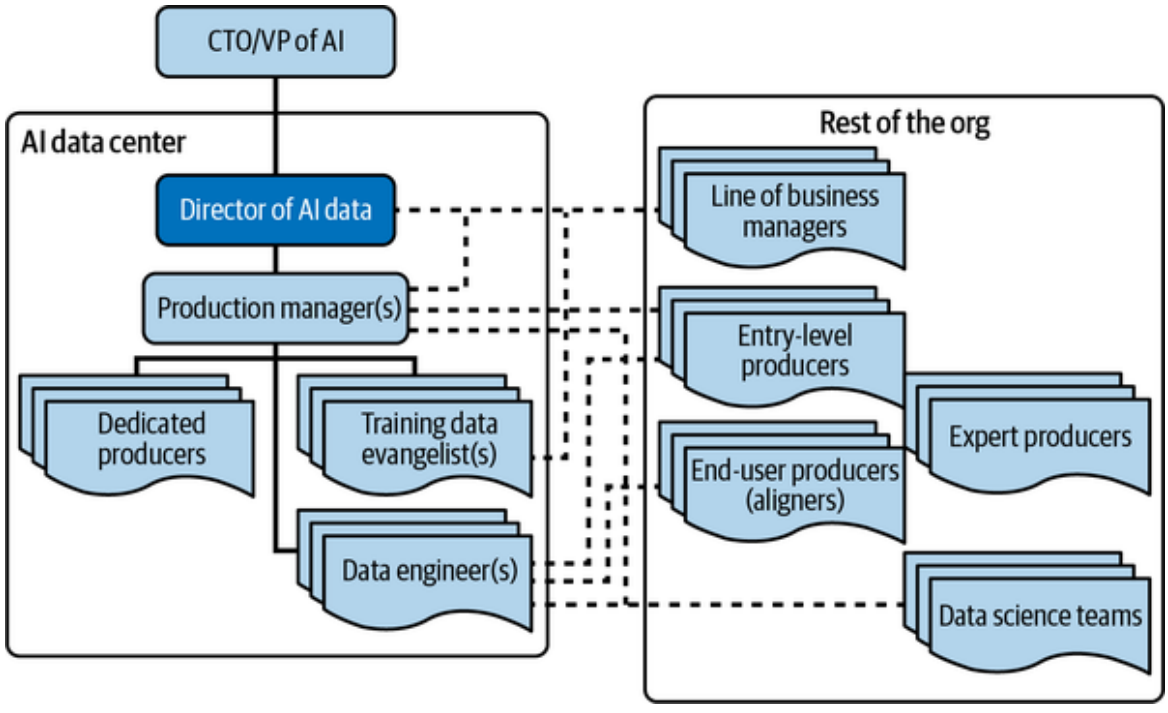


Figure 7-7. New org chart example

Note that Figure 7-7 refers to "teams" or plural "engineers." Of course, your org will not match this chart exactly. Think of it as a starting point.

Each box can be thought of as a *role*. One person may play all or most of the roles.

AI-focused company modifications

- The Producers will still vary depending on the company. For example, the Experts may be the end users, or they may be part-time and external, but still named partners not a generic “pool.”
- May have fewer or zero evangelists.
- Line of business manager may be the overall Product Manager.

Classic company modification

These may have fewer dedicated producers and more evangelists.

Spectrum of Training Data Team Engagement

Ideas around the primary organization pulling AI into its process is not mutually exclusive with the idea of there being a team or department for training data. In an AI-mature organization, the team may be primarily acting as advisors, staying on top of the latest trends, and maintaining the overall tools.

What’s right will depend on your specific org; my main intent here is to convey the general spectrum, and that the idea of a separate team is needed, even if they aren’t the ones doing the literal production of the annotation data.

The team will have three main areas of responsibility:

- Advisory and training
- Maintaining tool sets used by producers in other teams
    - Data ingest/access
    - Support for annotation production

- Actively managing production of annotation data

Dedicated Producers and Other Teams

Dedicated producers are direct reports of the production manager. This is for cases where the volume of work is such that the person's full-time job is annotation, and they are not attached to any specific business unit. Again, long-term, this may be rare, but it's a reality for teams getting started, transitioning to this structure, and for various projects where no other production capabilities are available.

For the sake of simplicity in the diagram, outsourced teams can be thought of as dedicated producers.

Organizing Producers from Other Teams

Producers in other business units run the spectrum, from entry-level to expert. End users, who may be different, may also produce their own annotations. Usually, end-user annotations are more "by accident" as part of using the application, or providing some form of minimal feedback.

Some of these titles speak for themselves, but others may be a bit more opaque. The following section walks through how each role contributes to better AI. Let's dive in!

Director of AI data responsibilities

Chiefly, this person is responsible for the *overall production and consumption of data*, with a focus more on production as the primary priority.

This includes the following main areas of responsibility:

- Turning the line of business needs into successfully produced training data.
- Generating work for training data production by mapping business needs to training data concepts.

- Managing a team of production managers who facilitate the day-to-day annotation production.
- Managing “Evangelists” who work with the line of business managers to identify training data and AI opportunities. This is especially relevant when it comes to feasibility concerns regarding annotation. Of the various ideas proposed by, say, a line manager, only a handful may actually be cost-effective at that moment in time to annotate.
- Managing the training data platform.

Besides general efficiency and visibility into annotation work, this person must map the productivity in annotation back to the business use case. *The Evangelist may do this too, with the Director being the second line.*

And there are some additional things the Director of AI should attend to:

- Managing the consumption of data via coordination with data science (DS) or also managing DS directly.
- Acting, indirectly, as a check and balance on data science (or internal DS teams), acting as a check on the business results and on the output of DS, going beyond purely quantitative statistics.
- Normal director-level responsibilities, including if applicable: headcounts, profit and loss responsibility, KPIs, supplier and vendor relationships, reporting, planning, hiring, firing, etc.

Naturally, the director can fill in for most any of the following roles as needed.

Training Data Evangelist

This role is that of an educator, trainer, and change agent. Their primary responsibilities include two areas:

- Working closely with the line of business managers to identify key training data and AI opportunities.

- Working “ahead” of production managers, establishing the upcoming work and acting as the glue between the line of business managers and the production managers.

In a company focused exclusively on AI products, their focus should be training:

- Educating people on the best usages of modern supervised learning practice.

In a classic business, there will be more breadth:

- Educating people in the organization on the effects of AI transformation. In practical terms, converting interest to actionable annotation projects.
- Recruiting annotators from that line of business. On a practical level, this would be about converting an employee doing regular work into someone who, say as 20% of their job, is capturing their work in an annotation system.
- Training. In the context of part-time annotators especially, this person is responsible to explain how to use tools and troubleshoot issues. This is distinct from the production managers, who are more geared toward training full-time annotators. This is because you train a doctor differently than you train an entry-level employee.

Training Data Production Manager(s)

This person is chiefly concerned with being a taskmaster for actually getting annotation work completed:

- Interfacing with data science to set up the schemas, setting up the tasks and workflow UIs, doing the admin management of training data tooling (generally non-technical).
- Training annotators.
- Managing the day-to-day annotation processes.

- In some cases, basic data loading and unloading can also be done by this person.
- When in change-management type discussions, this person is responsible to explain the reasonableness of annotation work to people new to the matter.
- Using data curation tooling.

Annotation Producer

Annotation Producers usually fall within two buckets:

- Part-time, which will increasingly become part of everyone’s job to a degree
- Full-time, dedicated and trained people
    - May be newly hired people or reassignment from existing work

The primary responsibilities of Annotation Producer include the following:

- Implementing day-to-day alignment of AI to business needs
- Surfacing issues in the AI schema
- Daily front-line usage of AI tools, and tools that use AI

Data Engineer

Data Engineers have the following responsibilities:

- Getting the data loaded and unloaded, the technical aspects of training data tools, pipelines setup, pre-labeling, etc.
- Especially important is organizing getting data from various sources, including internal teams
- Planning and architecting setup for new data elements
- Organizing integrations to understand the technical nuances of capturing annotations

Data Engineers interface regularly with the data science team(s). Previously, this was out of necessity—often, projects would simply not work without close planning. Now, there are growing standards around AI datastores that make it possible to interface more on conceptual levels, rather than hashing out every implementation detail.

HISTORICAL ASIDE

There are a few reasons the Production concept wasn't needed before:

Classic machine learning the datasets already existed (even if messy), so there was no need to "Produce" a dataset.

Earlier efforts were more separated from day-to-day business goals. This meant there was more of a rationale to do one-time planning, one-off projects, isolated projects, etc. As AI transformation moves into the mainstream of your business, this separation becomes an artificial barrier.

Use Case Discovery

How do we identify viable use cases? What is required and what is optional? In this section. I provide a basic rubric to identify valid use cases. Then I expand into more context to help further identify good use cases.

This section is organized from the most concrete to the most abstract:

- Rubrics for good use cases
- Example use case compared against the rubric
- Conceptual effects, second-order effects, and ongoing impacts

The simplified rubric may be a go-to in day-to-day work, while the rest can act as supporting knowledge. While you are welcome to use the rubrics exactly as they are, I encourage you to think of everything here as thought starters, merely an introduction to thinking about use cases for training data.

Rubric for Good Use Cases

At the highest level view, a good use case must have a way to capture raw data, and at least *one* of the following:

- Repeated often
- Involves experts
- Adds a new capability

The more of these things that are present, likely, the greater the value of the use case. A rubric is shown in Table 7-5. Note that this rubric is focused on training data-specific use case questions and is not a generic software use case rubric.

Table 7-5. Rubric for good use cases

Question	Result (with example answers)	Requirement
Can we get the raw data?	Yes/No	Required
Is it repeated often?	Yes/Sometimes/No	At least one of these is required
Does it involve an expert?	Yes/Sometimes/No	At least one of these is required
Does it have a positive effect on multiple people's existing work?	Yes/Sometimes/No	At least one of these is required
Does it add a new capability?	Yes/Sometimes/No	At least one of these is required

That’s it! This can be a go-to reference rubric, and to support it when needed, you can use the more detailed rubric in Table 7-6.

A key note before we dive in, some of these use cases may be a major project that has a large scope. Naturally, the level of scope will be relative to your specific case. If a use case will be a big effort relative to your organization’s resources and capabilities, it is up to you to determine its applicability.[2]

Detailed rubric

Now that we have the general idea, let’s expand on it. In Table 7-6, I provide more detailed questions, especially expanding on and differentiating the new capability concepts. I add examples, counter examples, and some text on why it matters to help.

Table 7-6. Detailed use cases rubric

Test	Example	Counter example	Why it matters
Is the data already captured? Or is there a clear opportunity to add more sensors to capture the data (in its entirety)? (Required)	Existing documents (e.g., invoices), existing sensors, adding sensors	Car dealer in-person sales interaction	• Getting raw data capture is a required step. • If you can’t get the raw data then the rest doesn’t matter!

Test	Example	Counter example	Why it matters
Does it involve experts?	Docter, engineer, lawyer, some specialists	Grocery shopping	• Makes a constrained resource available to more people (and potentially more often, and in more situations). • Expert opinions are of high value. • More readily available data, often already in digital form.

Test	Example	Counter example	Why it matters
Is the work repeated often? Many times per minute? Hourly? Daily? Weekly?	Automatic background removal/blur Customer service and sales Administrative document review		• Already will have a well understood pattern (at least by humans). • Likely already is relatively well constrained. • Often repeated tasks, viewed in aggregate, have a high value. • Existing raw data may be already being captured.

Adds a new capability use case

Sometimes the goal is to unlock new use cases, beyond augmentation or replacement. (Add a new capability.) A use case rubric to help make great use cases when the goal is to add a new AI capability is shown in Table 7-7.

Table 7-7. Use case rubric when the goal is to add a new AI capability

Test	Example	Counter example
Is the work done rarely because of expense? Would it be of great value to increase the frequency?	Inspections	Stadium construction
Can we turn an approximate process into a more exact one? Does this process gloss over something because doing it in more depth is currently impractical? Are we currently substituting what we really want to figure out for something more generic? Would improving this process's accuracy lead to more benefits than harm?	Fruit ripeness, produce mold or bruising detection, dented can detection An airport detection system that just detects metal, versus something that can detect very specific threats	Loan underwriting

Test	Example	Counter example
Is there something that gets completely skipped because it would take too long or is otherwise impractical (e.g., because of volume)? Would it be of great value if we could do it? Anything is relatively time-intensive (even if it happens rarely).	Analysis of video meetings and sales calls Porn detection in video uploads, comment moderation Insurance property review[a]	

[a] May take a while per house, but may only need to be done once per year or decade.

The main thing to distinguish this use case is that it's something that otherwise wouldn't happen. For example, a bridge may be inspected annually currently, but it would be impractical to inspect it daily. So an automatic bridge inspection system will add a new capability. This is a good use case, even though currently it is not repeated often (annually), and may or may or may not involve expert labor directly. For example, the actual inspectors may be looking for cracks and measuring them, while the engineering analysis is still done by someone else. Either way, it would be impractical for anyone to inspect it every day.

As we transition from no AI to AI-assisted work, or even relying solely on AI for some cases, there are naturally going to be overlaps, e.g., cost duplications. Also, the goals of adding AI vary widely, from new capabilities regardless of cost, to improving performance or economy.

AI has the potential to add a new capability that otherwise would have not existed. However, that new capability may be more expensive than even a

similar human capability, and may not perform even at the level of a human initially. It also might be generally not economically viable.

Repeating use cases

1. Don’t jump to assuming replacement—first think of augmentation.
2. Normal coding is fine for forms. Rather, think “what does a human do when they get the information?”
3. To get a good idea, look at how many “repeat” roles there are in the company. Are there thousands of people doing roughly the same thing? That’s a great place to start.

Specialists and experts

All work involves some degree of specialization and training. Instead of offering “low-hanging fruit” in specific cases, here are some of the areas that often have the most opportunity (not necessarily the easiest). The expert case is generally meant to signify something that is otherwise of great difficulty to get. Of course, what *expert* means will be up to you; my own mental model is something along the lines of “a skill that takes 5–10 years, after normal education, to get to a basic level of proficiency, or an area that is so cutting-edge as to limit the pool of available people.”

Evaluating a Use Case Against the Rubric

Here we present an example use case in some detail and then compare it to the rubric.

Automatic background removal

Have you recently done a video call and noticed someone’s background was blurred? Or maybe you already use this feature yourself. Either way, most likely you have already interacted with this training data–powered product.

Specifically, when you take a videoconference (e.g., Zoom) call, you may be using the “background removal” feature (Figure 7-8). This turns a messy,

distracting background into a custom background image or smoothly blurred background—seemingly magically.

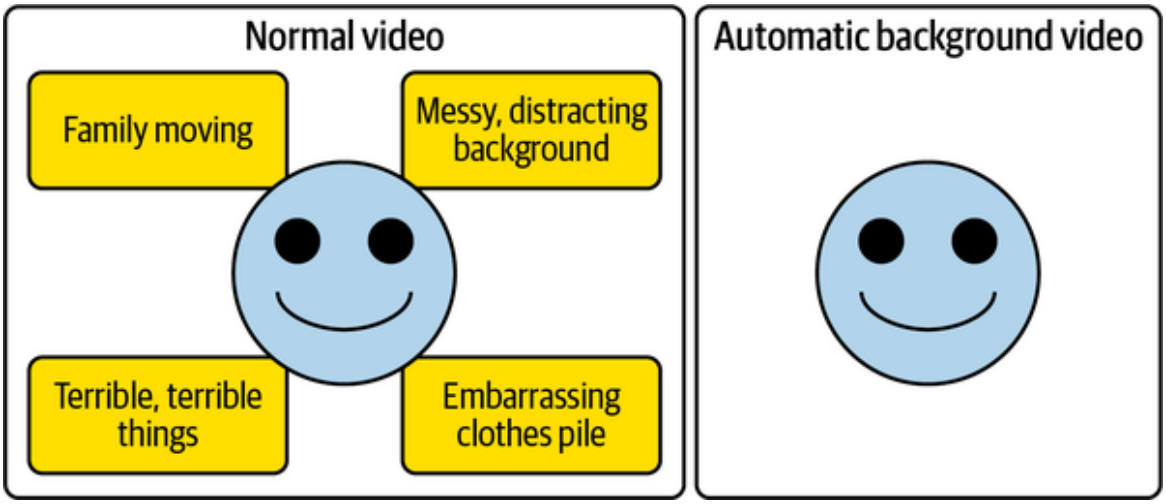


Figure 7-8. Comparison of normal video to video with automatic background removal

For context, this used to require a green screen, custom lighting, and more. For high-quality productions (like movies), often, a human has to manually tune the settings.

So, how is training data involved?

First, we must be able to detect what is "foreground" and what is "background." We can take examples of video, and label the foreground data as shown in Figure 7-9. We will use that to train a model to predict the spatial location that's "foreground." The rest will be assumed to be background—and blurred.

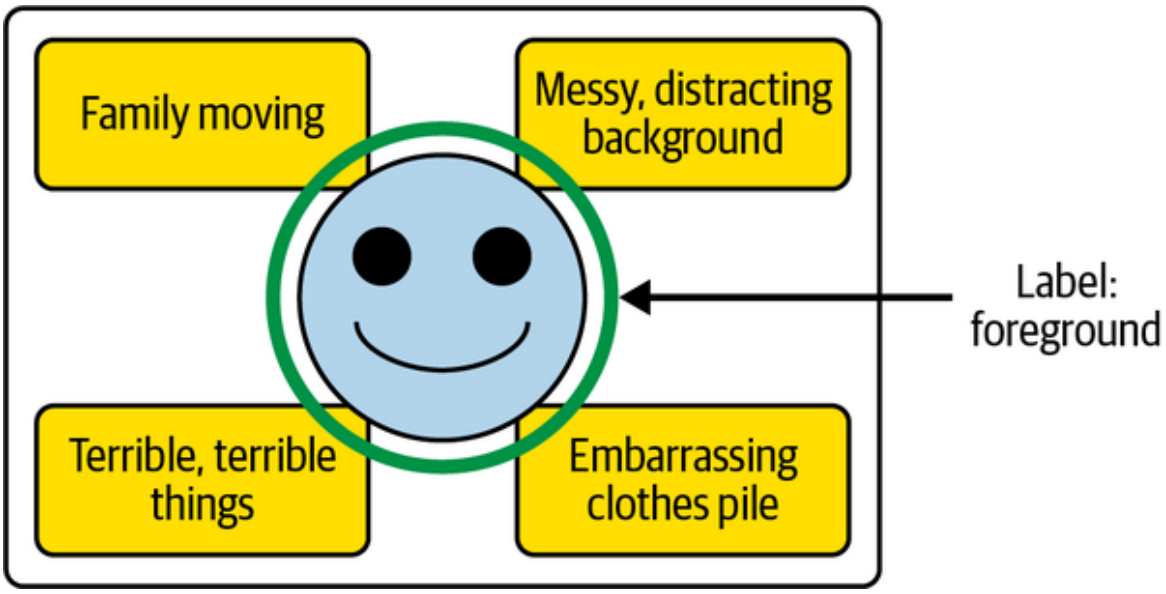


Figure 7-9. Example of labeling foreground

The point here is that the model is figuring out what patterns make up the "background." We aren't expressly having to declare what a messy pile of clothes looks like.

If we constrain the problem to assume that only humans will be in view, we could take an "off the shelf" model that detects humans and simply use that.

Why does a feature like this matter?

- Iterates more equality. Now it doesn't matter if I have a fancy background or not.
- It improves privacy and meeting efficiency. It helps reduce impact of disturbances (e.g., someone entering the edge of the calling area).

If this seems really simple—that's the point. That's the power of training data. Something that used to be *literally impossible* without a green screen becomes as easy as labeling videos.

Now a few gotchas to keep in mind:

- Getting a performant model to do "pixel segmentation" at time of writing is still somewhat challenging.

- There are public datasets of humans that already do a fairly good job. However, if you had to label a “Zoom video calls” dataset from scratch it would be a tremendous amount of work.

Evaluation example

Overall, the Automatic Background Removal use case scores fairly well in our evaluation, as shown in Table 7-8. It covers the requirement of being able to get the data. And gets a huge yes in the “repeated” category. Plus, depending on what subset of the use case you want to evaluate, it potentially avoids having to have an expert and adds a new capability. For example, I can’t imagine being able to easily get a custom background in the middle of a cafe or airport without this.

Table 7-8. Example of using the evaluation rubric on the Automatic Background Removal use case

Question	Result	Requirement
Can we get the data?	Yes, the video stream is already digitally captured.	✔️
Is it repeated often?	Yes, a single video call may remove thousands of background frames, a single user may have multiple calls per day, and there are many video callers.	✔️
Does it involve an expert?	Sometimes. It takes some expertise to set up a green screen effect, but is far from expert medical or engineering knowledge.	—
Does it add a new capability?	Sometimes. It was possible before to get a green screen, but in cases like traveling, even a person who owns a green screen would be unable to use it, so in that case adding a new capability.	—

Use Cases

The conceptual effects area has a small list of use cases. For the sake of this rubric and timing, I dive only into a single use case deeply. My intent is to convey the conditions with which any use case would be good, and provide tools to think about the value add and overall effects. Given the breadth of potential use cases, I believe this is much more valuable than attempting to iterate a list of all known use cases; many such up-to-date lists can be found with online searches.

Conceptual Effects of Use Cases

This is a fairly similar idea to above, but from a slightly different perspective. In the preceding section, we were looking at this from the perspective of “What is a good use case?” Now I am looking at “What are these use cases doing?” I have also included some surface-level, obvious second-order effects—effects caused by adoption of the technology outside of the scope of the technology itself. I suggest using Table 7-9 as merely a starting point for thinking about the second-order effects of these concepts.

Table 7-9. Conceptual use case examples and second-order effects

Concept	Examples	Second-order effects
Relaxing constraints on the problem itself (on a prior solved problem)	• Green screen -> Any background. • I must be of a certain age to drive. • Spellcheck - > Grammar check.	• My background is not used to evaluate my candidacy for a job interview. • “Taking the kids to x” takes on a new meaning if the parent isn’t driving. • Expectations around correct grammar change (in addition to correct spelling).

Concept	Examples	Second-order effects
Replacing or augmenting routine work	• Human counting sheep -> automatic counting of sheep. • Human driving -> Car driving (parity with human performance). • Human routing communication to a department -> automatic communication reporting based on intent (sales, support, etc.).	• The meaning of work changes. • Millions of jobs will be created and shifted. People will be required to learn new skills. • The suburbs may extend further out. • A company that is not using AI effectively will have a worse cost structure than one that is (e.g., the same as not using digital effectively).

Concept	Examples	Second-order effects
Making humans “superhuman”	• Airport security scanning. • Sports analytics. • Self-driving (accident reduction). • Acting as a “second set of eyes” on routine medical work.	• Airport security may become more effective and faster (Here’s hoping!). • Intensity of sports may increase since expectations of elite levels of coaching are extended to more people. • Accidents may become more rare and even more newsworthy.

Concept	Examples	Second-order effects
Making a constrained resource available to more people[a] (or without as many limits)	• Radiologist's time. Prior, a radiologist could only see as many people as they had time in the day; now an AI medical system can assist a radiologist in seeing a larger number of patients.[b] • This also removes the geographical limits. • Self-driving (more mobility because of lower cab fare due to shared resource).	• Medical care may become more accessible. • The meaning of a "second opinion" may change. • New dangers will appear, e.g., increased groupthink, data drift, decreased weight on human expert opinion.

[a] This is similar to the "Relaxing Constraints" but the examples are fairly different, so we'll keep them in separate categories.

[b] There are many limitations to the implementation details of this, of course.

Ongoing impact of use cases

At its core, the idea is that training data is an easier way to encode human knowledge into a machine.

The cost to "copy" human understanding approaches zero. While before, a radiologist's time was a scarce resource, it will become abundant. Before, a green screen was only in a film studio; now it's on my smartphone anywhere in the world.

I use the word “understanding” to differentiate between something that can be written down in a book, and something that up until now, could only exist in the human mind. Now machines, too, can form “understandings.”[3]

This has the following follow-on implications:

- It is possible to dramatically increase the frequency of actions.

  For example, prior a visual bridge inspection may only be able to happen once a year or once a decade. Now, an analysis of a similar level could happen every few seconds.

- Processes that were previously “random” will become relatively “fixed.”

  We all know car accidents happen. But eventually, they will be rare. What was previously a random process (we’ve all asked someone dear to us to “call me when you get home”) will turn into an all but sure thing. (When there is one, it’ll be international news! “The first car accident in the last 24 months just happened!”)

- Previously impossible things will become possible.

  For example, putting a “dentist in your pocket”; eventually, you will be able to point your phone’s sensors at your mouth and get a level of insight that previously would have required a dental visit.

- Increased personalization and effectiveness of “personal” assistance will increase.

The New “Crowd Sourcing”: Your Own Experts

Who is annotating and training data culture go hand in hand. The more people who understand training data, the more opportunities will rise up. The more your own staff and experts are involved, usually, the higher the quality will be. There are a few other things to keep in mind:

- Buying into the “training data first,” a data-centric mindset, is one thing.
- Who is actually doing the annotation is a different decision.
- Reframing your existing teams as “The Crowd” will help you get higher-quality data at a greater quantity.

By leaning into your own staff, you can get better results while controlling costs.

Key Levers on Training Data ROI

These factors will ultimately determine the “shelf life” of training data and its ability to be leveraged into tangible productivity improvements:

Talent, or who is annotating

> To quote someone who managed a large team of annotators: “The biggest determinant of annotation quality is the person who created the annotation.”

Degree of training data culture

> This can be a step function–type delta. Either people are aware something can be turned into training data or they aren’t.

When it comes to AI annotation talent, if you have to pick one, lean toward quality over quantity.

What the Annotated Data Represents

Thinking about your annotation in that context further supports the need to have a dedicated business unit focused on it. It also showcases the need for awareness at all stages of the buying process. For example, if you are buying a system to automate something, and the vendor is responsible for

the annotated date, what does that mean for your future? This is what's at stake:

- Your business know-how, trade secrets, processes, and competencies
- The massive labor investment you needed to make to create, update, protect, and maintain it
- Your key to staying competitive during the AI transformation

Trade-Offs of Controlling Your Own Training Data

Arming your team with the knowledge they need to tackle training data in house can bring many benefits to your business:

- Better quality by leaning into existing experts and domain knowledge
- A better cost model by repurposing or reframing existing work
- Creating a durable, shareable, and reusable library of training data
- Control over both the economics and quality of the output
- More internal management overhead, potentially less flexible
- Requires organizational knowledge and change efforts

The Need for Hardware

First, let's get some sticker shock out of the way. Big, mature AI companies spend tens to hundreds of millions of dollars on AI computing resources (e.g., GPUs, ingestion, storage). This means that hardware cost is a key consideration.

Second, training data is your new gold. It's one of your most important assets. People may be your most important asset, but this is a literal embodiment of people. It's important. How do you protect it?

Realistically, if you aren't controlling the hardware, there is very little you can do to protect that data. What happens if there is a contract dispute?

What if the vendor's controls aren't as good as you thought? You have your key business data and records under your control—and training data must be the same way. To be clear here, I don't just mean to say the annotation tooling. Any data that any of your vendors are capturing that's used for training AI must be considered here.

This means that while a SaaS solution may be OK for getting started, proof of concept, etc., for ongoing training data requirements, the hardware costs, and degree of importance to the company, are too great to not take control of it.

Practically speaking, if you leave any type of prediction, annotation automation, etc., in the hands of a vendor's server (unless it's being run on the client) means you will be actually negotiating a huge amount of hardware costs hidden inside the annotation tooling cost.

Common Project Mistakes

Under-resourcing is especially prevalent here. I have seen a number of solo professionals, from pathology to dentistry, be curious about AI. While this curiosity is naturally great, the reality is that you need a strong team effort and substantial resources to build something that will be even a true prototype level, or in production.

There are a few common roadblocks to getting started:

- Project is under-resourced. A single doctor is unlikely to make a general-purpose AI, even for their niche area.
- Mistakes on volume of data needed. For example, a major dental office's x-rays for all time for all patients may be substantial, but on its own, still probably is not enough data for a general-purpose dentist AI.
- Most AI projects have a very, very long time horizon. It takes on the order of months to years to build reasonable systems. And often, the expected lifetime and maintenance is measured in years or even longer.

Modern Training Data Tools

The effective usage of training data tools can make orders-of-magnitude differences. The first step is to gain awareness of what high-level concepts exist. If you have read all the way through, you have already taken a big step in this direction; getting this book and other material in front of your team is a great way to help accelerate this process.

Software can help provide guardrails and encouragement to a transformation, but it is only a part of the overall transformation.

Training data–focused software is designed to scale to the breadth and depth of complexity that's needed to actually capture your business processes with the fidelity required. There's a big difference between tagging a whole image, versus a polygon with complex attributes, that has gone through a dedicated review process.

Training data software has come a long way, and has had millions of dollars of investment since its humble origins. Modern training data software is more in line with an office suite, with multiple complex applications interacting together. While perhaps not quite as complex line for line, directionally, it will be in a few years' time.

Think Learning Curve, Not Perfection

There is a tendency to seek out perfection and familiarity in training data software, especially if an existing early team happens to be familiar with a certain pattern. The simple truth is that all software has bugs. The other day, I used Google search, and it duplicated the menus and search results. That's a product with oodles of engineering effort over decades! If we think back to early computer applications, they had terribly obscure UIs. People had to learn many concepts to perform simple tasks. The same general principle applies here.

The other truth to understand is that these application and use cases are continuing to increase in complexity. When I first started this, I could provide a demo of most of the key functions in half an hour. Now, even if I

scope the demo to a specific persona (like an annotator), and a specific media type (like an image), it still may take half an hour! A complete end-to-end coverage would be days, similar to how even a basic training course, for someone who's never used a word processor or spreadsheet, would take days.

So how do we tackle this learning curve?

Having deeper and broader discussions

> While UI design and customization is important, over-focusing on these elements can cause us to miss key points. If the SMEs are so busy that they can't take some basic training (or just time) to figure out how to use the application, then realistically, are they going to provide quality annotations?

Customization and configuration

> Further, while off-the-shelf software should absolutely be the starting point, we have to recognize that there will always be the need for some degree of customization and configuration.

Training and new knowledge

> New training and knowledge are required, from leadership to frontline staff, and from concepts to specific tooling.

New Training and Knowledge Are Required

Let's discuss some specifics on what training is needed—people training in this case, not training data.

Everyone

- The introduction: high-level overview of what Supervised AI is and how it relates to your specific business.
- Clarity of AI's role. For example, if it's the case that AI will generate Proposals, and Humans will do alignment.
- The effect of AI on work. For example, that AI supervision leads to more productive and interesting work.
- The task: To bring up ideas of processes that are ripe for this type of supervision.

Annotators

- The basics of annotation tooling. To continue the office analogy, knowing how to use annotation tooling will be the modern equivalent of learning word processing.
- More in-depth training, such as, in part, reading this book, and further training on sensitive issues like bias.

Managers

All of the above, and:

- Knowing what questions to ask regarding new and updated processes.
- Understanding how to identify financially viable training data opportunities.
- Reflecting on productivity goals in this new world of AI annotation. Every moment of work done that's not captured in annotation is a moment lost.

Executives

Will need to reflect on the company organizational structure and consider elements such as the creation of a new training data unit

Should germinate, nurture, and guard the culture around training

Will need to carefully consider vendor choices in relation to securing future AI goals

How Companies Produce and Consume Data

Who is making the software that consumes training data? Who is producing the data?

Here are the major themes I have seen in practice:

- A software-focused company produces an AI-powered product, and the bulk or all of the training data is also produced by this company. The firm releases the software to consumers, or another firm buys the software and is the end user.
- A company with in-house training data production capabilities creates the software for its own internal use. Often, this may involve leaning on external partners or very substantial investment.
- A software firm produces an AI-powered product, but leaves the bulk of the training data to be produced by the end user buying the software.

The only case where the end-user company is not involved in the training data production is 1. In general, that either shifts the core competence of the business to that software provider, or it means there is a relatively static product being provided. An analogy here would be buying a “website” that can’t be updated. Since few people want a website they can’t update, in general, the trend will likely be toward companies always being able to produce their own training data in some way.

From the executive viewpoint, in a sense, the most key questions are: “Do you want to produce your own data?” and “Do you want AI data to be a core competency of your business?”

Trap to Avoid: Premature Optimization in Training Data

Optimizing your training data too early in the process can bring challenges. Let’s explore this further.

Table 7-10. Premature optimization, warning signs and how to avoid it

Trap	How it happens	Warning signs	Avoiding it
Thinking a trained model = Done	1. Effort is taken to train a model. 2. It sort of works and people get excited. 3. Assume it just needs some small adjustments. 4. Realize it's far from done.	• The "Trained Model" is discussed as the end goal. • Ongoing continuous annotation is not discussed. • Iteration is talked about, but in the context of a limited time window.	• Educate people th the goal i set up a continual improven system, n single on model. • Discuss u front wha performa level is g enough t ship versi 1. For example, self-drivi some hav taken the approach "equal to human is good enough."
Committing to a schema too early	1. Use up a lot of resources annotating.	• Schema does not change significantly	• Expect th schema t change.

Trap	How it happens	Warning signs	Avoiding it
	2. Realize the labels, attributes, overall schema, etc. do not match their needs. E.g., were using bounding boxes and realize they need keypoints.	between early pilot work and more major work. • Schema was determined with minimal data science involvement. • “Final” schema was determined without proof that achieving success will solve downstream problems.	• Try many different schema approach with actu models to what actu works—d assume anyone h the right backgrou to know t answer in advance. • Ask: if th model ma perfect predictio does this actually s our downstre use case? E.g., if it perfectly predicts t box on re will that s the overa problem?
Committing			

Trap	How it happens	Warning signs	Avoiding it
to automations too early	1. Look at how many resources human annotation may take. 2. Look for automation solutions. 3. At first are happy with automation results. 4. Realize the automation isn’t quite doing what they that it was doing.	• Unrealistic expectations arise, e.g., automation expected to virtually solve annotation. • Automation explored without involving data science. • Automation plans being discussed in detail before any significant human annotation work has been done. • A reduction in management mindshare in training data under false assumption that	• Realize it doesn’t ta much hur annotatio start getti directiona understar of needs. • Use mini or zero automatic until you have don enough manual human annotatio well understar the doma • Do more human annotatio than you think you need. Aft you will l the best position t choose th most effe automatic

Trap	How it happens	Warning signs	Avoiding it
		automations handle it.	• Think of automatic as an expected of the process, r silver bul

Trap	How it happens	Warning signs	Avoiding it
Wrongly calculating volume of work	1. Look at overall dataset size. 2. Project how many annotations. 3. Assume will need all of them.	• Assuming all available data needs to be annotated (data can be filtered for most valuable items). • Not considering ongoing accumulation of data or production data. • Not considering diminishing returns, e.g., that every further annotated item adds incrementally less value than the prior item.	• Get a larg enough sample of actual wo to unders how long each sam usually ta • Realize th it's alway going to l moving target. Fo example, work per sample m get harde the mode get better

Trap	How it happens	Warning signs	Avoiding it
Not spending enough time with tooling Not getting the right tools	See “Tools Overview”.	• Overfocus on “getting a data set annotated” instead of what the actual result is going to be. • Unrealistic expectations; treating it more like a fly-by shopping website than a serious new suite of productivity tools.	• Realize tl new platforms like “Photosh walks int bar and n a gruff Taskmast who moonligh a Data Engineer. It’s comp and new. • The more powerful tool, the r the need t understar Treat it n like learn a new sub area, a ne art.

[a] I’m not commenting if this approach is right or wrong.

No Silver Bullets

There are no silver bullets to annotation. Annotation is work. For that work to have value, it must be literally labored at. All of the means to improve the productivity of annotation must have a grounding in that real-life reality.

Training data must be relevant to your business use case. To do this, it needs the insights from your employees. Everything else is noise or in-depth expert, situationally specific concepts.

Instead of looking at some of these optimizations as true gains, rather see missing out on them as being functionally illiterate.

This caution is important because, being such a new area, the norms are poorly established, making a clear understanding of the history, future goals, and latest conceptual frameworks that much more important.

Culture of Training Data

Getting everyone involved with training data is at the heart of AI transformation. In the same way that the IT team doesn't magically re-create every business process on their own, each line of business manager brings a growing awareness of the capabilities of digital tools and what questions to ask.

One of the biggest misconceptions about modern training data is that it is exclusively the realm of data science. This misconception really holds teams back. Data science is hard work. However, the context in which that hard work must be employed is often confused. If the AI projects are left as "that's the data science team's responsibility," then how likely do you think success company-wide will be?

It's very clear that:

- Non–data science expertise, in the form of subject matter knowledge, is the core of training data.
- The ratio of people who are SMEs to data scientists is of vast fields to a single grain.

- Data science work is increasingly becoming automatic and integrated into applications

The great thing here is that most data scientists would be happy if other people knew more about training data. As a data scientist, I don't want to worry about the training data for the most part, I have enough of my own concerns.

Not everything, at least right away, is a good candidate for training data. Being able to recognize what will likely work and what won't work is a key part of this culture. The more bottom-up the ideas for this, the less likely it will turn out to be something that appears easy, but is actually terribly hard, and fails.

New Engineering Principles

The first step for getting a better data engineering setup in place is to recognize the intersection of the business need, and the expert annotators, with the day-to-day concerns about data. That will lead to needing an AI datastore that can combine all of your BLOB, schemas, and predictions in one place:

Separating UI/UX concerns of annotation experiences

> For example, creating UI/UXs embedded into existing and new applications.

Making the training datastore a central component

> To run a web server, we use standard web server tech. It's the same with training data. As the complexity and investment in this space continues to grow, it makes more and more sense to shift that training data into a dedicated system, or set of systems.

Abstracting the training data from data science

> A simple break point here is dataset creation. Instead of a dataset being something that's static, if we allow training data to be created on its own accord, and data science to query it, we get a much more powerful separation of responsibility. Of course, in practice there will be some interplay and communication, but directionally that provides a much cleaner scope of responsibilities.

Summary

The age of AI is here. Are you ready? Human guidance is the foundation of AI transformation. Data-centric AI is about creating new schemas, capturing new raw data, and doing new annotations to build AI.

To recap, some other key points we covered in this chapter:

- Appoint a leader like a Director of Training Data to drive your AI strategy, oversee data production, and interface with data consumers.
- Identify high-impact AI use cases using rubrics focused on frequency, experts, and new capabilities.
- Rethink your annotation talent strategy to leverage your existing experts as annotators.
- Implement modern training data tools for annotation, workflow, validation, and infrastructure.

Next I'll cover automations, and we'll look at some case studies.

---

[1] As a quick note on outsourcing, even if a percentage of the annotators are outsourced, there is still such a substantial budget involved that it's probably better to think of them as part-time or even full-time equivalents.

[2] I have also left off general use case/project management questions like “Scope acceptable relative to our resources?”

[3] Technically, this is the “higher dimensional space,” which we are unable to reasonably represent visually. Many machine learning models have hundreds of dimensions, while we can only reasonably graph four (space (x,y,z) and time (t)).

OceanofPDF.com

Chapter 8. Automation

Introduction

Automation can help create robust processes, reduce tedious workloads, and improve quality. The first topic I'll cover in this chapter is pre-labeling—the idea of running a model before annotation. I'll cover the basics and then step through more advanced concepts like pre-labeling just a portion of the data.

Next, interactive automations are when a user adds information in order to help the algorithm. The end goal of interactive automations is to make annotation work a more natural extension of human thought. For example, drawing a box to automatically get a tighter location marked by a polygon feels intuitive to us.

Quality assurance (QA) is one of the common uses of training data tools. I cover exciting new methods like using the model to debug the ground truth. Other tools automatically check base cases and look at the data for general reasonableness.

Pre-labeling, interactive automations, and QA tools will get you far. After covering the foundations, I'll walk through key aspects of data exploration and discovery. What if you could query the data and only label the most relevant parts? This area includes concepts like filtering an unknown dataset down to manageable size and more.

I will touch on data augmentation, common ways it's used, and cautions to be aware of. When we augment data, we derive new data based on the existing base information. From that viewpoint, it's easier to think of the base information as the core training data and then the deriving process—augmentation—as a machine learning optimization. So while a portion of the responsibility here is outside the scope of training data, we must be

aware of it. Simulation and synthetic data have situation-specific uses, but we must be up front about the performance limitations.

We have a lot to unpack and experiment with in this chapter. Let's get started by taking a closer look at the project planning process and techniques that are commonly used today.

Getting Started

High labor costs, lack of available people, repetitive work, or cases where it's nearly impossible to get enough raw data: these are some of the motivations to use automations. Some automations are more practical than others. I first outline common methods used, followed by what kind of results you can expect and not expect. This is rounded out with the two most common areas of confusion about automations—fully automatic labeling and proprietary methods.

I wrap this section by looking at the costs and risks. This section is a view of how the concepts map together, and ultimately of how they actually help your work. It can also help direct your reading and act as a reference to quickly look up common solution paths.

Motivation: When to Use These Methods?

When working with training data, you'll likely encounter problems that automation can help with. Table 8-1 covers some of the most common problems with automation-focused solutions.

Table 8-1. Reference of problem and corresponding automation solution

Problems	Solutions
• Too much routine work • Annotations too costly • Subject matter expert labor cost too high	• Pre-label
• Data annotated has a low value add	• Pre-label • Data discovery
• Spatial annotation is tedious • Annotator day-to-day work is tedious	• Interactive automations • Pre-label
• Annotation quality is poor	• Quality assurance tools • Pre-label
• Obvious repetition in annotation work	• Pre-label • Data discovery
• It is near impossible to get enough of the original raw data	• Simulation and synthetic data

Problems	Solutions
• Raw data volume clearly exceeds any reasonable ability to manually look at it	• Data discovery

Check What Part of the Schema a Method Is Designed to Work On

Automations work on different things. Some work on labels, some attributes, others spatial locations (e.g., box, token position), others more general concepts outside of the schema. For example, object tracking is generally oriented toward spatial information and doesn't usually help as much with the meaning (labels and attributes). When possible, I will highlight this, otherwise research the specifics in depth, as it usually comes down to the implementation details. As an example of this impact, a method that offers a 2x improvement on the spatial location will be of little importance if 90% of the human time is in the attributes, defining the meaning of what is actually present. This is relevant to all automations that impact annotations directly.

What Do People Actually Use?

With so many new concepts and options, it can be easy to feel overwhelmed.

While these methods are always changing, here, I highlight the top methods I have seen people use successfully and that are the most broadly applicable across all projects. The two questions I will most try to answer:

- What are the best practices?
- What do you need to know to be effective and successful with training data automations?

Commonly used techniques

These approaches may be commonly used; however, this does not mean they are easy, always applicable, etc.

You must still have sufficient expertise to use the approaches effectively—expertise you will gain in this chapter. These are the most frequently used techniques:

- Pre-labeling[1]
- Interactive automations
- Quality assurance tools
- Data discovery
- Augmentation of existing real data

Domain-specific

These techniques depend on the data and sensor configuration. They can also be more costly, or require more assumptions about the data:

- Special purpose automations, like geometry and multi-sensor approaches
- Media-specific techniques, like video tracking and interpolation
- Simulation and synthetic data

A note on ordering

In theory, there is some order to these methods; however, in practice these methods have very little universal order of usage. You can pre-label as part of a data-discovery step, during annotation, or much later to grade production predictions. Sometimes, data discovery is only really relevant after some significant percent of the data has been labeled. Therefore, I have put the more common methods toward the start of the chapter.

What Kind of Results Can I Expect?

Being a new area, it's common for there to be few expectations about what these methods can do. Sometimes, people invent or guess at the expected results. Here, I unpack some of the expected outcomes of using these methods.

Table 8-2 covers these methods. First, it shows the method itself. Next is the expected result when well implemented. And finally, the "Does not"

covers the most common issues and confusions.

Table 8-2. Overview of automation methods and expectations

Method	Does:	Does not:
Pre-labeling	• Reduces the less meaningful work. • Usually within a single sample, e.g., an image, text, or video file. • Shifts the focus to correcting odd cases. • A building block for other methods.	• Solves labeling entirely. • A human must still review the data and do further work with it.
Interactive automations	• Reduce tedious UI work. • For example, drawing a box, and getting a polygon drawn tight around an object in the box.	• Completely eliminate all UI work.
Quality assurance tools	• Reduce manual QA time on ground truth data. • Discover novel insights on models.	• Replace human reviewers entirely.

Method	Does:	Does not:
Data discovery	• Keeps human time focused on the most meaningful data. • Avoids unnecessary similar annotations.	• Works well in cases where there is already a well-functioning model and the main goal is grading it.
Augmentation	• Provides a small lift in model performance.	• Works without having base data.
Simulation and synthetic data	• Cover otherwise impossible cases. • Provide a small lift in model performance.	• Work well for cases where raw data is already abundant or situations are relatively common.

Method	Does:	Does not:
Special purpose automations, like geometry and multi-sensor approaches	• Reduces routine spatial (drawing shapes) work by using geometry-based projections. • Some limited methods can work largely independent of humans; others usually require labeling of the first sensor, and then the rest is filled in.	
Data-type-specific, like object tracking (video), dictionary (text), auto border (image)	• Reduce routine work by utilizing known aspects of the media type.	

Common Confusions

Before we dive into this, let’s address two of the most common areas of confusion, automatic labeling for novel model creation and proprietary automatic methods.

“Fully” automatic labeling for novel model creation

There’s a common myth that I hear repeated often: “We don’t need labeling, we made it automatic,” or “We automatically label to create our model (no humans needed).”

There are many techniques closely related to this, such as distilling larger models into smaller ones, or labeling specific portions of data automatically, etc. Many related techniques are explained in this chapter. These techniques are a far cry from “we completely automated data labeling for new model creation.”

Consider that when an original AI model makes a prediction (generates a label) that is different from creating that “original” AI model in the first place. This means that some form of non-automatic human labeling must be involved to create that original model. Further, GPT techniques, while again related, don’t yet solve this either for original model creation. A GPT model can “label” data, and you can fine-tune or distill a large model to a smaller one, but the alignment of the “original” GPT model still requires human supervision.

While this may eventually change, it is out of the scope of commercial concerns today. If we get to a place where we have true fully automatic labeling of arbitrary data for novel model creation, then we will have achieved the elusive idea of artificial general intelligence (AGI).

Proprietary automatic methods

“Use our method for 10× better results”—sometimes, this is a myth, and sometimes, it’s true.

The intent of this chapter is to convey the general methods and concepts that are available. As you will see, as these methods stack with each other, it is possible to get relatively dramatically better results than by purposefully going as manually and slowly as possible.

However, there are a vast number of concrete implementations. The single most common theme I have seen with secret vendor-specific approaches is that usually they are very narrow in scope—for example, it may work for only one type of media, one distribution of data, one spatial type, etc. It can be difficult to verify in advance if your use case happens to meet those assumptions. Statistically, it’s unlikely.

The best way around this is to be aware of the general approaches as explained in this chapter, and trend toward tools that make it easy to run the most up-to-date research and use your own approaches.

User Interface Optimizations

All tools have a variety of user interface–based optimizations. They are things along the lines of copy and paste and hotkeys. They are worthwhile to learn. As these are specific to each tool, and more of a "trivia" thing, I do not cover them in the book.

Outside the scope of "standard" UI expectations, there are interactive optimizations, which are specific named concepts and covered separately following.

Risks

All automations, even those operating as expected, introduce risk. Here are a few that are specific to annotations:

Lack of net lift

> This is when automation doesn't actually help. It is surprisingly common, especially with automations that involve user interface.

Worse results

> Automations can make results worse. For example, the super-pixel approach can lead to blotchy annotations that are less accurate than a traced polygon. Don't assume the failure state is equal to manual annotations.

Cost overruns

As detailed in the next section, automations have many costs, including time to implement, hardware costs, human training cost, etc.

Method-specific risks

Every method is unique and comes with its own unique risks.

Even seemingly similar methods can have different effects; for example, correcting a pre-labeled box is different from a pre-labeled polygon, and different again from an attribute classification.

Treating it too casually

Sometimes, these automations feel like just a little time-saver, or some kind of obvious thing. We have to remember that every automation is like a mini system. That system can have errors and cause problems.

Trade-Offs

Automations can sometimes feel like magic. Wouldn't it be nice if the system could just do *x* for us? This magic feeling shrouds some of these methods with an almost religious zeal. While many automations can and do improve training data results, they need careful trade-off analysis, planning, training, and risk analysis to be used safely and effectively.

It's important to note that most of these techniques have shifted over time, so they are now less about being incremental savings opportunities, but rather have become degrees of standard expectation. This is similar to how

a company that uses a word processor doesn't think of it as savings in comparison to a typewriter anymore.

It's quite incredible how quickly a dataset can be improved by a careful combination of automation methods. However, automations are not magic, and all have trade-offs. In this section, I'll provide some general rules to help evaluate some of the conceptual trade-offs of automations:

- The conceptual nature of automations
- Setup costs
- How to benchmark well
- How to scope the automation relative to the problem
- How to account for correction time
- How methods stack, which can also stack costs and risks

It's worth keeping in mind that no matter what automation is used (GPT included), subject matter expertise is still required.

Nature of Automations

The general guidelines for using automations is that the humans should always be adding true value with every annotation. If the annotation is too repetitive, it's a strong signal that a change is needed, and automation is an option among other options, like changing the schema.

Some automations leverage knowledge from another model, something akin to knowledge distillation. Now, we cannot hope to get a better new model by directly running an existing model in a loop, but we can leverage the existing knowledge to make the work of the annotator more focused on adding true net value. This is part of why automation can't "annotate" for the purposes of new model training, but can be a powerful assistant to a human.

It is difficult to scale a project without the use of some forms of automation. Avoiding automations entirely may make projects intractable financially, in completion time, or both. A project without automations may be hard to get beyond a basic threshold of getting started. An analogy here is having CI/CD processes. It's possible to work without them in theory, but realistically, you need them to scale your team and projects in a commercial setting.

Setup Costs

All automations require some form of setup. Setup can take a few different forms:

- Training on how to use them
- Implementation efforts
- Time needed to understand the technical risks

Even the ones with the most minimal of setup require a degree of training and understanding around their assumptions. While this may seem obvious, it's often glossed over, and must be remembered.

How to Benchmark Well

Previously, it was common to compare automations against manually completing 100% of the project.

Nowadays, completing an entire project with 100% manual annotation is rare. By manual annotation, I mean no model review, no pre-labeling, no UI assistance features, none of the methods in this chapter. Yet, automation methods are often presented as a percent savings relative to this mythical 100% manual annotation.

Instead of thinking of it as a comparison to manual annotation, think of it as comparing to best practices—and best practice is to use these automation methods appropriately.

There are a few benchmark questions that you can use to assess your organization's use of automation:

- Are we reasonably aware of all commonly available methods? Of the ones relevant to our case, what percent of them are we using? For example, if there are four methods available, and you are using all four, then at the strategic level, you are already there.
- By survey, find out how unique do annotators feel the work they are doing is? Do they feel they are making small adjustments or adding foundational value with every annotation? This can still be quantitative, but the point is to get away from lower-level metrics that are easily gamed or noisy.
- What is the ROI of our automation methods? For costs, consider all supplies (e.g., hardware), vendor costs, administration, and data science costs. For the benefit, if possible, try the actual next best alternative. This requires being clear-eyed about the comparison.

How to Scope the Automation Relative to the Problem

Imagine you have a lawn to mow. It's a small front lawn about the size of a parking spot. You can get a push mower or a ride-on mower. The ride-on, being about as big as the parking spot, is so effective it can pretty much just start up and instantly cover the entire lawn.

Naturally, the ride-on mower is orders of magnitude more expensive than the push mower, and it requires storage, supplies, gas, maintenance, etc. So while once in place, it will mow the lawn the fastest, its startup costs and ongoing costs far outweigh the benefits.

Setting a good scope of automations is important. A few questions to consider:

- Does the automation require technical integration? Can it be done from a UI or through a wizard?
- What are the expected startup and maintenance costs?

- Is an off-the-shelf method adaptable to our needs? Do we need to write a specialized automation?

If this sounds like a project management discussion, that's good, because that's what each automation is—a project. Obviously, a giant lawn mower is too large for a small lawn. So what are the right size metrics for training data?

Correction Time

While it might not always be factored in, time spent performing corrections should be accounted for. An automation that will cause a lot of need for corrections may not really save time in the end. If a spatial speed-up method is being used, try just drawing the polygon points directly by tracing the object. When you account for correction time, determine which method was actually faster.

I have seen research papers that reference "clicks." They say something like: We reduced the number of clicks from 12 to 1; therefore, it's 12 times better. But in reality, there is another tool that allows you to trace the outline, meaning that there are no clicks, and it's just as fast to trace the outline as it is to fiddle with the errors the automation makes.

As you can see in Table 8-3, the magic tool still takes some time to draw. In this case, it takes more time to correct, meaning that while it appears to be faster, it's in fact slower.

Table 8-3. Example comparison of time to draw versus time to correct

Tool	Time to draw	Time to correct
Trace outline	17 seconds	N/A
Magic tool automation	3 seconds	23 seconds

Subject Matter Experts

It's also worth understanding that virtually all of these methods still end up requiring a similar level of subject matter expertise. An analogy is that collectively, the tooling represents something akin to a word processor. It saves you having to buy ink, to mail letters, etc. But it doesn't write the document for you. It automatically formats the characters you instruct it to write.

To be clear here, requiring a subject matter expert does not mean the SME has to literally do every annotation themselves. They may instruct others on how to annotate specific sections, use tools, etc. Further, there may be cases where it's possible to contrast degrees of expertise.

Using SMEs does not mean you have to jump to the most expensive or most expert person in the world. As a concrete example, there may be many medical radiology annotations that can be done by a technologist, assistant, resident, etc. Referring to a radiology SME does not presume that it must be a full radiologist, or radiology professor.

Consider How the Automations Stack

Many of these improvement methods stack together. This means that you can use multiple methods together to achieve a positive effect, as detailed in later sections. The risk here is that, similar to taking multiple medications, the interactions and side effects are poorly studied. This can create hard-to-determine errors, make troubleshooting more difficult, and incur multiple costs.

Pre-Labeling

Pre-labeling is inserting predictions into training datasets. Pre-labeling is popular, and one of the most universally usable automation concepts.

Three of the most common goals of pre-labeling are to:

- Reduce the effort to improve the model (standard pre-labeling)

- Correct or align existing model results (QA pre-labeling)
- Add custom annotations on top of existing baseline predictions (custom data pre-label)

It is usually best to clearly define what is a pre-label and what is a human annotation, and to understand well the intent behind pre-labeling. One of the biggest risks with pre-labeling is a "feedback loop" where successive iterations increasingly generate more and more uncaught errors. Pre-labeling is usually focused on labels within a sample, for example, bounding boxes or segmentation masks in an image. Pre-labels can also be used for data discovery; see "Data Discovery: What to Label" for more.

Standard Pre-Labeling

Let's imagine we have a model that is sort of good at predicting faces. It usually gets good results, but sometimes fails. To improve that model, we may wish to add more training data. The thing is—we already know that our model is good for most faces. We don't really want to have to keep redrawing the "easy" examples. How can we solve this?

Pre-labeling to the rescue! Let's unpack this in Figure 8-1. First, some process runs, such as a model prediction. Humans then interact with it. We then typically complete the "loop" by updating our model with the new data.

Initially the user reviewing the data is shown all five predictions. A user may directly declare a sample to be "valid," or the system may assume that any samples not edited are valid by default. However it's marked, the net result is the second stage—four correct examples and one incorrect one that needs to be edited.

The annotator must still verify the correct samples. The verification time should be less than the time it would take to create it in the first place in order for this approach to be effective.

The general theme of this method is that over time, it focuses the human user on the hard cases, the "net lift."

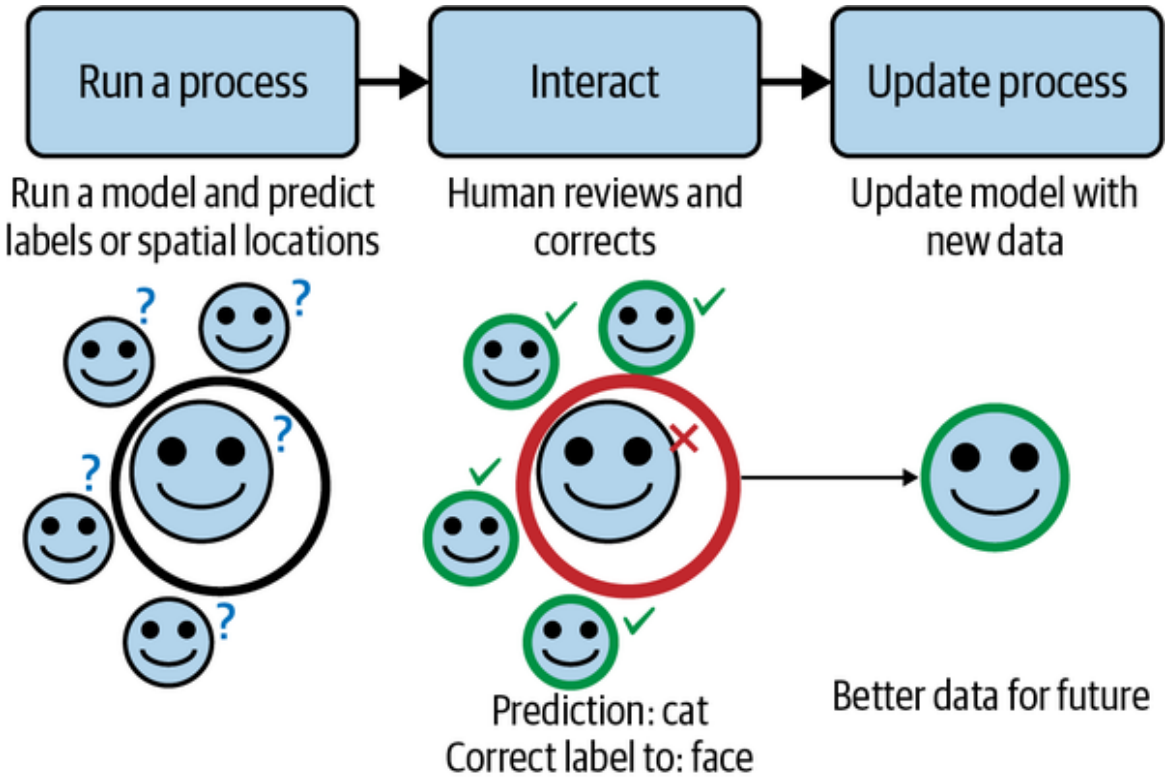


Figure 8-1. Pre-label overview

Benefits

Pre-labeling is popular because it works on nearly anything, is conceptually clear, and is relatively easy to implement. There are some clear advantages to pre-labeling:

Universal works on anything that can be predicted

> Standard pre-labeling is not tied to any specific method. It works across nearly any domain. Essentially, anything that a model, system, or existing process is generating can be used as a pre-label. This means it works on nearly any schema concept, from What something is to Where it is.

Conceptually clear and easy to implement

Pre-labeling is relatively easy to implement and understand. An existing system (usually a model) generates a prediction, and that's shown to the user to review, add more data, etc. Because these systems can generate the predictions at any time, and then be reviewed at a later time, it is relatively straightforward to store, load, and show this data.

Quality assurance is baked in (sometimes)

In some cases, such as when a model is generating a prediction, pre-labeling acts as a quality assurance control. Specifically, it ensures that people are looking at the actual predictions, as opposed to some other automation method where the entire process is more hidden. Making this data visible creates a relationship to the actual model that can be very valuable.

What's the difference between model quality assurance and pre-labeling?

There is a similar technical process to load the data, and the actual correction mechanisms are usually fairly similar in the UI. With quality assurance, generally the intent is to still improve future models, and this is again similar to pre-labeling.

Caveats

Pre-labeling still requires a system to generate the initial data, can create unexpected errors, can be misleading in terms of efficiency, and can potentially become confusing in more complex cases. There are a few more cautions to keep in mind:

Requires an existing relevant model (or system)

The primary caveat is that you need an "initial" model or system to generate the data. In general, pre-labeling is a "secondary" step in the process. This can be offset in that some methods and contexts require very small amounts of data—for example, maybe even a handful of images and you can start—but it's still a blocker. Transfer learning does not solve this, but it does help reduce how many samples are needed to start using this new model.

Can introduce new error modes, bad feedback loops, and diminishing returns

Instead of looking at data with "fresh eyes," it may be easy to miss something that "looks good," but actually is wrong. It can also reinforce errors through bad feedback loops. Pre-labeling can have diminishing returns when the schema difficulty is not well aligned.

Sometimes can be slower or misleading

In the context of literal annotation, editing may be slower than creating a new thing for some cases. If a model is predicting a complex spatial location and it requires lots of time to correct, it may be faster to simply draw it from scratch. Zooming out to a more conceptual level, pre-existing models can be misleading in terms of automation value. For example, an existing model predicts a fishing boat, because it was already trained on fishing boats. This may seem

> obvious, but it's best to be clear what data the model was already trained on to understand what is actual new work versus existing data. In the context of standard pre-labeling, it's best to only be correcting the models we plan to use, otherwise what's correct and not can become confused quickly. However, as I'll speak to next, you can also separate your data value add from the existing model.

Pre-Labeling a Portion of the Data Only

You can pre-label a portion of the data with a different model. The portion of data labeled could be things like a top-level label, or just the spatial location, etc.

Usually a model used for pre-labeling a portion of the data is not appropriate to be used as the actual primary model. This is because either it's too narrow (e.g., just spatial location, like segment anything) or because it's too large, compute intensive, or unpredictable, like a massive GPT model.

An extension of pre-labeling is to use a model to predict something it knows, and use it as a starting point for predicting something unknown. An example of this is dividing responsibilities (e.g., spatial information and attribute updates). For example, using an existing model that we don't plan to update for spatial predictions, and focusing our responsibility and effort on adding new information, e.g., attributes. In some approaches, a "micro model" gets discarded. The micro model is only used within the scope of annotation automation. This approach can help by saving time on the dimension it's good at, while the humans add meaning or other needed dimensions for the "real" model.

For example, imagine we have a face detector that's great at predicting faces but it doesn't know anything else. We can run that model first, get the spatial locations of the faces, and then add the new information, like happy,

sad, etc., as shown in Figure 8-2. While it may seem subtle, it's a big difference from standard pre-labeling. We aren't specifically trying to correct the spatial locations; we are strictly using it as a time-saver to get to labels and attributes.

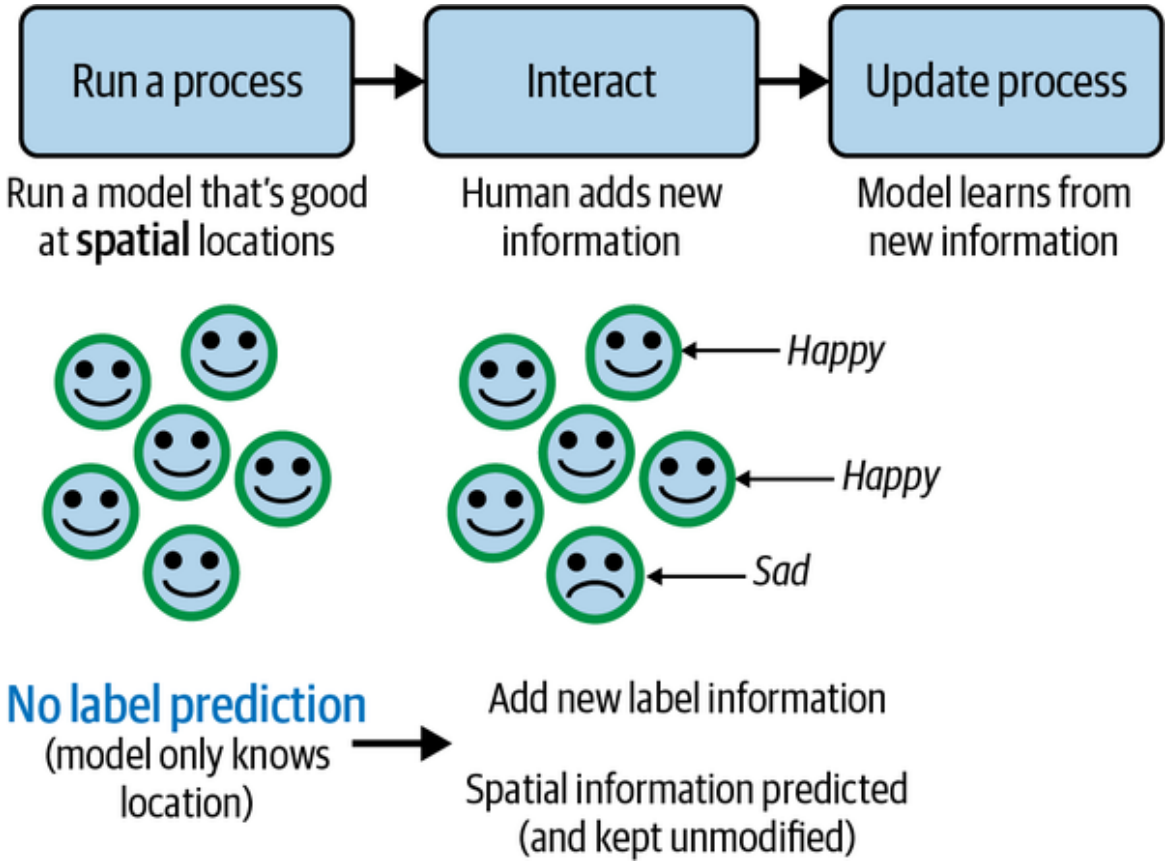


Figure 8-2. Pre-labeling a portion of the data only

Imagine a sports broadcast of a basketball team—identifying and tracking every player in each frame manually would be a lot of annotation work. There are fairly good "people detectors" that can run, as well as tracking algorithms, etc. These are responsible for determining the spatial location. Then, the human annotation can be focused on the relevant items, like what action the players are doing—things that a generic tracking/detector algorithm doesn't provide. Note that these approaches are also known as: micro model, custom data on existing model pre-label, and sometimes knowledge distillation.

Use off-the-shelf models

Because our goal here is to add new information (not to correct an existing model), we can use off-the-shelf models. This means that even if we don't have any training data for our Happy/Sad model, we can get started with an existing face detector. This can mean faster setup time. Also, because we aren't directly concerned about improving the model, if built-in models are available, they can be used directly without drawbacks.

Clear separation of concerns

Pre-labeling the specific portion of the data either helps speed up the process in the moment—or it doesn't. This is a clear separation of concerns relative to the more general pre-labeling, where the separation of concerns of in-the-moment annotation and overall data changes is more blurred.

An off-the-shelf model may not exist, or it may require upgrading to be useful. Sometimes they are not directly helpful for improving spatial detections or are not very interactive, and fairly static.

Note that some GPT-related pre-labeling processes may be less clear-cut or may have different trade-offs. This is a developing area.

The "one step early" trick

Speaking to the idea of automation more generally, it's possible to get a prediction "one step early." This might mean a generic object or event detection without knowledge of the class.

Typically, this is combined with some other method, e.g., pre-labeling.

How to get started pre-labeling

The data ingestion chapter covers some of this in more detail, but here I review the steps required to get started:

1. Identify your existing models.
2. Map the data from the model to the training data system.
3. Create tasks using the existing data.

4. Annotate.
5. Query and use the data to retrain new models.

Next let's talk about a more dynamic concept, interactive automation.

Interactive Annotation Automation

Interactive automations are used to reduce UI work. Conceptually, this is where it's clear to the human what the correct answer is, and we are trying to get that concept into the system faster. For example, a user knows visually where the edges of an object are, but instead of tracing an entire outline, the user guides an interactive segmentation system. Popular examples include SAM (Segment Anything Model), box to polygon (e.g., GrabCut, DxtER), and tracking algorithms.

These work best when a user interacts with them, usually by adding information, such as a region of interest, in order to help an interactive algorithm generate something useful. In theory, this makes UI work a more natural extension of human thought. Or put another way, as users provide "interactions" (inputs), then we can derive "interactive" speed-up approaches. Back to the semantic segmentation example, what if I could just point the computer toward what general area of the image I'm looking at, and it would figure out the specific shape?

Note that interactive automation is a developing area, and the examples shown are meant to be illustrative, not definitive.

A major assumption here is that the user is standing by, ready to add that initial information and then review the result of the interactive process. This process is shown in Figure 8-3, and is essentially composed of three steps:

1. User generates initial interaction.
2. Process runs (usually on the order of seconds of time).
3. User reviews results.

The key test to help compare interactive automation to other methods is that it should be impossible for the process to run without some kind of initial user input. In contrast, full image or full video methods can be run as a background computation without direct user input. This is the crucial difference. Another difference is that usually it's not the model of interest actually running.

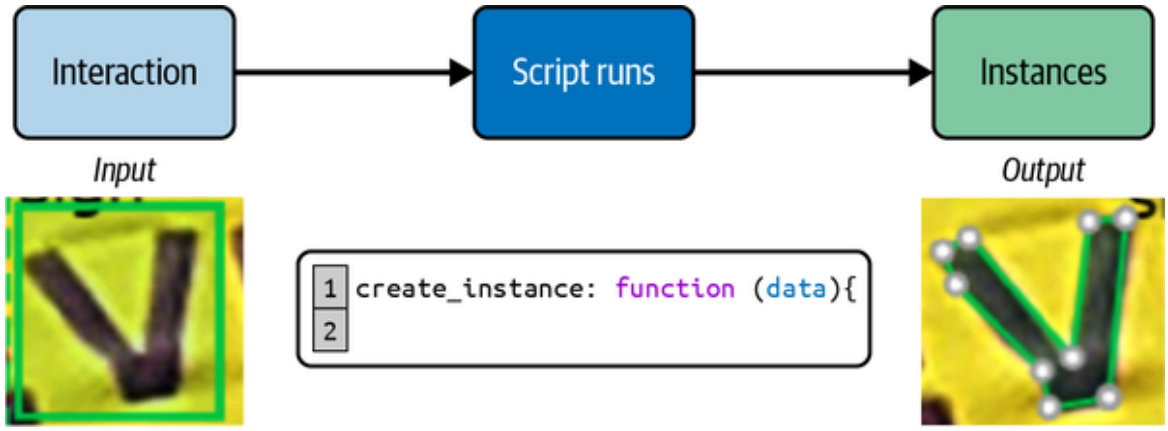


Figure 8-3. Conceptual overview of interactive automations

There are a few pros and caveats to consider when looking into interactive automation:

- Pros
    - For complex spatial locations, it can be extremely effective.
    - It usually does not require a trained model, or an existing model.
- Caveats
    - Can sometimes be "recreating the wheel." E.g., if you have a model that's good at detecting people, it's probably better to use that to start than to manually identify each person (even if it's a single click).
    - Often requires more user training, patience, etc. Sometimes, these methods can take so long to run that they don't really save much time.

- To really get these methods working well, it often requires some form of engineering work over and above wiring input/output.

Creating Your Own

Some tools provide integrated methods, or off-the-shelf approaches. For those tools, you may not have access to modify how they work. In those cases, the following examples will serve as a guide to the inner workings behind the scenes. Diffgram provides a compiler to enable you to write your own automations. This means you can use your own models, the latest open source models, adjust parameters to your desire, etc.

Technical Setup Notes

For the scope of these examples, I'm using open source Diffgram's automation library.

The example code is all in JavaScript. For the sake of brevity, most of the examples are pseudocode. Please see the linked examples for full executable code examples.

What Is a Watcher? (Observer Pattern)

Any time a user does something, such as creating an annotation, deleting an annotation, changing labels, etc., we think of it as an *event*.

We do so at a semantically meaningful level for training data.

For example, a `create_instance` event is more meaningful than a regular `mouse_click` if I want to do something—such as running a model—after the user creates the instance.

How to Use a Watcher

We must first define a function that will do something upon a `create_instance` event, shown below. After we enable this script, and the user draws an example annotation, we see the example output in Figure 8-4:

```
create_instance: function (data){ // your function goes here }

create_instance: function (data){
    console.log(data[0])
}
```

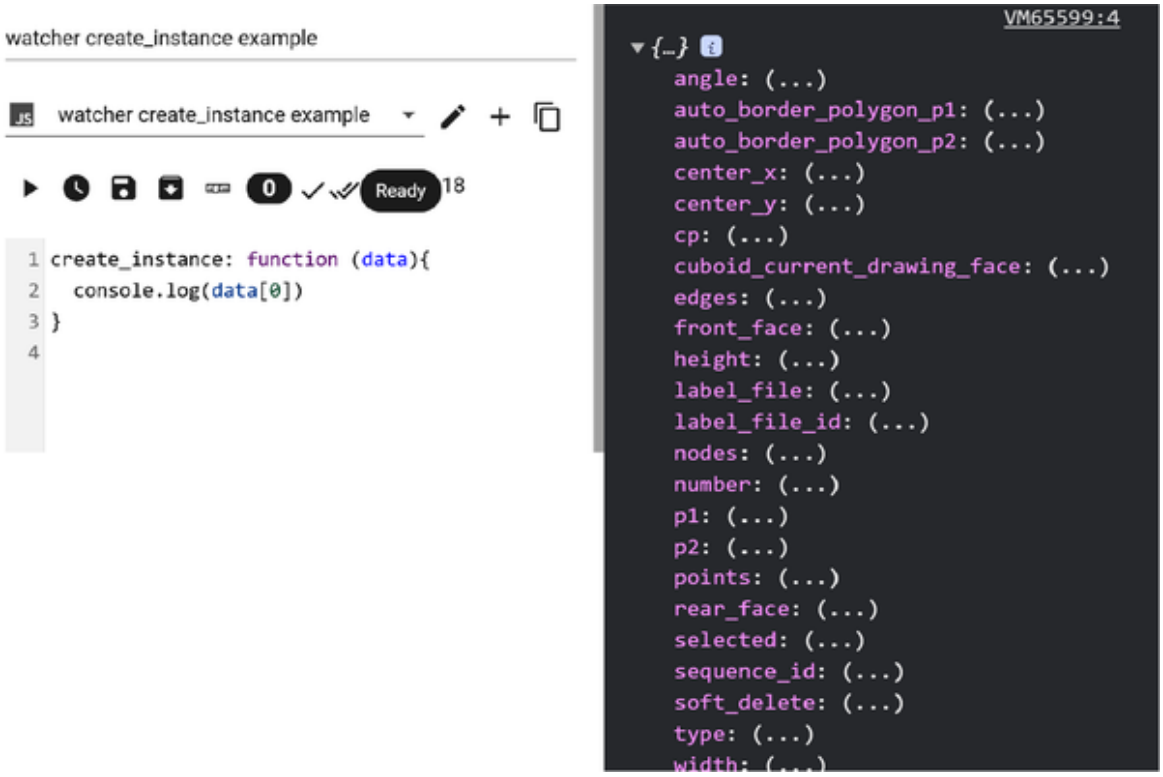


Figure 8-4. Left, example of code editor for interactive annotation; right, logging annotation instance created

Interactive Capturing of a Region of Interest

Here is an example of code that captures the canvas region of interest based on a user's annotation. While this may seem fairly simple, it's the first step toward running an algorithm based on just the cropped area, as shown in Figure 8-5.

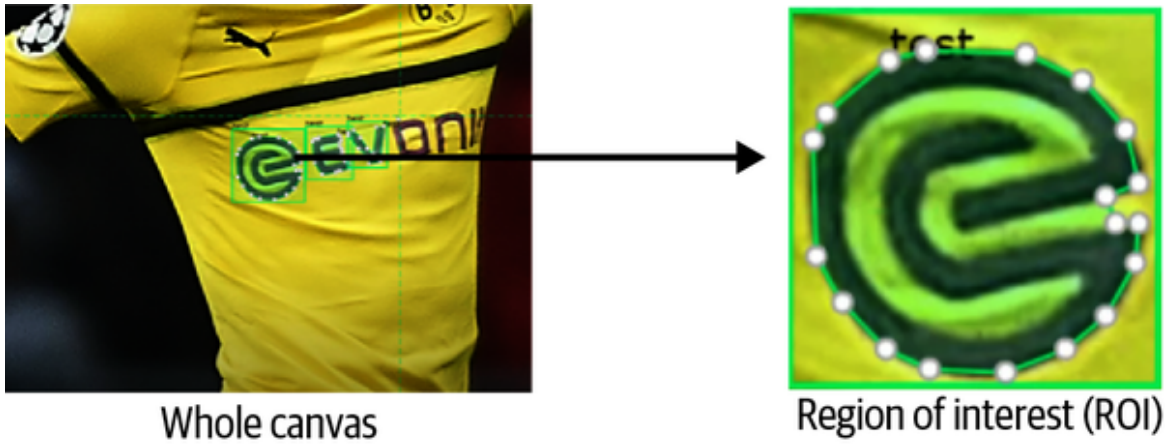


Figure 8-5. Example of whole canvas versus region of interest

The code to capture this looks like this:

```
create_instance: function (data){
  let ghost_canvas = diffgram.get_new_canvas()
  let instance = {...data[0]}
  let roi_canvas = diffgram.get_roi_canvas_from_instance(
    instance,
    ghost_canvas)
}
```

More generally, the idea here is to use some input from the user to preprocess what we feed to our model.

Interactive Drawing Box to Polygon Using GrabCut

Now, this example will take our region of interest canvas, run the standard OpenCV GrabCut algorithm and output polygon points, which will then be converted into a human-editable annotation.

Crucially, you can replace GrabCut() with a preferred algorithm of your choice. Here is the simplified pseudocode (JavaScript):

```
let src = cv.imread(roi_canvas);
cv.grabCut(src,//args)  // replace with your model or preferred algorithm
cv.findContours(//args) // because it's a dense mask
cv.approxPolyDP(//args)  // reduce to useful volume of points
points_list= map_points_back_to_global_reference()
```

```
// because roi is local, but we need the location relative to whole image
diffgram.create_polygon(points_list)
```

If you'd like to view the full example, you can find it here.

Full Image Model Prediction Example

Zooming out from the directly interactive example, we can also run file-level models. You can imagine having a series of these models available, with the user perhaps choosing at a high level which one to run.

The high-level idea is to get the information from the user interaction and the raw media, and then run the automation.

An example in JavaScript could look something like this:

```
this.bodypix_model = await bodyPix.load()
let canvas = diffgram.get_new_canvas()
let metadata = diffgram.get_metadata()
segmentation = await this.bodypix_model.segmentPerson(canvas, {});
Points_list = get_points_from_segementation()
diffgram.create_polygon(points_list)
```

The full example is available here.

Example: Person Detection for Different Attribute

Now, going back to a more simple example, we can also use these automations to simply run models.

Here, we used the BodyPix example in a Diffgram userscript. It ran on the whole image and segmented all the people. Here, we now select the person to add our own attribute, "On Phone?" as Shown in Figure 8-6. This is an example of using a model that's not directly our training data goal to avoid having to "recreate the wheel" for parts that are well understood. You can imagine swapping this model for other popular ones, or using your own models for the first pass here.

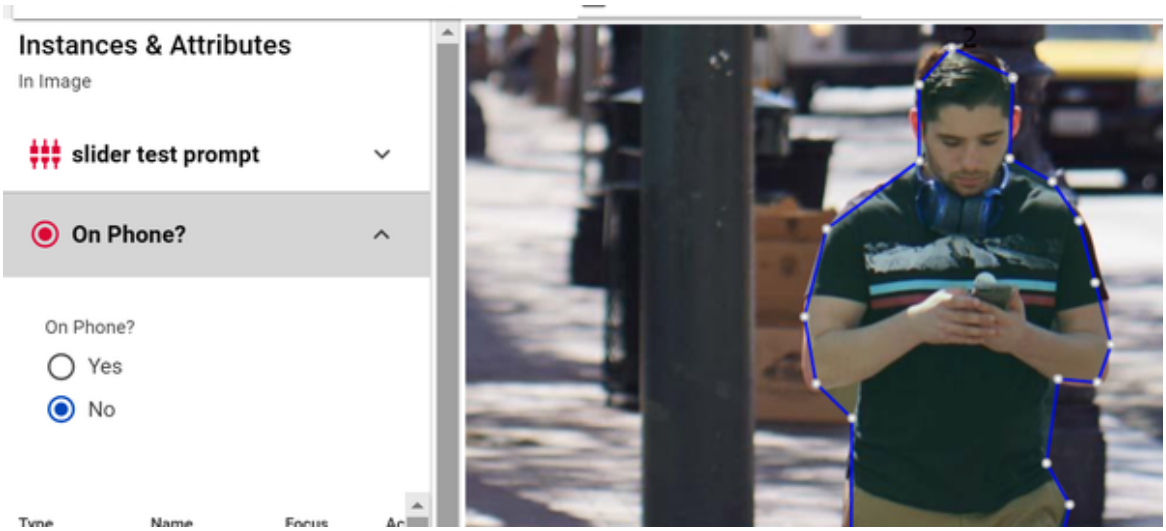


Figure 8-6. Example of spatial prediction automation with attribute value add

Quality Assurance Automation

There are novel training data quality assurance (QA) tools to debug ground truth data and reduce quality assurance costs.

Using the Model to Debug the Humans

The big idea here is that models can actually surface ground truth errors. Sounds crazy? So, the way to think about this is essentially that if a model has many examples of truth, it can overcome small errors. You could compare it to the intuition that sometimes allows us to spot errors on a quiz answer sheet because even though it says it's the answer, we think about it and feel confident that it's not the answer.

In practice, one way to do this with images is to use Intersection over Union (IoU). If the model prediction doesn't have a nearby ground truth, then either the model is wrong or the ground truth is. This can be surfaced by applications as a ranked list. Or put another way, this is about identifying the biggest delta between predictions and existing ground truth.

Automated Checklist Example

While this may sound simple, a checklist is an essential part of the QA workflow.

This can literally be a checklist for human review, or if possible, it can be programmed. Some tools may include some of these by default, but often the best checks are set up more as “test cases,” where you define parameters relevant to your specific data (like a checksum). Here are some example parameters:

- Is the count of samples reasonable?
- Are the spatial coordinates reasonable?
- Are all attributes selected or approved by a human?
- Are all instances approved by a human?
- Are there any duplicate instances?
- (Video specific) Are there tracks where an instance is missing (e.g., in frame n, missing in frame n+1, reappears in frame n+2, and not marked as end sequence)?
- Are there issues with dependency-type rules (e.g., if class A is present, then class B should also be present, but class B is missing)? This is different from heuristics in that it’s usually more about reviewing existing instances than generating new ones.

Domain-Specific Reasonableness Checks

Depending on your domain, you may be able to use or build specific checks. For example, if the class is a “person,” and 95% of the pixels in the spatial location of the sample are all a similar color, that’s unlikely to be correct. Or, you can check to see if the majority of pixels are in a different histogram distribution than what’s expected (e.g., an object that is known to never have red pixels).

Data Discovery: What to Label

Data discovery tools can help us understand existing datasets, identify the most valuable data to add value to, and help us avoid working on data that is showing diminishing returns. This goes back to the concept of wanting every annotation to provide net lift, to make a positive incremental improvement in the model performance. In this context, “rare” or “different” data is simply any data that will help get you net lift.

Conceptually you can broadly think of these methods in three main categories:

- Human exploration
- Metadata based
- Raw data based

Human Exploration

All methods involve some form of human review at some point. Common human data discovery steps include the following activities:

- Visually exploring data in a UI catalog
- Querying data to identify subsets of interest (for example, “Carrots > 10”)
- Reviewing the results of tooling (e.g., similarity search)

Human exploration happens at multiple stages in the process. This can be at the stages of raw data without annotations, data that has some annotations and may need more value added, subsets of data, and results from various tooling practices.

Human review is critical to catch mistakes in automation. Automations may make faulty assumptions about the data. For example, if you have a limited amount of data, reducing it in any way may cause unwanted results. While

automation approaches for data discovery can be a very important part of the process, human understanding is always the first, second, and third step.

Raw Data Exploration

Raw data exploration methods are all about looking at the actual raw data.

An example is similarity search. One use case is to find new samples of a specific known sample type. This could be in the context of knowing you are low on annotated data similar to a given sample. The process is to run a similarity search on that sample to find more samples that are similar (but not yet annotated). Then, you create tasks to review those samples.

Another example of raw data exploration is identifying groups of data. For example, imagine you had both daytime and nighttime pictures. You could use a raw data exploration tool to look at that data and segment it into two batches. The system may not actually know that it's day and night pictures, but it will know by looking at the raw data there are two (visually) distinct groups. You can then sample from each, for example, 100 daytime samples and 100 nighttime samples. This is to avoid getting 1000 samples of a similar distribution as that shown in Figure 8-7. A good time to use this method is when your data contains many similarities and exceeds any reasonable ability to manually look at it.

Another use is identifying outliers. For example, you might flag rare samples that may not have enough volume to be understood correctly.

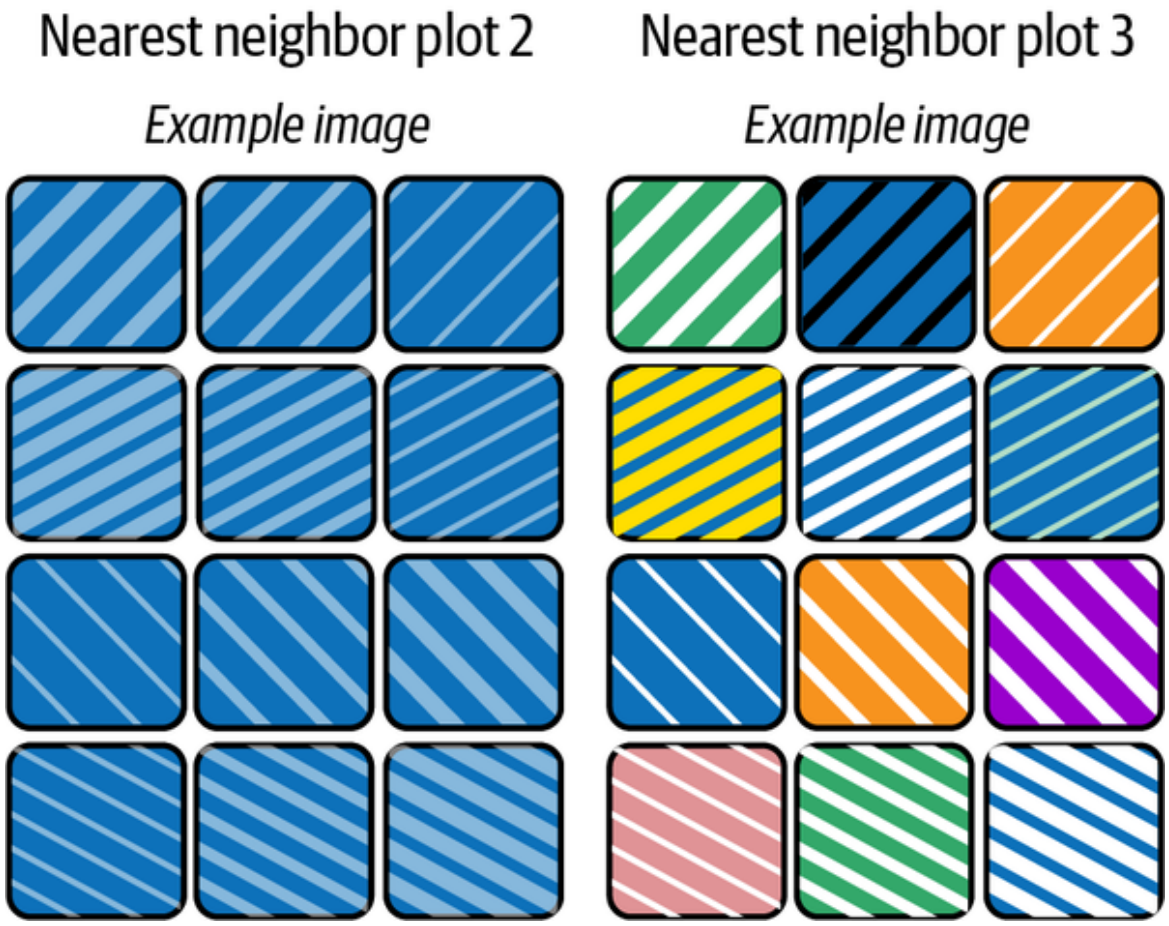


Figure 8-7. Example of similarity comparison

Metadata Exploration

Independent of the literal raw bytes of the samples, there is often attached metadata about the sample. For example, if you want to do something like this automotive company does in Figure 8-8, you can query based on metadata like the time of day, microscope resolution, position, etc.

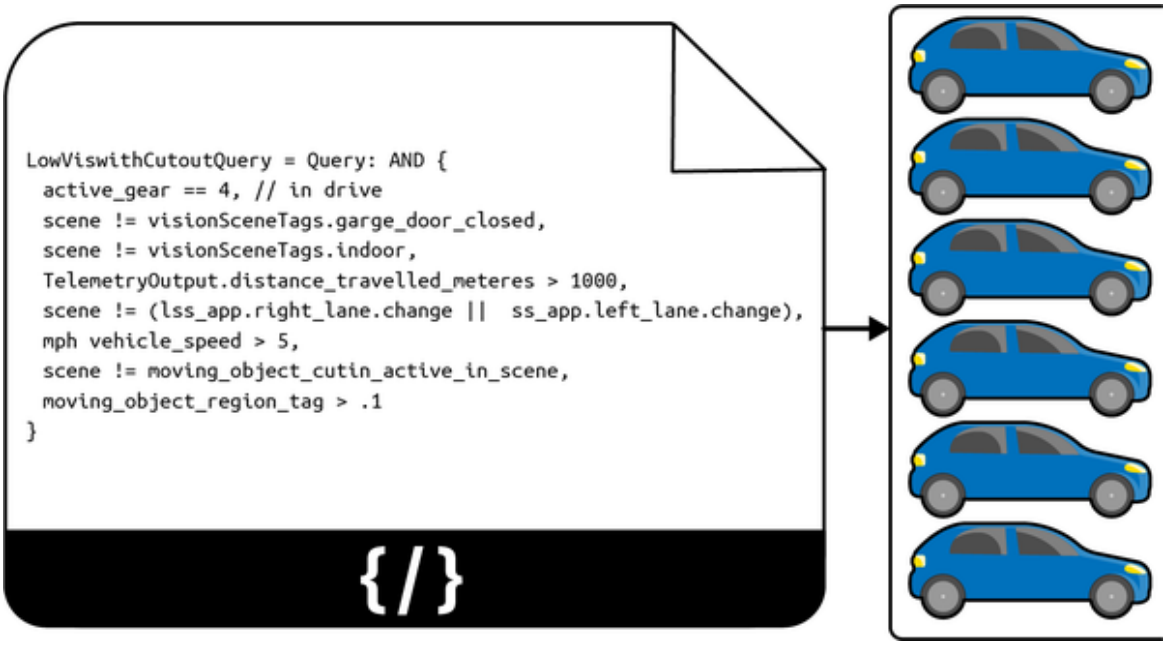


Figure 8-8. Example of query (pseudocode)

Make sure to add the metadata during insertion so it can be queried well. The example in Figure 8-8 is pseudocode. Diffgram has a good resource if you'd like to explore query syntax reference further.

Adding Pre-Labeling-Based Metadata

An option is to use an existing network to pre-label at a course level, for example at the whole file level. For example, for a robotics company, say the intent is to label only "daytime" images, but for some reason the metadata of the data is unavailable. You can run a network to tag images as "nighttime" or "daytime," then only label the "daytime" images. In practice, there can be some rough edges to implement this, but in theory this can dramatically save time.

Augmentation

Augmentation is modification of real data. For example, skewing existing images, changing the brightness, introducing artifacts, etc. Augmentation is an active area of research, and opinions vary. Sometimes augmentation is proposed as a way to reduce training data effort. For example, popular

libraries and processes often do a heavy amount of data augmentation, but what actual lift this is providing is sometimes overlooked. Generative models can also be used to augment data, and this is a rapidly developing topic at the time of writing.

To put it simply, augmentation is often a temporary crutch, and/or provides a relatively small lift, or even a decrease in performance. Think in terms of –10 to 10%. It can still be useful, and it's good to be aware of when and how it can be used.

In general, there seems to be a consensus toward doing augmentation at runtime, and more minimally than during the exuberance of the early approaches. The implication of this is that for most cases, it's not something for you to overthink, the same way we don't think about compiler optimizations for regular code.

Augmentation has its place, but we must remember that it's, at best, a small performance boost, not a replacement for real data.

Better Models Are Better than Better Augmentation

One general theme is that as the models and algorithms get better, augmentation becomes less effective. This may not always be true, but is a good rule of thumb.

A simple way to think of this is that there are nearly infinite subtle combinations of data values, be it text, pixels, etc. So, trying to train an ever-growing amount of subtly different data just to have a better understanding of what it is is a losing proposition.

Think about it from the human perspective—I don't need to see 100 different variations of essentially the same thing in order to recognize it.

To Augment or Not to Augment

The first thing to appreciate with any form of augmentation is that it significantly alters the playing field.

Next, the best evaluation of any augmentation method is what “net lift” it provides. This is sometimes trickier to measure than is first apparent.

For example, imagine a system that pre-labels instances. The spatial location is pre-labeled reliably. Expert users then add additional information, such as what type of crop disease it is. The “net lift” in this case is primarily time savings. So if the cost (literal cost, complexity, etc.) to set up the pipeline, run the model, etc., is low enough, then every spatial location that’s pre-labeled is a direct time savings. In practice, if a robust model is available for this, or is already being run in production, then this is valuable.

Alternatively, let us consider some negative scenarios.

An approach (e.g., superpixels and similar) is used to more “quickly” label spatial locations. Except, it introduces artifacts and hard-to-correct errors. It’s easy for it to look “OK” at a glance, so these errors go unnoticed until significant time is spent on the dataset. This may cause the need to rework and/or hard-to-understand production errors.

A pre-label approach that requires constant correction can also be problematic. Correction is ~3x slower than just drawing the spatial location correctly the first time. So, in the abstract, the pre-label must be very accurate to break even, and more so to provide net lift.

Any scenario in which an “external” model is used can cause difficulties. It can introduce bias into the new training data, polluting it, so to speak. As a practical example, imagine a person detector that appears to reliably detect people. It’s used as the “spatial location” first pass, to which your customer labels are then added. If that detector has a bias, perhaps favoring an ethnic majority, it may be very difficult to spot, since it’s “hidden.” You may need an extra QA step expressly to check for this potential.

At a high level, the general rules of thumb for these augment approaches are:

- Explore with caution—there are always trade-offs, and remember that augmenting methods hide the trade-offs well.

- Consider the “net lift,” taking into account costs such as the added complexity, reduced flexibility, literal compute/storage costs.
- Many of the approaches are very specific—they may work very well for one case, of one spatial type, etc., while failing on many others.

I have a few other general observations to keep in mind:

- The most effective options generally seem to favor spatial types over class labels.
- Generally, it’s best to create a “seed” set, before exploring any type of “model”-based approaches there.
- Sometimes “augmented” training data is somewhat hard to avoid, e.g., in the case of “correcting” a production pipeline.

Training/runtime augmentation

As long as the augmentation method and process are reproducible, you may not need to store it alongside regular training data. Augmented data can be very “noisy” and is not typically that useful for human consumption. However, even when using readily available tools, implementing augmentation at training time comes with challenges, for example, memory and computational costs. In this context, moving it to runtime isn’t always straightforward.

Patch and inject method (crop and inject)

The concept here is to take patches—meaning crops or subsections—of the data and create novel combinations. For images, you can imagine placing rare classes in a scene. This is sort of like a hybrid simulation where real data is simulated in different places.

This is still an area of research. In general, a model should be robust to these types of scenarios.

Simulation and Synthetic Data

Closely related to augmented data is simulation and synthetic data. Simulation and synthetic data techniques depend heavily on domain-specific contexts. Often, they are a blend of other arts and disciplines outside of training data.

All synthetic data methods still require some form of real data and human review to be useful. This means synthetic data shows promise, but is not a replacement for real data. That's why it's best thought of as the cherry on top of real data.

Let's get the big question out of the way real quick—does simulated data work? The super-short answer is "Yes, but probably not as well as you may be hoping."

The next question is, "Will it eventually work really well?" This one is actually easier—it's unlikely. And there's a simple thought experiment to reflect on this. To simulate the data, to the level of realism that is required, is on the artificial general intelligence level of hard problems. It's relatively easy to get photorealistic looking renders—but there's a big difference between those simulated renders and real life.[2]

Typically, simulated data, if used well, can provide a small lift in performance. It can also be sometimes used for situations where it would be impossible, or very difficult, to get the data beforehand.

Another way to think about this is that using a simulation to automatically create training data is somewhat akin to using a heuristic. Instead of humans supervising and saying what right is, we are back to trying to build a bottom-up type model—just one level removed.

Simulations Still Need Human Review

Shown in Figure 8-9 is an example from a major company that they presented publicly. They claimed it was "perfect" data. As you can see, there is a patch over the crosswalk—part of what they touted as being more realistic—but then they forgot to account for that in the training data generation, generating perfect crosswalk lines over the imperfect pavement.

Were they trying to teach it that the black splotch was actually a crosswalk? This doesn't make sense. Of course, perhaps they were meaning to post-process it later to *project* a perfect crosswalk. And perhaps, some super-specialized network could theoretically do this. But at least from the training data perspective, this is just a plain error.

Figure 8-9 shows just a small example, but it gives the idea that assuming that a simulation will render perfect data is far from correct.

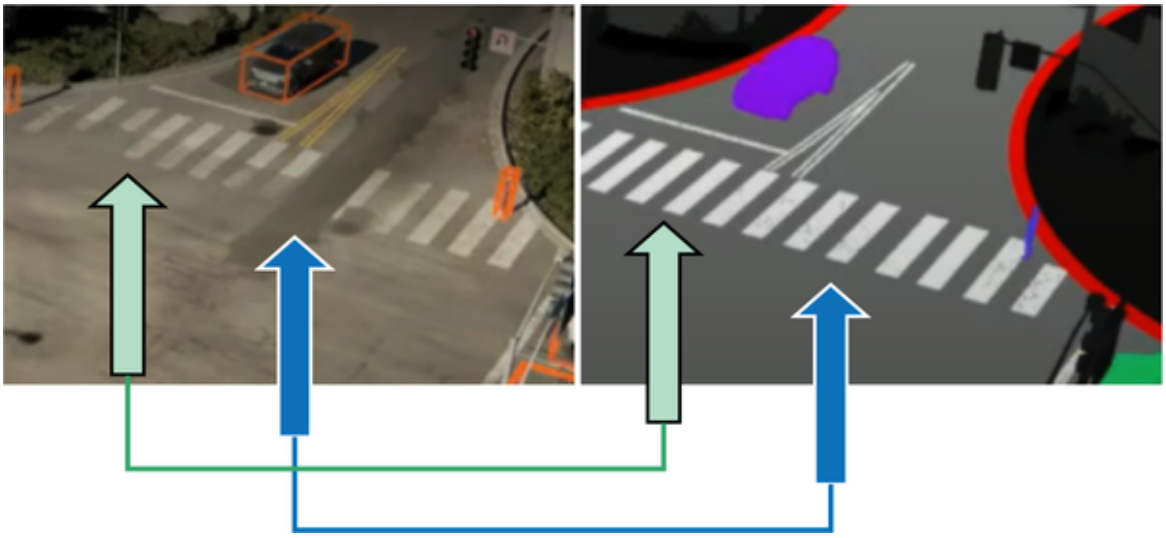


Figure 8-9. Example of training data that was claimed to be perfect but has clear labeling mistakes

In Figure 8-9, the left image shows the simulated image with faded white lines, and some white lines obscured entirely (by what appears to be pavement). The right image shows a training data rendering that has annotations that guess at where the line should be, ignoring the obscured and missing lines. If this data was actually trained on directly, it would be misleading to the model, since we would be saying that patch of pavement equaled a complete line, when clearly no line is visible.

For the impossible and rare cases, I would really think of it less as "automatically creating training data" and more as "automatically creating scenes, for which humans can then create training data."

For example, a system may be able to automatically identify which pixels are from which object. But, if the simulation is sufficiently random, that

may not be relevant for events. In other words, it’s important to think about what the simulation actually *knows.*

A way to think about this is if I have a simulation of shelves at a supermarket, those shelves will look similar, unless the simulation is engineered specifically to render different shelves. But what dimension will they be different in? Will those differences be relevant to the production data? Who is programming that in? Remember that even subtle shifts in the data can have huge effects on the result.

There are some clearly beneficial uses of simulation:

- Product images. If you already know what the product looks like, that should be a clear enough case for a shopping-type robot.
- Rare scenes of autonomous driving.

And there are some overall pros:

- Simulations can improve performance.
- Simulations can create otherwise impossible or rare cases.

And cons:

- The improvement is usually only by a small amount; think on the order of a relative 0–10%. There may be case-specific exceptions to this, so it’s worth some research.
- Often, the fidelity of simulations is a lot further from the real world than may first appear. For example, what looks visually good in a video, may be drastically different pixel for pixel.
- Simulations are often costly to set up, maintain, and operate.

There are a few things to think critically about when considering simulations:

- What are we actually simulating? Are we simulating lighting conditions? Camera angles? Whole scenes?

Media Specific

Many automation methods work well for all media types. Here, I cover some of the highlights of how popular methods intersect with specific media types. Then I cover some common domain-specific methods. This does not cover all media types. The main intent of this section is to give you awareness of the relationship between specific media and automation types and provide an introduction to some of the domain-specific types.

What Methods Work with Which Media?

As you can see in Table 8-4, generally, most of these methods can stack well together. Of course, each method takes a degree of work and understanding. There is no requirement to use all of them. You can have a very successful project that completely skips a lot of these.

A ✔️ indicates that usually the method will match. All methods require some degree of setup and have exceptions and cases where they may not work. For example, using pre-labeling with data discovery approaches for images is better defined than for video. I have included more extensive footnotes here to expand the rationale where required and keep the chart easy to read.

Table 8-4. Media formats commonly used with each technique

Methods/data type	**Video**	**Image**	**3D**	**Text**
User interface best practices	✔️	✔️	✔️	✔️
Pre-labeling	✔️	✔️	✔️	✔️
Interactive automations	✔️[a]	✔️	✔️	Some
Quality assurance automations	Research area	✔️	✔️	✔️
Data discovery	Sometimes[c]	✔️	Research area	
Augmentation[d]	Sometimes[e]	✔️	Research area	Rese
Simulation and synthetic	✔️	✔️	✔️[f]	Rese
Media specific[g]	Varies—check method			
Domain specific Special purpose automations, like geometry and multi-sensor approaches	Few commonly used methods	Sometimes	Few commonly used methods	Few comm used

Methods/data type	Video	Image	3D	Text

[a] May take more effort to set up.

[b] There are a variety of “real-time” model training concepts, but often this is more single-us

[c] If the method requires conversion to images, this may prevent the use of video-specific au of discovering segments, and keep the video form, as it may improve compatibility.

[d] Augmentation methods tend to be domain specific. Opinions vary.

[e] Many augmentation methods are focused on images, so may not be applicable for event de prediction tasks.

[f] Because most simulations are 3D by default.

[g] Like object tracking (video), dictionary (text), auto border (image). Because the methods a double-check when choosing a specific method.

Considerations

Just because you are using a certain data type, e.g., video, doesn’t mean that an automation will be available for that case. For example, maybe you have no problem identifying people in videos, but identifying specific interactions may be harder.

All tools have a variety of UI concepts that can help improve performance, including common things like hotkeys, copy and paste, etc. These methods sometimes get confused with other things, so I think of this as a separate category.

Media-Specific Research

An important thing to keep in mind is that some methods are more or less established for certain media types. For example, something that may be considered “standard” for images may be barely researched for text, or not

even have a direct analog. For some people, this is not obvious because they may focus on one media type only (e.g., NLP or computer vision):

Video specific

If you are interested in *event detection*–type actions, some of these automations are not recommended. For example, object tracking would confuse a situation where you want to know exactly in which frame an event occurs:

- Object Tracking: The big idea here is to look at the data itself, and track an object throughout multiple frames. There are a variety of tracking concepts, many need only a single frame to provide an effective track forward through time.
- Interpolation: The idea here is humans create keyframes and the in between data is filled in, as shown in Figure 8-10.



Figure 8-10. Example of interpolation

Polygon- and segmentation-specific—auto bordering

I covered how to use auto bordering in Chapter 3. Here, I briefly want to state that using techniques like that to snap to edges, as simple as it may seem, is a great use of practical UI automations.

Text (NLP)-specific

> Pre-labeling with GPT, heuristics (covered earlier in this chapter), and dictionary look ups are three of the most common methods.

Domain Specific

Many domains have specific approaches that work well for that domain only. For example, for real-world robotics, there is 3D geometry that often matters, whereas that type of geometry usually isn't relevant for all digital PDFs. There are many domain-specific options that can be implemented well. For instance, if you have multiple sensors, you can get a "two for one" deal by labeling one sensor, and using known geometry to estimate into the other sensors. Incorporating interpolation and object tracking can speed up video. Using dictionaries and heuristics can help with text. I'll walk through all of these at a high level so you are confidently aware of them.

Geometry-Based Labeling

In some cases, you can use the known geometry of a scene, sensors, etc., to use geometry-based transformations to automatically create some labels. This case is highly dependent on the specific context of your data.

Multi-sensor labeling automation—spatial

The main idea here is to use mathematical projections to assume where something is in space based off of where the sensors physically were. For example, if you have six cameras, it can be possible to construct a virtual 3D scene. Then, you can label one camera, and project that into the other five. Often this means projecting into 3D and back into 2D. Note that this does *not* require a 3D-oriented sensor like LiDAR or radar. But in general, it's considered easier if one of those sensors is present.

In a theoretically optimal case, you get something like a 5:1 return. E.g., for every one camera you label, you get labels in five other views. Keep in mind that you still need to review, and often correct, the projections, so in reality the return is more like 3:1 at best. Keep in mind it requires having multiple sensors, extra metadata, and the ability to project into 3D.

Spatial labeling

This is similar to the multi-sensor labeling, but is more focused on cases with a large extent of geometry-focused concepts, like lane lines.

Heuristics-Based Labeling

In general, heuristic labeling has been NLP-focused, and concentrated in two main areas:

- Dictionary-based labeling
- User-defined heuristics

This is a controversial area. If you have written a perfect set of heuristics, then why even train a machine learning model at all? The main problem is that the more well-defined a heuristic is, the more it looks like we are coding and doing work there, and basically reinventing feature engineering. Since one of the whole points of deep learning–based approaches is automatic feature engineering, this essentially is self-defeating. Now, this is not to say heuristics-based methods are without merit—far from it. There is some incredibly interesting research in this area, especially for text-based applications.

Summary

The theme, time and time again, is “Humans.” From high-level data discovery tools, to framing specific metadata queries—human review takes center stage. When thinking about how to construct automated quality assurance flows and reasonableness checks, it’s all about human knowledge

when doing interactive automations to extend human thought. When humans are viewing and adding value to existing predictions with pre-labeling. When you, as the humans, are evaluating the trade-offs and risks of automations. There's always something to automate, and always more human steps needed to set it up, control it, monitor it, and maintain it.

You have learned an incredible amount of practical knowledge in order to understand the high-level risks of automations, the rewards seen of common methods, and many specifics, from media types to domains. You have had both a relatively comprehensive overview and the details to conduct further research for your specific challenges. Next, I'll dive into real-world case studies to help tie up all the concepts you have learned so far.

Overall, there are many high-quality approaches that will substantially automate your training data process. There are also limitations and risks to consider when implementing automation. I provided a high-level overview of the most common methods you can actually use today, what results to expect, the trade-offs involved, and how they work together. For the most popular methods, you now have a deeper dive into some of the specifics.

Next, let's dive into real-world case studies.

---

[1] For experienced folks, grading production predictions is also rolled into this concept.

[2] Anecdotally, I have run this by both research- and industry-oriented people, and there's usually common agreement that it's true.

OceanofPDF.com

Chapter 9. Case Studies and Stories

Introduction

Now I will share some exciting case studies and stories. I have aimed to provide a breadth of views, from engineering to annotation quality assurance. Stories from big companies to multiple sizes of startups, and education-centric data science competitions.

You may be just starting your AI path, or you may already be knowledgeable. No matter where you or your team is on your AI journey, these studies have been carefully selected to provide value and key insights. Each study will help you understand how others have implemented AI in the real world. Each study will be different, with some being more verbose and others being more brief anecdotes.

Some of the studies will be simple stories. Others will be more detailed and open, with the high-level concepts to set the level of technical depth and perspective of the scenario from the outset. In these more detailed ones, I'll point out some of the nuances that make each example relevant and will also cover lessons learned that you can take away and incorporate into your organization.

One of the biggest recurring themes is how *new* modern training data is. Imagine we are at the dawn of compilers becoming common. Compilers convert high-level code to machine code, so if someone presents a way to manually optimize code,[1] it wouldn't be very useful to a new organization that has access to compilers.

This newness brings many challenges. There are often gaps in knowledge. Case studies can be very hard to come by because of lack of volume, relevance, or confidentiality. This chapter aims to provide new resources,

modified and anonymized, but still based on real stories. One caveat is that since this is a rapidly evolving space, it's fully expected that some specifics may have changed since the case occurred.

For simplicity, I've made up a company name, a fictional "X Company," to reference companies in the case studies with ease. I'll change X to other letters for each study. The letter, e.g., "Y," will be whichever company is in focus in the case study, the real story behind every company is different, and this will help us keep that in mind. All examples in this chapter have been made general purpose to reflect only industry-level knowledge and respect confidentially. Any resemblance to a specific company or context is not intentional, and/or the work is in the public domain.

Industry

All of these in Industry stories and case studies will help reinforce the AI transformation concepts described in earlier chapters. I will provide a wide range of views, from the high-level organizational perspective to the deeply technical team level.

As a preview I'll cover:

- A security startup adopts training data tools
- Quality assurance at a large-scale self-driving project
- Big tech challenges
- Insurance tech startup lessons
- Four brief additional stories

Let's get started.

A Security Startup Adopts Training Data Tools

This story is about a large security startup that found that adopting a training data platform yielded benefits across virtually every team.

Annotators were using an in-house solution that had a bunch of manual forms. They switched to a tool that moved all those forms into a single search box. By being able to search thousands of attributes, across what was previously five plus separate forms, and by having the pre-data loaded when possible, the velocity to annotate improved dramatically. Their internal benchmark showed the improvement to go from minutes per annotation to seconds. The clear general takeaway is that well configured or customized tooling can significantly reduce the number of hours worked.

Another change was replacing the manual file transfer process. Prior, there was a clunky manual file transfer process. This meant that geographically distributed teams often had multiple copies of the data, and the security controls had many holes. By moving to a single standardized training data system, all the security controls were centralized to one place. This also helped reduce data transfer costs, since objects were noted by reference instead of being physically moved. This was sort of like going from manually trucking around gold to modern banking.

Data scientists were perhaps the most to benefit. Before, two different teams treated labels (e.g., with bounding boxes) and attributes as two different things. This meant confusion. By moving to a training data system, they were able to unify their schemas into one. Further, they now query the data directly from the system, and even get notified when sets are ready to be reviewed or progressed.

Finally, by starting to “systemize” everything, they were able to unlock an entirely new product direction that removed much PII and annotation concerns.

In summary, the benefits of their adopting a standard training data system were:

- Security improved, going from A to B to C to D to just one PII-compliant place. Plus, data transfer costs were reduced now that all the data could remain in one place at rest.

- Now data science uses exactly the same schema, instead of three different copies among three different teams, leading to faster model production. Velocity to annotate went from an intractable amount of time to seconds, and “all-new” forms of annotation were now possible.
- An all-new product line was created.

While this may seem “too easy” or overly simplified, this was a real-world thing that happened and was primarily driven by the decision to use a standard off-the-shelf system, and the political capital to drive a core team of change agents to implement and evangelize the change. This is textbook positive AI transformation.

Quality Assurance at a Large-Scale Self-Driving Project

This study is focused on annotation quality assurance (QA) learnings. In this case, the observations are going to focus on the actual QA annotation aspects and will zoom in more specifically on the annotation side. The imaginary “X Self-Driving Company” routinely had to change the schema labels because of confusion. These are some of the QA lessons learned from that process.

The overall theme of this is failure to update the schema. The team kept expanding the instructions and making changes that arguably made the schema worse, instead of holistically expanding the schema to meet the minimum needs of modeling and raw data.

Tricky schemas should be expanded, not shrunk

The model had difficulty deciphering between “Trucks with Trailers” and “Large Vehicles.” As a result, the schema was revised, and the team ended up not using the Trailer class, choosing to label them all as Large Vehicles instead. This label included several other vehicles, including RVs, semi-trailers, and box trucks. This reversion to a more generic label helped the annotation effort, but left a huge hole in model performance. My take is that “shrinking” the schema in this context was a suboptimal approach. It would be better to still label something like “Vehicles,” and then specify “Size”

and “Attachments” as attributes. The “Attachments” section could have an “Unclear” category. That way, the true state of the vehicle can be known. It’s worth remembering that we can aggregate labels and attributes during ML training. For example, data science may place “Trucks” and “Large Vehicles” into a single category later as a preprocessing step for the ML training.

Don’t justify a clearly bad schema with domain-specific assumptions

At X, small towed objects were labeled as the object towing it. For example, a generator being towed by a large vehicle would get rolled into the “Large Vehicle” label. Their approach was to have the QA team clarify what “Small” and “Large” was. Because their use case was related to semi-trucks, a pickup truck (half ton) was counter-intuitively considered a small vehicle. My take is that this is still a bad schema. The schema should be clear to human annotators and only expert terms should be left for SMEs. This means that it would be better instead to use a label like “Vehicle,” and then add attribute types (pickup truck, van, etc.). Having to clarify to everyone that “large” was used in a supposedly use case–specific meaning (large relative to a semi-truck) leaves too much room for error.

Tracking spatial quality and errors per image

To manage quality, track the expected number of errors and normal ranges, as shown in Table 9-1.

Table 9-1. Normal ranges for common pixel and annotation errors (in context of images as the media form)

Expected number of errors	Normal ranges and notes
Of wrong pixels per image	• As far as pixels, this could be as many as 1,000 pixels. • In general, the target was 200 or less, but they would routinely get errors of over 800 pixels. • Setting an accuracy threshold was needed. For example, sometimes QA would try to fix a single pixel on a line, and it was just not an effective use of time.
Of wrong annotations per image	• They would generally expect less than 0.02 errors per image. • 0.1 errors per image was a cause for major alarm. • Practically, this would mean that for every 50 images, which may be over 500 instances, there should only be 1 error. Note that they would set a threshold for this above a certain pixel value. So, for example, “corrections” of less than 50 pixels or 50% change (whichever is greater) may still be required, but not counted as “errors” for the sake of this metric.

The level of quality from the provider ranged from 47 to 98% depending on the label. In general, with more severe effort from the customer, the

customer could self-audit to 98%+. In general, it appeared that getting above 98% was almost certainly in the realm of different opinions or guesswork, and was more than what could actually be determined from the image.

In general, there was a relationship between volume and errors. Usually, the higher the volume, the lower the relative error. In theory, this could be because annotators were getting more familiar with the data and self-correcting.

I have two main takeaways:

- Surprisingly, at least to me, it is possible to track quality using this method.
- 98% or above is as “perfect” as you can get for spatial-related work.

Schemas for lane lines need to represent the real world. Company X had recurring trouble with lane lines. While the images can’t be reproduced, visually I would say that part of the issue is that lane lines were simply very ill-defined. For example, lane lines appeared with a smidge of black lines (not white), there were painted-over lines, and lines where the “ghost” of the paint was there, but there was no actual paint. My take is that they did not align the schema to the raw data well in this case. It’s not that hard, looking at it, to iterate out some of these common cases, but instead, they would blame the annotation team (instead of fixing the schema).

Regression and focused effort do not always solve specific problems

Continuing the example of the lane lines, they found that focused attention did not automatically solve the problem. For example, during weekly reviews, they would highlight a problem area, and that problem may actually get worse the next week.

To highlight errors (e.g., of the lane lines) they would use a comparison format, where the image was duplicated with the fix shown on the second image. This worked fairly well, but the degree of error was hard to understand because the zoom/crop rate was variable. This meant that

sometimes an error looked small when it wasn’t, or vice versa. Including some kind of mini-map or zoom percent may help alleviate this.

Overfocus on complex instructions instead of fixing the schema

A recurring theme in this project was a failure to fix the schema, as they instead attempted to solve it by ever more specific, complex, and frankly confusing instructions, as shown in Table 9-2.

Table 9-2. Contrast of labeling instructions changing over time with author's commentary

Initial instruction	Problem	Second instruction	Author's commentary
"Don't label faded lane lines."	Model struggled with images with faded lane lines.	"Label all lane lines no matter what."	Label all lines AND put an attribute of "faded" on faded lines. If needed, de-emphasize overall volume to ensure critical quality threshold is met.
"Do label all side guards even if you can't see them."	Model overpredicted side guards.	"Only label what you can see."	Only labeling what you can see is usually the best default starting point.
Label "vegetation" differently from grass and weeds, etc.	Annotators easily confused "vegetation" with forms of vegetation such as grass and weeds.	Not addressed.	Always be clear about what is an aggregate and what is a specific thing. Rather, have "vegetation" as a top-level label and then [grass, weeds] as attributes.

They also had trouble with the drivable road surface being different from the background (e.g., a road that's visible, but not accessible from the current place). Again, this is not that hard to think about conceptually; even

simplistically, this can be two different things, but instead of changing the schema, they just kept growing the instruction set (it literally became a Wiki with hundreds of pages, which is just crazy!). Yes, good instructions are important, but they should not replace having a good schema. Always try to correct problems in the schema itself before changing instructions. An analogy is like “correct the form on the website instead of adding more and more instructions about how to use the form.”

Trade-offs of attempting to achieve “perfection” in nuanced domain-specific cases

One of the themes was tension between wanting to achieve a high degree of quality, but also not wanting too much time spent on relatively small corrections. At one point, a limit was set that if greater than 4× zoom was used, they were to not make the correction.

They did not do a good job separating purely observable information from what we as humans were conceptually adding. For example, a human might guess that a mailbox (or “*x*” thing) is beside the road, so it’s labeled as such, but the observable image (without the assumption of the context) is just a dark patch that doesn’t show anything.

Another tension point was that sometimes the “what can you tell from visual only” was expected to include a near-complete driving experience, with knowledge of the laws of the local region. For example, to determine if it’s a road or a shoulder a person must sometimes know the meaning of line colors and types, and the expected width of the road in cases where the road surface is larger than reasonable and part of it is considered to be the shoulder.

The nuanced cases in particular are very difficult to determine a fixed solution for because they come across as arbitrary in many cases. In general, it seemed very difficult to achieve a high accuracy, at a reasonable QA cost, and especially with the borderline and nuanced cases. Let’s take a closer look at some examples of these nuanced cases.

Understanding nuanced cases

In the cases where there was occlusion, it was deemed OK to have a continuous label (overriding the visual evidence requirement). However, the trick was that every extent had to include known information. For example, a lane that starts on the left and is visible, then is blocked by a truck, then is visible again, was OK. But a lane that is only visible at the start or end was considered not OK.

Distinctions between easily confusable things caused a lot of issues; here are two examples:

- “Paved” versus “unpaved.” This is more ambiguous than it sounds. For example, is stonework considered “paving”? Is a lane on the right of a solid white line road surface or shoulder? In California, for example, the answer to that is ambiguous in many cases (for example, dual turning lanes).
- “Terrain” versus “shoulder” was overloaded (multiple meanings) and ill-defined. For example, many roads have shoulders that are essentially also terrain, so the term is overloaded. Guardrail sections that intersect with other labels face a similar problem. For example, the few pixels of ground a guardrail is stuck into would probably be better labeled “guardrail” or “guardrail base,” since you can’t drive into that space without hitting the guardrail!

The value of tracking “pixel-wise” error seems to be questionable because it’s highly correlated with the class (e.g., terrain has 100× more pixels than a railing). It seems that method is only useful if scoped to the same class.

Stacking ranking errors by class did appear to highlight the most common errors, but the practical value of this appeared to be somewhat limited.

Learning from mistakes

Here, I will recap some of these mistakes and offer my take on better approaches.

Define occlusion well

Occlusion was poorly defined in general. For example, a transparent fence was to be called the underlying class. A better choice would be to have two labels. A commonly confused thing is that people think that you can only have one label for a given “slot” of space. That’s not true. You can have overlapping labels. You just need to specify the occlusion or z-order (which in turn can be predicted by the ML system). So in this case, there should be a label “fence,” which could also have an attribute “foreground” or “most visible.” Then there is a second annotation “road,” which contains the attribute “background.”

Expand schemas

They used overly broad class definitions. For example, is a truck-mounted RV a “large vehicle”? Yes, by most measures, but not for their domain-specific (semi-trailer size) area.

A better approach is to have a much more diverse set of annotation classes (e.g., using labels and attributes) that allow the annotator to be as accurate as reasonably possible, and then have a set of relationships back to whatever label you want to train on. So the annotators should still just say “RV,” and then the system can map that to “large vehicle” if that’s how you want to train it. Trying to fix overly broad classes with instructions is jamming a square peg into a round role.

Remember the null case

A general theme was that “things outside the immediate view” are hard to label. Is that blur of gray and green on the extreme edge of the camera vegetation or buildings? I don’t know. In general, they seemed to be missing “unknown” or “null” classes.

Another example was the difference between green hills in the background and the “vegetation class.” The green hills *are* vegetation, but the request was to label the hills as background because they (using knowledge outside of what is visually obvious) are farther away.

Missing assumptions for language barriers

Keep in mind that annotators may not be native speakers of English, so a distinction like “dashed” versus “dotted” lines may be lost in translation or lack of background context. Also, for most of the images I saw, at the resolution being used, “dashed” and “dotted” was ambiguous, so a person could argue either way, meaning this was probably a bad class name. While local or in-house subject matter experts may not have an English language barrier, the domain knowledge and domain-specific terms form a language barrier in and of themselves and so this problem remains.

Don’t overfocus on spatial information

Sometimes, too much attention was paid to spatial location in some class types. For example, for an advertising sign (not a legal road sign), they would expect the support posts to be labeled differently from the “sign” itself. Whether this was needed for the ML model, I don’t know, but it likely took ten times as much time to annotate every sign, and made QA harder. My two cents is that would also make it much harder for a model to predict it, since it would take what otherwise could be a bounding box and turn it unnecessarily into a segmentation problem.

Big-Tech Challenges

This study is zooming out and looking at the broad organizational effect around training data. “Y Company” (a fictional company) is a leading producer of consumer electronics and has a massive AI organization.

There are many talented people at Y Company, but their overall production of training data is lagging behind competitors. Despite being the best in many other areas, their AI product is not competitive with other products in this space. They have decided to make some changes to fix their organization and, while the results of those changes are yet to be seen, I think the reasons for those changes and desire to make them highlights some important training data lessons.

To set the stage, Y Company follows the textbook examples of the multiple small teams approach, where the overall problem is divided into a variety of sub-problems and each team works in a specific area. For example, there

are data engineering (infrastructure) teams, annotation team(s), data science teams(s), etc.

Let’s take a closer look at their structure and some of the challenges they encountered.

Two annotation software teams

In part because of the breadth of the space, Y Company has two different annotation software teams.

Each team is responsible for the entire end-to-end annotation process, including software tools. One team covers some interface types, such as images, whereas the other team covers different types, such as audio. The responsibilities look something like Table 9-3.

Table 9-3. Two annotation software teams doing similar tasks

Team one	Team two	Overlap?
Audio interface	Image interface	No
Import/export	Import/export	Yes
Storage abstractions	Storage abstractions	Yes
Human workflow	Human workflow	Yes
Automations	Automations	Yes
Third-party integrations	Third-party integrations	Yes
Scheduling, general operations	Scheduling, general operations	Yes
User admin, overall management	User admin, overall management	Yes
Hardware infrastructure	Hardware infrastructure	Yes
Much more...	Much more...	Yes

Can you spot the difference?

They realized that the teams were doing very similar work with only the surface level UI being different. And clearly, having multiple software teams doing essentially the same thing is suboptimal (think 95%+ overlap). After much effort, they started a multi-year project to merge the systems.

But wait, you may say—surely there are valid reasons for this? Integration with audio must not be the same as images? That argument, in this overall system design context, confuses the fruit with the fruit cart.

The overall ingestion system, and high-level integration concepts around shared principles like labels, attributes, storage adapters, etc., is the cart. The specific data type that will naturally have a different format is like the type of fruit. You set up a route, and a driver, and get the cart once, and then can transport different kinds of fruit.

Shifting to the context of using off-the-shelf systems, for a very large setup, the hardware configuration of the audio-focused instance could be different from the image-focused instances. But that's a configuration detail, not each team making totally novel independent design decisions on hardware.

For human tasks, yes, there are some practical differences between a long-form media type like video or 3D, and certain more simple types of images. But this comes back to UI customization, and relatively small changes. The core principles around task management, user management, etc. remain the same.

Again, this may seem obvious in retrospect, but if the project starts as a "get it done quick" order from data science, and there are apparently narrowly defined data types, labels, volume of users, etc. it's relatively easy to miss.

Confusing the media types

To put it plainly, having two similar platforms is costly.

Since this seems so obvious in retrospect, I will explain how it appears to have come to pass at Y Company. Chiefly, this is a confusion between the UI concerns and the platform needs.

When these projects start, the annotation interface (the UI) is rarely thought of as a *platform*. People get fixated on things like the end-user-facing interface (images, audio, etc.) and forget about the vast amount of behind-the-scenes work to arrive at that.

The clear mitigation step here is to think of a training data *platform* as the first choice, and the specific interfaces that it uses as a second choice. The trend currently is for platforms to support all popular interfaces anyway, so if you are buying an off-the-shelf platform, this may already be addressed.

Why does the platform concept come into play? Because, as I showed, all media types have similar underlying challenges. Be it images, video, or something else, it all needs ingestion, storage, human workflow, the annotation interface, the automations, the integrations, the connections to training, and more. The media-type-specific interface is a sub-concern of the annotation interface area.

Non-queryable

Y Company has a dedicated team for different layers of the process.

Unfortunately, in this case, the different layers were defined in terms of data science steps without regard to the centrality of training data.

Data science is in a tough spot, because much of the metadata around objects is not accessible to them upon query. Essentially, this means they had to query each object, get it, and inspect it, in order to then build their dataset, as shown in Figure 9-1.

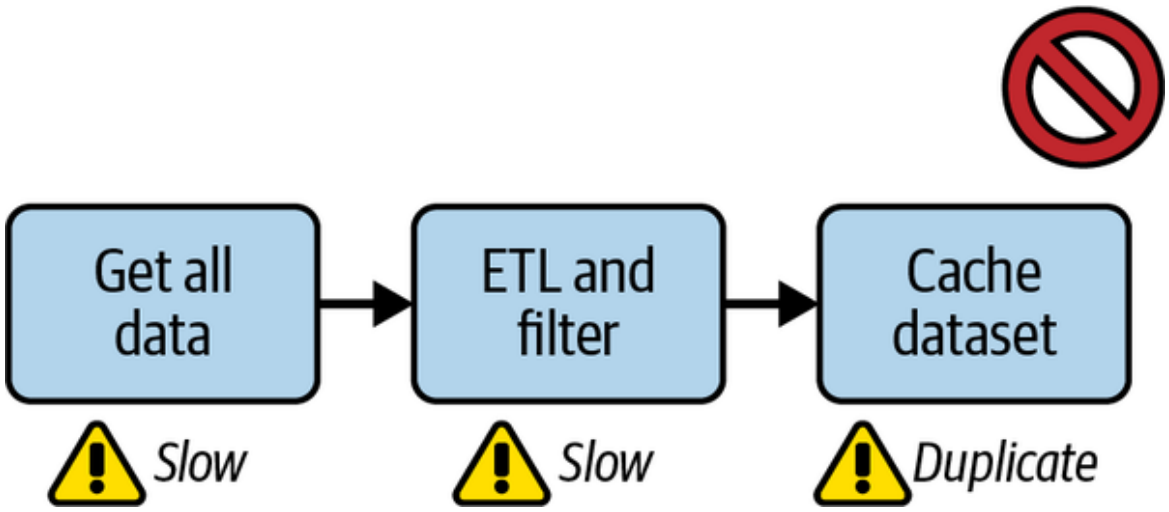


Figure 9-1. The slow and error-prone ETL process at Y Company

There are several practical implications:

- Most of the security controls designed by the import team were routinely defeated. Data that was supposed to be deleted in a few years would last for much longer in the data science team under the guise of datasets.

- At this scale (think 10s–100s of millions of records) there was no hope of any speedy updates. This meant that they may need to pull and store a million records, in order to get the few thousand they wanted.
- This put huge burst load problems on the storage team, since they were expected to support absolutely massive on-demand access (think <10k QPS and >500 modes).

While some level of "loading extract and transform" (or the various orders of those concepts) is standard, in Y Company's case, it becomes excessive and without clear value added. The better approach is for the prediction and raw media data to go more directly to a known format and be pulled and queried from a shared known format and logical (or even physical) place.

Different teams for annotations and raw media

For data science to actually get the raw data, they had to talk with two different teams. First, they would get the annotations from the annotation team, and then usually they needed to separately get the raw media from the storage team. This led to inevitable problems like having to merge records and finger pointing. They would have the ID for the annotation, but not the raw data, or vice versa. Deletions were a huge problem. A record may be deleted in one system, but exist in the other.

This means that the process shown in Figure 9-1 was sometimes further duplicated, or additional processes must be created to broker or merge between them, as shown in Figure 9-2. This also caused issues in reverse, e.g., that the training data team needed to correlate and load predictions and then merge them with raw media.

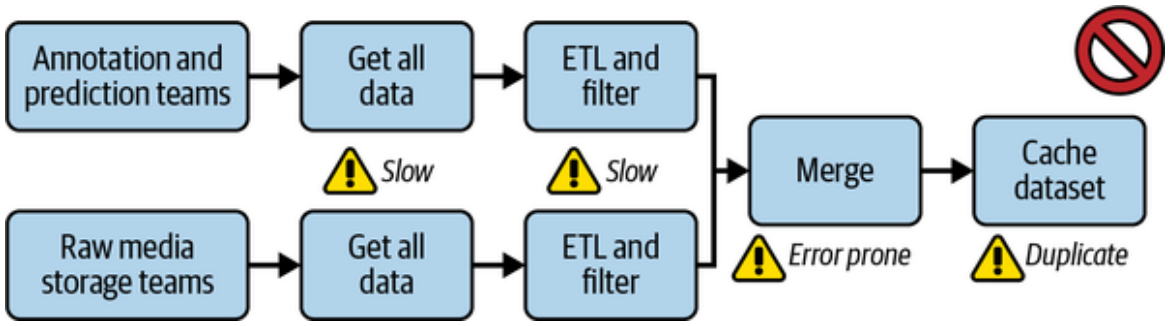


Figure 9-2. Multiple teams exacerbating ETL process further

Part of why this was such a problem is that there was no clear upstream. No clear flow of data. No clear producer and consumer.

This resulted in major issues, both of which remain largely unsolved at Y Company:

- Data science is forced to "greedily" extract data so as to ensure they can actually get it. Basically, they can't "trust" the upstream system, so they are routinely caching the data. And in fairness to the raw storage team, it wasn't that they were doing anything wrong, it's just that they didn't control the annotation. This was an organization mistake—no single team could correct it on their own.
- Annotation (teams) have to use the storage as an unnecessary proxy to access data. They often end up caching the data too.

Moving toward a system of record

It is very clear there is a desire to move toward a more unified approach, a single system of record.

For example, training data teams can access the storage layer directly instead of the unnecessary proxy. At the time of the case study, Y Company was evaluating this training data database as this system of record, with key concepts shown in Figure 9-3.

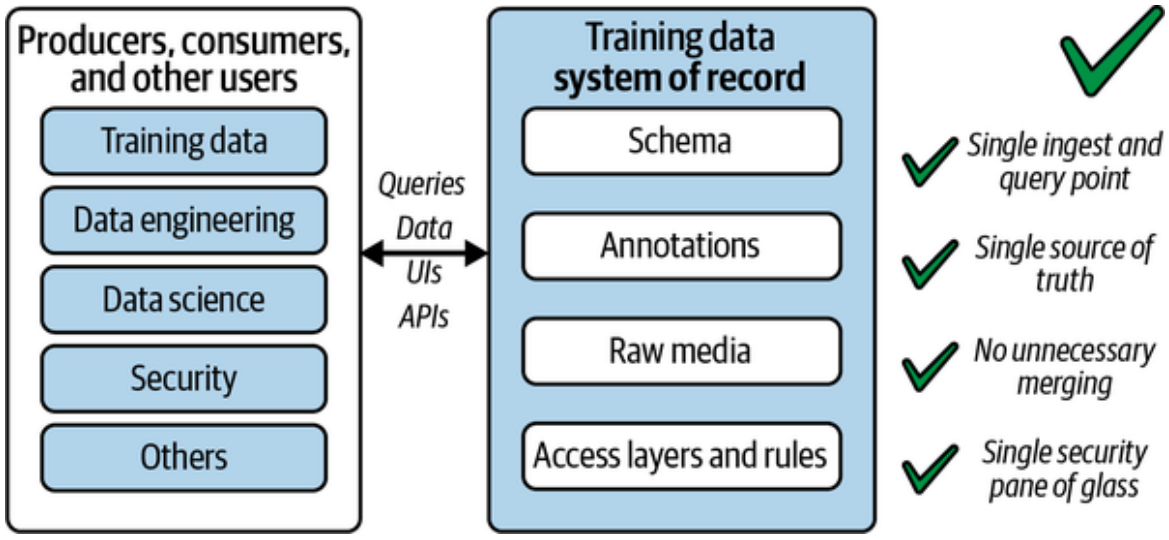


Figure 9-3. Users and goals of a system of record for training data

There is a shared state of annotations and raw media. This means that when data is ingested (for example, a production-level prediction), that representation is framed in the same way that the annotation team will frame it. A record that's stored as raw storage will be accessed directly from annotation.

This also changes the reporting relationship. There can now conceptually be a flow of training data producing and data science consuming. In practice, this does not mean there is a massive data pipe going from training data to data science. Rather, think of it as a notification. Data science can track the activity of the training data production, but when they pull data, it's from the same source as that annotation wrote to.

To understand why training data becomes the center of gravity for these systems, consider these specifications:

- Data science must export the data, on an ongoing basis, in order to use it.
- The formats, privacy rules, data, and organization change routinely, sometimes hourly or daily.
- Data science consumes training data and produces predictions consumed by training data.

Missing the big picture

Despite Y Company having many oddly specific things as top-level "named concepts," training data was not previously one of them. This confusion was at the heart of the issue; there was not an overall concept of training data, with each team doing their own data processes, and no "big picture." As training data became a named concept, the pattern of having a system of record for it became more obvious. This changed what was previously a total mess into the earlier shown Figure 9-3.

This also helped create clear streams, such as from input to training data producers to data science consumers. A key point of consideration for careful readers of this book is that while conceptually, training data produces and data science consumes, at a large organization, there are many other processes and loops to consider. For example, data engineering may have production predictions that are fed back into training data. A security team may desire a "single pane of glass" for the data. All of this is relatively straightforward with a single system of record.

Solution

The solution consisted of two stages:

1. See the big picture, see the training data mindset, and observe the overlaps between multiple teams.
2. Move toward a system of record to align multiple teams.

While the new direction was still a work in progress, the expected results of this change at the company were impactful:

- There is now a single source of truth (including ingest and query points): The data science team accesses the latest version of the data. The need of data science to query the data, form named datasets, create immutable versions, etc. are all given first-class consideration. This reduces the otherwise painful overhead of trying to fight to get the data, converting it into a smooth process. An analogy here is like

querying a database versus having to get an FTP transfer and then reconstruct it.

- There is no longer unnecessary merging: The data is captured at ingest in such a way so as to support annotation from day one. This means that the need to annotate is again a first-class consideration. This eliminates the need for the annotation team to have to create their own methods here. To be clear, there can still be a team of people working on the literal interface, but they are using access methods defined by the training data system, not creating their own secondary ingestion process into their system.

This also creates much more linear reporting relationships. If a new production of data is needed, it can be clearly defined and reasoned about instead of having to nebulously coordinate between three or more somewhat opposing teams.

Let’s address loops

This takes nothing away from pre-labeling, or other interactions, iterative improvement, etc. If anything, by creating a closer relationship between training data and data science, it enhances the ability to improve these automations. If it helps, you can imagine there being a loop between producers and consumers, and/or between the input and output. The preceding diagram is “unrolled” so as to clearly indicate the key relationships.

Human in the loop

One of the other confusions here is that there is sometimes the need for data to be graded by humans in near real time, or in other ways to use that grading loop. In the cases where that is truly necessary, presumably data science or data engineering is aware of this, and must still pass through them in some capacity. For example, just because a human annotator supervises a data point, that does not mean that exact output is what is presented to an end user. Normally some other process must run to transform that output. For example, the annotator may correct a label, but

the end product doesn’t present labels, it presents aggregated statistics or alerts, etc. There may be a literal direct technical integration from annotation, but the preceding diagram is more in the spirit of the human organization.

The net effect here is that there is a lot of data duplication. My take is that part of the problem was not having training data as a named concept. Instead, Y Company was too narrowly focused on the business stakeholder goal without thinking about the wider context in the company.

The case for aligning teams around training data

We covered a lot of ground in this case study. Let’s review our findings. A tech company had dedicated teams for different layers of the process, like an “ingest” team. This meant that often, different teams within the company would treat it as an unreliable source, hoarding the data for their own use. This meant that most of the security controls designed by the import team were routinely defeated. Data that was supposed to be deleted in a few years would last for much longer in the data science team under the guise of datasets. There is now a desire to start moving toward a more unified approach, where the same system manages both the dataset organization and the import identification.

A rough analogy of this is the cliche about microservices where three different teams all have a 300MS speed target. But then when the customer actually uses it, the services have to talk to each other multiple times, meaning that the actual input/output of the system becomes something like 5–10 seconds or more.[2] A similar thing happens with training data, with each team giving service targets that have little relation to the goals of the other team. For example, data science is going to care about dataset level access and queries. But if the data is provided by the import team in a way that is not queryable, then data science must get *all* the data first. Keep in mind that at this scale, we are talking about hundreds of nodes, so this is a major operation.

There are a couple of lessons to take from this. One is that treating each layer of the stack too independently is worse than treating it as a single end-

to-end process.

The main lesson here is to treat training data as a first-class named concept, and then lean on normal concepts like having a system of record for training data. The company is doing this, and already merged both of the teams, and is continuing to drive toward a system of record used by multiple teams for training data–focused concepts.

Insurance Tech Startup Lessons

This story is looking at the effect of training data on the life of a small startup.

A notable insurance tech startup used photos of accidents to improve the automatic claims process. I discussed the failure with the chief engineer of the team, and the primary conclusion from that discussion follows.

Will the production data match the training data?

First and foremost, the insurance adjusters were not ready for AI adoption. This had the immediate technical impact that they would take “bad photos.” While some of the training was high resolution, and in some cases video walk-around, the photos they had to use in production were of a much lower quality.

Unfortunately, there was not enough political credibility to invest further efforts after the initial production attempts failed. This may have been avoidable if a deeper understanding of the expected production photos was gained before stating the system was ready for production.

To use a more classic system analogy, the availability of production data needs to be considered at the earliest architectural stages of the system. If you leave it to the user acceptance phase, then there will be big trouble—it’s not a few UI pages you can correct at the last minute, it’s the core of the system.

The main lesson is to make sure that the production data will match the training data.

Too late to bring in training data software

The chief engineer had been pushing for use of commercial training data software for years. His efforts fell on deaf ears at the C-suite level, because at the time, the focus was on their novel augmented data method.

Around two years into the project, after it became clear that the augmented method was not working, the CEO revisited this issue with new guidance to look into training data software. At this point, it was too late, but the lesson was clear. Bring training data software early to a project. Many startups have tried to "beat" the system by skipping annotations and to my knowledge, not one has succeeded yet.

Stories

The following are four brief stories of training data in the wild. While not full case studies, they provide interesting and useful anecdotes to help frame your work. The stories are:

- "Static Schema Prevented Innovation at Self-Driving Firm"
- "Startup Didn't Change Schema and Wasted Effort"
- "Accident Prevention Startup Missed Data-Centric Approach"
- "Sports Startup Successfully Used Pre-Labeling"

"Static Schema Prevented Innovation at Self Driving Firm"

This story looks at a large self-driving car company and the effect of a lack of awareness of training data. A prominent self-driving company was using static labels. Something like "Car" or "Bush" was a static concept to the system, meaning that any time they wanted to add a new label, it was a huge effort.

In comparison, another firm had taken a more data-centric approach to these labels, allowing for the introduction of concepts like "Cut-off Detection" (when a driver gets "cut off" by another driver in traffic) more smoothly. While this is naturally a complex problem, the carmaker with the

static named concepts has missed many deadlines, while the one with the more flexible approach to labels has cars on the road today. Some lessons from this: set up the mental models, tools, and processes so that you can be flexible with your training data. Explore and change the label schema as the situation warrants. A flexible and growing schema is a must-have for any modern training data project.

To understand this better, a key blocker was that the way the org structure worked, the training data team focused just on the infrastructure, and was at the mercy of other teams for the definition of their product (including concrete predefined schema names like “Car”).

At the time, the demand from these other teams was around how to serve the data efficiently to their training system—not working with them on a flexible and dynamic schema. This lack of cross-team awareness and discussion of training data led to a lack of flexibility in model predictions and a more general stagnation of the tech.

This was a classic example of being too busy to try something new, as shown in Figure 9-4. It’s a good reminder to not be too attached to current ideas and solutions even if they appear to be working. Instead, it’s better to have a training data first mindset.



Figure 9-4. Cartoon showing over-focus on day-to-day work over new innovations

“Startup Didn’t Change Schema and Wasted Effort”

This story is about a company that had kept the same fairly simple (mostly top-level labels) schema for about two years. Every day, a team of 10+

annotators dutifully annotated new examples from scratch (not even pre-labeling most of the time). They had no real benchmark to realize that this was really bad! If the model can't detect something after a few months of annotating the examples, one million more examples is probably not going to help.

Taking a closer look at their data, it became clear that they were annotating an object X that looked completely different in different images. That meant that they were really confusing the model more than they were helping it. We improved their performance by adding in tens of new attribute groups that helped them zero in on model performance.

"Accident Prevention Startup Missed Data-Centric Approach"

This story is looking at the effects of a mid-sized startup struggling to grow from a company that makes dashcams to prevent accidents. The company was "high flying," but stagnated.

The company did not adopt a data-centric approach. The company had very few annotators and a relatively large engineering team. This meant that a lot of effort went into classic "engineering" approaches to solve problems that would have been easily solved with some annotations. Because they were paying about 20x+ more for an engineer over an annotator, this was especially problematic. This mainly appeared to be a lack of knowledge of a training data mindset.

Regarding tooling, the company acquired another small startup that had an in-house annotation tool as a side project. As you can imagine, this was not a robust tool, but they felt committed to it and wouldn't change for more modern tooling. This gap seemed to be from a lack of understanding of the big picture of training data and the cross-functional importance of a system of record for training data.

On use cases, they over-focused on the type of annotation method (e.g., image segmentation) instead of use cases. They had ample opportunities to expand existing use cases by adding deeper attributes to existing bounding

boxes, but missed it by getting blinders on regarding the spatial detection concerns.

Overall, there was little internal excitement about the company's current roadmap, and lack of product innovation was part of it. Given how heavily their product leaned into image use cases (from the road ahead, to driver monitoring, etc.), it was a big missed opportunity to lean into a training data approach.

There are few key takeaways:

- If your engineering team to annotator ratio is positive, that's bad, and an indication that you probably aren't using a data-centric approach.
- In-housing training data tooling is usually suboptimal; you should start with open source tools and build from there (and ideally contribute back to them).
- Use the Use Cases rubric in Table 7-5 to understand and scope better use cases.

"Sports Startup Successfully Used Pre-Labeling"

A sports analytics company has used pre-labeled data on videos to great effect. They managed to essentially reduce the box drawing time to approach zero, freeing up annotation time to work on attributes that would directly add value to their application. This effort was measured on the order of years.

I include this brief story to note that it is possible to have success with automations. Also, most of the effort over time was in annotations and sequence/event predictions, not on spatial locations.

An Academic Approach to Training Data

All of the previous studies and stories were from industry. I'd like to wrap up this chapter by talking about an academic challenge called the Kaggle TSA Competition.

Kaggle TSA Competition

In 2017, Kaggle, a data science competition platform, hosted its largest ever global competition: 1.5 million dollars in prize money, 518 teams of some of the top minds in the space. I participated in this competition too, so I can also speak from the participant's perspective. To this day, it remains one of the competitions that has attracted the most attention ever.

Officially, the theme was "Improve the accuracy of the Department of Homeland Security's threat recognition algorithms." The technical concept was to detect threats at airport security based on data from a 3D millimeter wave scanner, as shown in Figure 9-5. The data was provided in pre-processed slices of 64 images per scan, meaning that in general, most people treated it as plain RGB images. The principal goal was, given a novel set of scans, to detect with a very high degree of accuracy what threat was present and where. The prediction goal included localizing to 1 of 17 location zones. These zones were abstract, however, and did not reflect any kind of existing spatial training data.

Next time you are at the airport, if you look at the display beside these scanners, you will likely see it's mostly empty, with a general "OK," or a box on a generic model of a person. In this case, an "OK" means a super-high confidence that there is nothing present. A box on a specific location means a "probably nothing, but not quite enough to pass without additional check." The vast majority of people are allowed to pass. And the goal is to minimize manual intervention to improve process efficiency as well as the passenger experience. Achieving this goal means highly reliable algorithms with a very low false positive or false negative rate are needed.

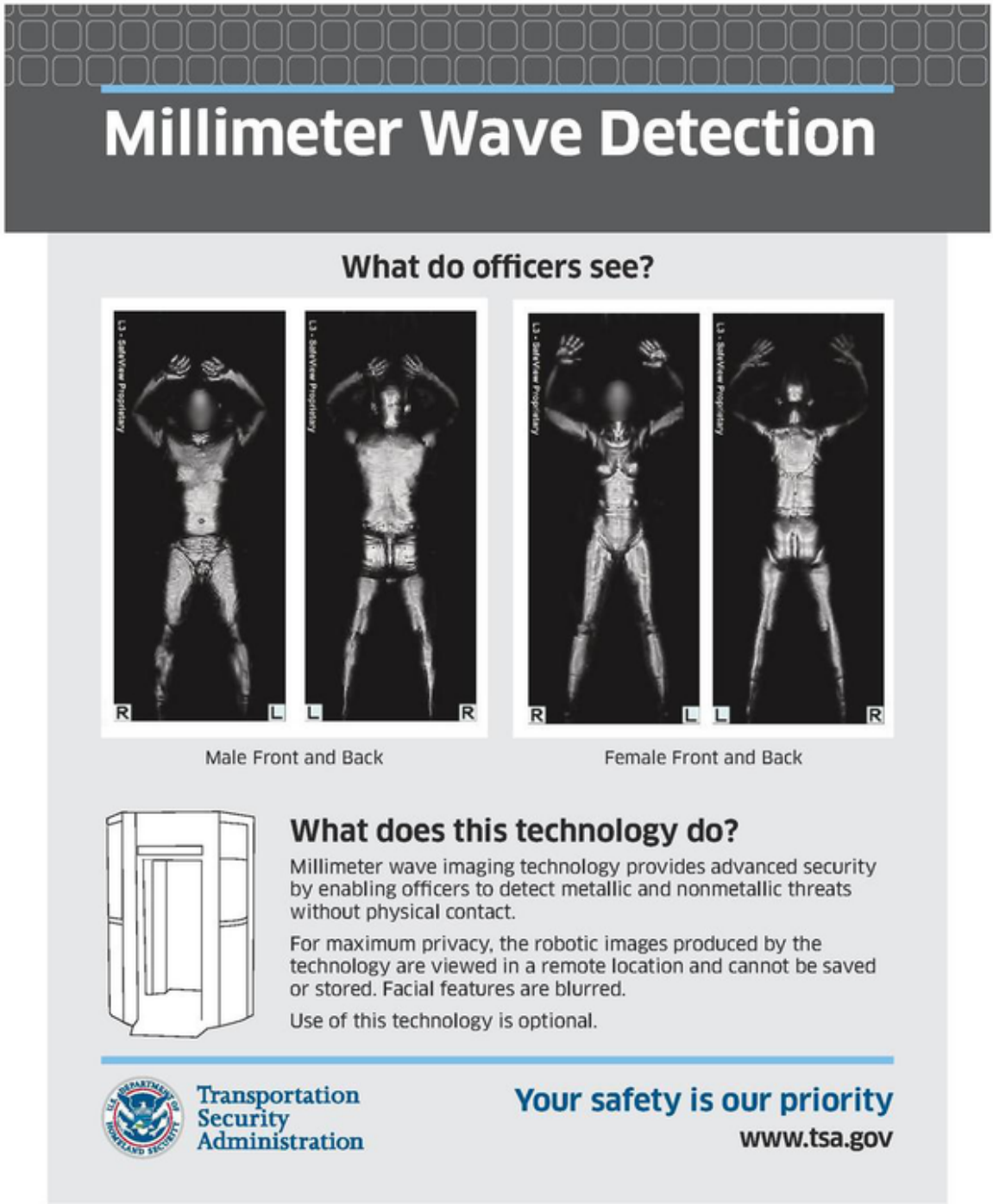


Figure 9-5. Conceptual overview of millimeter wave detection and example images from public domain (The image is in the public domain in the United States.)

Keying in on training data

Training data was what mattered, and every top approach focused around it. The winning approach centered around data augmentation. Many of the top-

placing approaches used new human-made annotations that captured spatial data. To capture an idea of how intense the competition was, one competitor missed out on a $100,000 place, because out of thousands of classifications, a single misclassification was included in their results.

That being said, the majority of approaches within the top 10% were actually fairly similar in performance, with the winners mostly doing competition-focused work like multimodal ensembles. Further, in that context, it was possible to get in a top rank simply by creating training data annotations for it. One approach was to manually annotate the data using bounding boxes (over and above the provided data). This newly annotated spatial data allowed me to create an extremely simple model setup. One model predicted if a threat was present, and the other predicted where on the body the threat was.

To try to frame the context of this, while it's common knowledge that actually winning a Kaggle competition often falls down to hyper-specific things, usually these things are to do with data science itself (e.g., model parameters, etc.), whereas in this case, it was much less about the models. All of the winning write-ups I have read have all stated most casually what actual model architecture they used. This is foundationally different from most other cases, where the model choice, feature selection tuning, etc. mattered most.

How focusing on training data reveals commercial efficiencies

One of the common criticisms of these types of competitions is that the winning approaches are impractical for actual use. Often, they involve orders of magnitude more computation than is reasonable, or over-fit dramatically to the specifics of the way the competition frames the problem.

For me, one of the biggest wake-up moments came back in 2017 when I realized that by using an "off-the-shelf" object detector, with the only real difference being the training data, I could in a matter of days beat or at least match virtually every other method. This was about as close to a legal cheat code as you can get.

To circle this back to the commercial efficiency, it meant that instead of needing to treat each dataset as this unbounded scientific research initiative, it could be reduced to the calculation of how much annotation effort was needed, plus some reasonable estimate amount to train the model. While this glosses over a lot, fundamentally it remains true today and is at the heart of the swarm of commercial interest in this technology.

Learning lessons and mistakes

Up to a certain point, adding new training data was like magic to improve performance. After a while, it reached a point of diminishing returns. These are some of the lessons I've learned, and reflect what I would do differently today for a production system.

I only annotated the high-level category of "threat," which meant it was hard to know how to improve the model for specific types of threats. Adding more training data to cases that are already well-covered does little. For example, I recall there were certain categories that were much harder than others. I manually shifted my efforts to focus on samples with the worst performance. However, this was all just ad hoc. Knowing what I know now, I would create a schema that addressed those cases directly. That way, I could statistically know how well each case was being covered, versus going with a "gut feel." In other words, a production version of this would need a much more detailed schema.

Bad training data is incredibly and disproportionately damaging. After a while, I realized that sometimes an image was being included without proper labels, or with inconsistent or wrongly placed labels. I fixed these errors after I realized what a negative effect they were having. A good rule of thumb is that a bad example is only offset by three or more correct examples.

There are many articles and books on how to improve model performance. I specifically want to highlight again that all of these points center around training data. It doesn't matter what type of model training process you are using; all of the preceding lessons will still apply in one or another.

Summary

There are a few key concepts and recommendations I hope you take from these case studies:

- Static schemas limit innovation. Flexible, growing schemas are important for training data.
- Raw production data should be similar to raw data in training data datasets. Consider this early in system design.
- Thinking they can "engineer their way out" instead of using a training data-centric or data-centric approach is a common mistake for startups.
- For large organizations, aligning teams around a training data system of record is better than slow, duplicated, and error-prone "free-for-all" approaches.
- Bad examples can disproportionately damage models. Track issues within specific schema attributes to learn from mistakes and more easily continuously improve training data.
- Thoughtful design of schemas, workflows, and quality assurance is important for overall success.

The recurring theme is that real-world training data is nuanced and complex. Hopefully, these experiences have given you some insights to apply to your own contexts.

---

[1] E.g., branchless conditional

[2] E.g., IN->A->B->A->C->B->OUT

OceanofPDF.com

Index

Symbols

3D points, BLOB (Binary Large Object) data, 3D

A

ABAC (attribute-based access control), Identity and Authorization

abstractions, AI and, A System Is Only as Useful as Its Schema

academic case study, Kaggle TSA Competition-Learning lessons and mistakes

access control, Access Control

AI (artificial intelligence), Training Data Introduction

- abstractions and, A System Is Only as Useful as Its Schema
- Director of AI data, Appoint a Leader: The Director of AI Data-Data Engineer
- hardware, The Need for Hardware
- successful, The Foundation of Successful AI

AI transformation, AI Transformation

- annotation, day-to-day work as, Seeing Your Day-to-Day Work as Annotation-Seeing Your Day-to-Day Work as Annotation
- data-centric AI, The Creative Revolution of Data-centric AI
    - data collecting, changing, You Can Change What Data You Collect

    - data creation, You Can Create New Data
    - data meaning, You Can Change the Meaning of the Data
- history, Data Engineer
- map, AI Transformation
- step functions, Think Step Function Improvement for Major Projects-Think Step Function Improvement for Major Projects
- types, Seeing Your Day-to-Day Work as Annotation
- use cases, identifying, Use Case Discovery-Ongoing impact of use cases

analytics

- annotation metrics, Annotation Metrics Examples
- data exploration, Data Exploration-Similar image reduction

annotation, Training Data Introduction, What Can You Do With Training Data?

- closed source, Open Source and Closed Source-Customization is the name of the game
- crowd sourcing mindset, The New “Crowd Sourcing”: Your Own Experts-Common Project Mistakes
- day-to-day work as, Seeing Your Day-to-Day Work as Annotation-Seeing Your Day-to-Day Work as Annotation
- direct, Direct Annotation
    - attributes, Attributes
    - business processes, Business Process Integration
    - interactive automation, Interactive Automations

    - labeling, depth, Depth of Labeling
    - supervising data, Supervising Existing Data
    - video, Video-Common issues in video annotation
- instances, Simple Schema for a Strawberry Picking System
- metrics example, Annotation Metrics Examples
- open source, Open Source and Closed Source-Customization is the name of the game
- quality, Quality
- repeatability, Think Step Function Improvement for Major Projects-Think Step Function Improvement for Major Projects
- tools, Tools Overview

annotation interfaces, Annotation Interfaces

Annotation Producer, Annotation Producer

annotations, Annotations

- attributes and, Attribute concepts

annotator setup, Annotator Setup

- embedded, Embedded
- portal, Portal (default)

annotators, Annotators

- as partners, Annotators Are Partners
    - annotator's needs, Annotator needs
    - data supervision, Who supervises the data
    - errors in data, All training data has errors

- tool use, How Long Will the Average Annotator Be Using It?
- trust, Annotator Trust

application engineers, Application engineers

applied sets versus research sets, Applied Versus Research Sets-Applied Versus Research Sets

architecture, security, Security architecture

artificial intelligence (AI), Training Data Introduction

attack surface, Attack surface

attribute-based access control (ABAC), Identity and Authorization

attributes, Schema Deep Dive Introduction, Simple Schema for a Strawberry Picking System

- annotations and, Attribute concepts
- as what, Attributes Introduction
- columns, Introduction to Labels
- complexity, Attribute Complexity Exceeds Spatial Complexity-Example of sharing attributes between labels
- constraints, Attribute concepts
- definitions and, Attributes Introduction
- depth, Attribute depth
- depth/hierarchy of labels/meaning, Attribute concepts
- direct annotation, Attributes
- directional vector, Attribute concepts
- examples, Examples of attributes

    - technical example, Technical example of an attribute
- form type, Attribute concepts
- generic, Attribute concepts
- instances and, Example of an attribute in relation to an instance
- labels, sharing, Example of sharing attributes between labels
- occlusion, Attribute concepts
- predefined selection, Attribute concepts
- questions for humans, Attribute concepts
- schema complexity and, Schema complexity trade-off
- scope, Attribute concepts
- templates, Attribute concepts
- truncation, Attribute concepts

augmentation, Augmentation-Patch and inject method (crop and inject)

authorization, Identity and Authorization

automation, Introduction

- (see also interactive automations)
- academic case study, Kaggle TSA Competition-Learning lessons and mistakes
- augmentation, Augmentation-Patch and inject method (crop and inject)
- benchmarking, How to Benchmark Well-How to Benchmark Well
- correction time, Correction Time
- data discovery, Data Discovery: What to Label

    - human exploration and, Human Exploration
    - metadata exploration and, Metadata Exploration
    - raw data exploration and, Raw Data Exploration
- domain-specific, Domain Specific-Heuristics-Based Labeling
- industry case studies
    - big tech, Big-Tech Challenges-The case for aligning teams around training data
    - insurance tech startup, Insurance Tech Startup Lessons-Too late to bring in training data software
    - QA at self-driving project, Quality Assurance at a Large-Scale Self-Driving Project-Don’t overfocus on spatial information
    - security startup, training data tools, A Security Startup Adopts Training Data Tools-A Security Startup Adopts Training Data Tools
- industry stories, Stories-“Sports Startup Successfully Used Pre-Labeling”
- interactive, direct annotation, Interactive Automations
- labeling, novel model creation, “Fully” automatic labeling for novel model creation
- media specific, Media Specific-Considerations
- media-specific research, Media-Specific Research-Media-Specific Research
- methods
    - commonly used techniques, Commonly used techniques
    - domain specific, Domain-specific

    - order, A note on ordering
- nature of, Nature of Automations
- proprietary methods, Proprietary automatic methods
- QA tools, Using the Model to Debug the Humans-Domain-Specific Reasonableness Checks
- results, What Kind of Results Can I Expect?
- risk, Risks-Risks
- schema, Check What Part of the Schema a Method Is Designed to Work On
- scoping, How to Scope the Automation Relative to the Problem
- setup cost, Setup Costs
- simulation, Simulation and Synthetic Data-Simulations Still Need Human Review
- synthetic data, Simulation and Synthetic Data-Simulations Still Need Human Review
- UI optimization, User Interface Optimizations
- versus discovery, Discovery versus automation
- when to use, Motivation: When to Use These Methods?

B

benchmarking, automations, How to Benchmark Well-How to Benchmark Well

bias, Bias

binary classification, Binary Classification

Binary Large Object (BLOB) data, Raw data

BLOB (Binary Large Object) data, Raw data

- 3D points, 3D
- geospatial analysis, Geospatial
- images, Images
- medical, Medical
- storage, Data Storage: Where Does the Data Rest?
- text, Text
- video, Video-Video

bounding box, Box (2D)

box (2D), Box (2D)

budgeting, The budget

business processes

- direct annotation, Business Process Integration

business transformations, Business Transformation-Business Transformation

C

case studies, Introduction

cataloging tools, Tools Overview

circles, Ellipse and circle

classes, labels, Simple Schema for a Strawberry Picking System

classification upgrading, Upgraded Classification

closed source annotation, Open Source and Closed Source-Customization is the name of the game

cloud computing, signed URL, Cloud connections and signed URLs

Cloud-Optimized GeoTIFF (COG), Geospatial

COCO (Common Objects in Context), Transfer Learning's Relation to Datasets (Fine-Tuning), Applied Versus Research Sets

code, training data and, Training Data Is Like Code-Training Data Is Like Code

COG (Cloud-Optimized GeoTIFF), Geospatial

columns as attributes, Introduction to Labels

Common Objects in Context (COCO), Transfer Learning's Relation to Datasets (Fine-Tuning), Applied Versus Research Sets

compound files, User-Defined Types (Compound Files)

computer vision

- spatial types
  - box (2D), Box (2D)
  - circles, Ellipse and circle
  - complex types, Complex spatial types
  - cuboids, Cuboid
  - ellipses, Ellipse and circle
  - full image tags, Full image tag
  - keypoint geometry, Keypoint geometry
  - multiple uses, Types with multiple uses

    - polygons, Polygon, Polygons and raster masks
    - raster mask, Raster mask
    - spatial templates, custom, Custom spatial templates

configuration, security, Security configuration

connections, external references, External Reference Connection

constraints, attributes, Attribute concepts

correction time for automation, Correction Time

cost, Costs

- automation setup, Setup Costs
- reductions, Costs

crowd sourcing mindset, The New “Crowd Sourcing”: Your Own Experts-Common Project Mistakes

cuboids, Cuboid

culture of training data, Culture of Training Data

D

data

- errors, All training data has errors
    - common causes, Common Causes of Training Data Errors
- historical, theories, Working with Historical Data-Working with Historical Data
- intentionally chosen, Intentionally Chosen Data Is Best
- resting, Data Storage: Where Does the Data Rest?

- supervising, Who supervises the data
    - direct annotation, Supervising Existing Data
- upgrading over time, Upgrading Data Over Time
- uploading, Getting Data to Models
- usage assumptions, Surface Assumptions Around Usage of Your Training Data-Use definitions and processes to protect against assumptions
- users
    - annotators, Annotators
    - application engineers, Application engineers
    - data scientists, Data scientists
    - ML programs, ML programs
    - stakeholders, Other stakeholders

data access, Data Access

- exporting and, Disambiguating Storage, Ingestion, Export, and Access
- file-based exports, File-Based Exports
- ingestion, Disambiguating Storage, Ingestion, Export, and Access
- storage disambiguation, Disambiguating Storage, Ingestion, Export, and Access
- streaming data
    - benefits, Streaming benefits
    - drawbacks, Streaming drawbacks
    - ecosystem integrations, Integrations with the Ecosystem

    - fetch and stream, Example: Fetch and stream
    - queries introduction, Queries Introduction

data catalog setup, Data Catalog Setup

data collection, AI transformation and, You Can Change What Data You Collect

data configuration

- installation
    - data lifecycle, Data configuration
    - versioning resolution, Data configuration

data discovery, Data Discovery: What to Label

- human exploration, Human Exploration
- metadata exploration, Metadata Exploration
- raw data exploration, Raw Data Exploration

data engineers, Data Engineer

data errors, A Game of Telephone

data exploration, Data Exploration-Similar image reduction

data lifecycle, Data configuration

data management

- completed tasks, Completed Tasks-Completed Tasks
- freshness, Freshness
- metadata maintenance, Maintaining Set Metadata
- quality, Quality

- task management, Task Management

data meaning, changing, You Can Change the Meaning of the Data

data relevancy, Data Relevancy

- raw data collection and, Raw data collection
- system design and, Overall system design

data representations, engineering and, Data representations for engineering

data science access, security and, Data science access

data scientists, Data scientists

data setup, Data Setup

data supervision theory, Who Supervises the Data Matters-Who Supervises the Data Matters

data-centric AI, The Creative Revolution of Data-centric AI

- data collection
    - changing, You Can Change What Data You Collect
- data creation, You Can Create New Data
- data meaning, You Can Change the Meaning of the Data

data-centric ML (machine learning), Data-Centric Machine Learning

databases

- installation, Database
- reasons to use, Training data–centric (system of record)

dataflow, Basics

- data organization
    - dynamic organization, Filters and dynamic organization

    - filters, Filters and dynamic organization
    - folders, Folders and static organization
    - static organization, Folders and static organization
- pipelines, Pipelines and Processes
    - dataset connection, The dataset connection-Hooks
- processes, Pipelines and Processes
    - dataset connection, The dataset connection-Hooks
- streaming, Overview of Streaming

DataMaps, Defining DataMaps

datasets, Distinctions Between a Dataset, Model, and Model Run

- definition, Training Data Is Here to Stay-New Types of Users
- versus human supervision, Human Supervision Is Different from Classic Datasets-Discovery
- manual creation, Let’s Manually Create Our First Set-Let’s Manually Create Our First Set
- pipelines, The dataset connection-Hooks
- processes, The dataset connection-Hooks
- transfer learning and, Transfer Learning’s Relation to Datasets (Fine-Tuning)-Transfer Learning’s Relation to Datasets (Fine-Tuning)

debugging, ML modeling and, Using the Model to Debug the Humans-Using the Model to Debug the Humans

decision making, scope, Decision-making process

dedicated tooling, Realizing the Need for Dedicated Tooling-Advent of new standards

definitions, attributes and, Attributes Introduction

development system, Development System

- sequentially dependent discoveries, Sequentially dependent discoveries

Diffgram, Installation

direct supervision, human supervision, Human Alignment Is Human Supervision

Director of AI Data Responsibilities, Director of AI data responsibilities

disambiguating storage, Disambiguating Storage, Ingestion, Export, and Access

discovery, Discovery

- versus automation, Discovery versus automation

Docker, Packaging

domain-specific automation, Domain Specific-Heuristics-Based Labeling

domain-specific query languages, Data Catalog Setup

dynamic organization, Filters and dynamic organization

E

efficiency, Training Data Efficiency-Training Data Efficiency

ellipses, Ellipse and circle

embedded supervision, Embedded Supervision

employees, cost center, People, Process, and Data-People, Process, and Data

end users, New Types of Users

- input, cost reduction and, Costs

engineering, data representations and, Data representations for engineering

errors in data, All training data has errors

- common causes, Common Causes of Training Data Errors

ethics, supervised data, Ethical and Privacy Considerations

event-focused analytics, Video

events, interactive automations, What Is a Watcher? (Observer Pattern)

exploring data, Data Exploration-Similar image reduction

exporting

- data access and, Disambiguating Storage, Ingestion, Export, and Access
- file-based exports, File-Based Exports

external references

- connections, External Reference Connection

F

failures, Failures-Failures

- deployed system example, Failures-What Training Data Is Not

file-based exports, File-Based Exports

files

- compound, User-Defined Types (Compound Files)
- user-defined types, User-Defined Types (Compound Files)

filters, Filters and dynamic organization

folders, Folders and static organization

form type, attributes, Attribute concepts

full image tags, Full image tag

G

game of telephone analogy, A Game of Telephone-When a system of record is needed

- naive approach and, Naive approaches

GenAI (generative AI), Generative AI-Human Alignment Is Human Supervision

General Language Understanding Evaluation (GLUE), Applied Versus Research Sets

generative AI (see GenAI)

generative pre-trained transformers (GPTs), Generative AI

generic attributes, Attribute concepts

geolocation, Raw Data Storage

geometry, keypoint, Keypoint geometry

geometry-based labeling, Geometry-Based Labeling

geospatial analysis

- BLOB (Binary Large Object) data, Geospatial

GeoTiff, Geospatial

GLUE (General Language Understanding Evaluation), Applied Versus Research Sets

Gold Standard training, Gold Standard Training

GPTs (generative pre-trained transformers), Generative AI

GrabCut, Interactive Drawing Box to Polygon Using GrabCut

guides, Guides and Instructions-Judgment Calls

H

HCI (human–computer interaction), Human Computer Supervision

HCS (human computer supervision), Human Computer Supervision

heuristics-based labeling, Heuristics-Based Labeling

historical data

- theories, Working with Historical Data-Working with Historical Data

human alignment, Human Alignment Is Human Supervision

- (see also human supervision)

human computer supervision (HCS), Human Computer Supervision

human supervision

- constitutional set of instructions, Human Alignment Is Human Supervision
- direct supervision, Human Alignment Is Human Supervision
- GenAI and, Generative AI
- indirect supervision, Human Alignment Is Human Supervision
- integration, Human Alignment Is Human Supervision
- prompt engineering, Human Alignment Is Human Supervision
- versus classic datasets, Human Supervision Is Different from Classic Datasets-Discovery

human tasks, Why Are Human Tasks Needed?

- setup, Basics-Basics

human training, New Training and Knowledge Are Required-Executives

human-centered definitions, Attributes Introduction

humans, data and, The human role-The human role

human–computer interaction (HCI), Human Computer Supervision

I

IAM (identity and access management) service, Access Control

identity, Identity and Authorization

identity and access management (IAM) (see IAM (identity and access management))

image classification, Image Classification (Tags)

ImageNet 2009, Applied Versus Research Sets

images, BLOB (Binary Large Object) data, Images

indirect supervision, Human Alignment Is Human Supervision

industry case studies

- big tech, Big-Tech Challenges-The case for aligning teams around training data
- insurance tech startup, Insurance Tech Startup Lessons-Too late to bring in training data software
- QA at self-driving project, Quality Assurance at a Large-Scale Self-Driving Project-Don’t overfocus on spatial information

- security startup, training data tools, A Security Startup Adopts Training Data Tools-A Security Startup Adopts Training Data Tools

industry stories, Stories-“Sports Startup Successfully Used Pre-Labeling”

ingestion, data access and, Disambiguating Storage, Ingestion, Export, and Access

initial usage, Initial Usage

installation, Installation

- data configuration
  - data lifecycle, Data configuration
  - versioning resolution, Data configuration
- databases, Database
- packaging
  - Docker, Packaging
  - Kubernetes (K8s), Packaging
  - single language-specific package, Packaging
- storage, Storage

installed solutions, security and, Security benefits

instance concepts, Instance Concept Refresher

instances

- annotation, Simple Schema for a Strawberry Picking System
- attributes and, Example of an attribute in relation to an instance

instructions, Guides and Instructions-Judgment Calls

integration, modeling, Modeling Integration

integrations, Integrations

- categories, Integrations

interactive automations, Introduction

- (see also automation)
- creating, Creating Your Own
- drawing box, GrabCut and, Interactive Drawing Box to Polygon Using GrabCut
- events, What Is a Watcher? (Observer Pattern)
- model prediction, Full Image Model Prediction Example
- observer pattern, What Is a Watcher? (Observer Pattern)
- person detection, Example: Person Detection for Different Attribute
- region of interest capture, Interactive Capturing of a Region of Interest
- SAM (Segment Anything Model), Interactive Annotation Automation
- setup, Technical Setup Notes
- watchers, What Is a Watcher? (Observer Pattern)

interfaces

- annotation interfaces, Annotation Interfaces
- customizing, Do You Need to Customize the Interface?

iterations, Iterations

J

judgment calls, Judgment Calls

- per-sample calls, Per-Sample Judgment Calls

K

Kaggle TSA Competition case study, Kaggle TSA Competition-Learning lessons and mistakes

keypoint geometry, Keypoint geometry

Kubernetes (K8s), Packaging

L

labeling, Prioritization: What to Label

- automation, “Fully” automatic labeling for novel model creation
- direct annotation, Depth of Labeling
- geometry-based, Geometry-Based Labeling
- heuristics-based, Heuristics-Based Labeling
- metadata, Adding Pre-Labeling-Based Metadata
- pre-labeling
    - portions of data, Pre-Labeling a Portion of the Data Only-How to get started pre-labeling
    - standard, Standard Pre-Labeling-Caveats

labels, Schema Deep Dive Introduction

- as what, What Do We Care About?
- attributes, Introduction to Labels
- classes, Simple Schema for a Strawberry Picking System
- SQL comparison, Introduction to Labels

languages, Python, Training Data Is Like Code

large language models (LLMs), Generative AI

licensing models, cost reduction , Costs

lifecycle, data, Data configuration

LLMs (large language models), Generative AI

loss prevention

- metadata, Preventing Lost Metadata-Preventing Lost Metadata

M

machine learning (ML), Training Data Introduction

maintenance

- actions
  - schema depth, Increase schema depth to improve performance
  - spatial type, raw data and, Better align the spatial type to the raw data
  - task creation, Create more tasks
- net lift, Net Lift-Net Lift
- raw data, Change the raw data

media

- automation and, Media Specific-Considerations
- raw data, Raw data media types

media-specific research, Media-Specific Research-Media-Specific Research

medical files

- BLOB (Binary Large Object) data, Medical

metadata, Metadata-Metadata

- loss prevention, Preventing Lost Metadata-Preventing Lost Metadata

metrics-oriented tools, Need for Both Qualitative and Quantitative Evaluations

ML (machine learning), Training Data Introduction

- classic, Training Data Controls the ML Program
- data-centric, Data-Centric Machine Learning
- deep learning system, Training Data Controls the ML Program
- models
  - debugging humans, Using the Model to Debug the Humans-Using the Model to Debug the Humans
  - training data and, The Boundary Between Modeling and Training Data
- program ecosystem, ML Program Ecosystem
- software delivery, A Paradigm to Deliver Machine Learning Software
- supervising machines, The Art of Supervising Machines
- tasks, training data types and, Relationship of Tasks to Training Data Types
- tools, Training Data for Machine Learning

ML programs, ML programs

model runs, Distinctions Between a Dataset, Model, and Model Run

modeling integration, Modeling Integration

models, Distinctions Between a Dataset, Model, and Model Run

- data
    - uploading, Getting Data to Models

multi-user systems, Multi-User versus Single-User Systems

N

naive approaches, Naive and Training Data–Centric Approaches, Naive approaches-Naive approaches

net lift, Net Lift-Net Lift

novel data

- creation, What Can You Do With Training Data?
- maintenance, What Can You Do With Training Data?

O

object detection, Object Detection

OIDC (OpenID Connect), Identity and Authorization

open source annotation, Open Source and Closed Source-Customization is the name of the game

open source standards, history, Open Source Standards

OpenID Connect (OIDC), Identity and Authorization

optimization, Optimization

- premature, avoiding, Trap to Avoid: Premature Optimization in Training Data-Trap to Avoid: Premature Optimization in Training Data
- UI (user interface), User Interface Optimizations

organizing data

- dynamic organization, Filters and dynamic organization
- filters, Filters and dynamic organization
- folders, Folders and static organization
- static organization, Folders and static organization

P

packaging

- Docker, Packaging
- Kubernetes (K8s), Packaging
- single language-specific package, Packaging

per-export snapshot, versioning, Per-export snapshots

per-file versioning, Per file and per set

per-instance history, versioning, Per-instance history

per-sample judgment calls, Per-Sample Judgment Calls

per-set versioning, Per file and per set

permission setting, Example of Setting Permissions

personally identifiable information (PII), Cloud connections and signed URLs, Personally Identifiable Information

personas, Many Personas

PII (personally identifiable information), Cloud connections and signed URLs, Personally Identifiable Information

pixels, semantic segmentation, Semantic Segmentation

platforms, Platform and suite solutions-Platform and suite solutions

polygons, Polygon

pose estimation, Pose Estimation

pre-labeling

- data updates, Updating Data-Pre-labeling data prep process
- portions of data, Pre-Labeling a Portion of the Data Only
    - off-the-shelf models, Use off-the-shelf models
    - one step early trick, The “one step early” trick
    - separation of concerns, Clear separation of concerns
- standard, Standard Pre-Labeling-Caveats

predefined selection, attributes and, Attribute concepts

premature optimization, Trap to Avoid: Premature Optimization in Training Data-Trap to Avoid: Premature Optimization in Training Data

prioritization, Prioritization: What to Label

privacy, supervised data, Ethical and Privacy Considerations

process improvement, Process Improvement Opportunities

producers and consumers, Upstream Producers and Downstream Consumers-Spectrum of Training Data Team Engagement

- companies, How Companies Produce and Consume Data
- dedicated producers, Dedicated Producers and Other Teams
- producers from other teams, Organizing Producers from Other Teams

project structure, Tasks and Project Structure

push automation

- cost reduction, Costs

Python, Training Data Is Like Code

Q

QA (quality assurance), Introduction

qualitative evaluation, Need for Both Qualitative and Quantitative Evaluations

quality assurance

- annotators
  - as partners, Annotators Are Partners-Annotator needs
  - trust, Annotator Trust

quality of data, Quality-Quality

- annotations, Quality

quantitative evaluation, Need for Both Qualitative and Quantitative Evaluations

query languages, domain-specific, Data Catalog Setup

querying versus streaming, Data Access

question attributes, Attribute concepts

R

raster masks, Raster mask

raster methods

- semantic segmentation, Semantic Segmentation

raw data, Raw data-Raw data, Raw Data Concepts-Raw Data Concepts, Change the raw data

- data relevancy and, Raw data collection
- media types, Raw data media types
- spatial type and, Better align the spatial type to the raw data
- storage, Raw Data Storage
    - geolocation, Raw Data Storage
    - pass by reference, By Reference or by Value-By Reference or by Value
    - pass by value, By Reference or by Value-By Reference or by Value
    - storage class, Raw Data Storage
    - vendor support, Raw Data Storage

RBAC (role-based access control), Identity and Authorization

relationships, Sequences and Relationships

- when, When

remote storage, Remote Storage

research sets versus applied sets, Applied Versus Research Sets-Applied Versus Research Sets

ROI (return on investment), Large-scale thoughts, Key Levers on Training Data ROI

role-based access control (RBAC), Identity and Authorization

root-level access

- security, Root-level access

S

SaaS (software-as-a-service)

- versus installed training data, Installed Versus Software as a Service

SAM (Segment Anything Model), Trade-Offs with Types

sample creation

- binary classification, Binary Classification
- classification upgrade, Upgraded Classification
- geometric representation, Geometric Representations
- manual set creation, Let’s Manually Create Our First Set-Let’s Manually Create Our First Set
- strawberry picking system, Simple Schema for a Strawberry Picking System-Simple Schema for a Strawberry Picking System

scale, Scale

- defining, Why is it useful to define scale?-Transitioning from small to medium scale
- large scale, Large-scale thoughts-Large-scale thoughts

schema, Schema-Schema, Schema Deep Dive Introduction, Basics

- attributes, Schema Deep Dive Introduction
- automation, Check What Part of the Schema a Method Is Designed to Work On
- complexity, attributes and, Schema complexity trade-off
- depth, Increase schema depth to improve performance
- design failure, Schema
- joint responsibilities, Joint responsibility

- labels, Schema Deep Dive Introduction
- persistence, Schemas' Staying Power
- spatial representations, Schema Deep Dive Introduction
- system usefulness theory, A System Is Only as Useful as Its Schema-A System Is Only as Useful as Its Schema

scope, Scope

- attributes, Attribute concepts
- cautions, Cautions
- decision-making process, Decision-making process
- platforms, Platform and suite solutions-Platform and suite solutions
- point solutions, Point solutions-Point solutions
- suites, Platform and suite solutions-Platform and suite solutions

scoping automation, How to Scope the Automation Relative to the Problem

security

- access control, Access Control
- architecture, Security architecture
- attack surface, Attack surface
- authorization, Identity and Authorization
- configuration, Security configuration
- data science access, Data science access
- identity, Identity and Authorization
- installed solutions benefits, Security benefits

- PII (personally identifiable information), Personally Identifiable Information
- root-level access, Root-level access
- URLs, signed, Signed URLs
    - cloud connections, Cloud connections and signed URLs
- user access, User access

Segment Anything Model (SAM), Trade-Offs with Types

segmentation

- SAM (Segment Anything Model), Trade-Offs with Types
- semantic, Semantic Segmentation-Image Classification (Tags)

semantic segmentation, Semantic Segmentation-Image Classification (Tags)

separation of end concerns, Separation of End Concerns

sequences, Sequences and Relationships

- when, When

sequentially dependent discoveries, Sequentially dependent discoveries

setting permissions, example, Example of Setting Permissions

setup

- annotator setup, Annotator Setup
    - embedded, Embedded
    - portal, Portal (default)
- data catalog setup, Data Catalog Setup
- data setup, Data Setup

- tasks setup, Tasks Setup
- workflow, Workflow Setup

simulation, Simulation and Synthetic Data-Simulations Still Need Human Review

single language-specific package, Packaging

single-user systems, Multi-User versus Single-User Systems

SMEs (subject matter experts), Subject Matter Experts

social bias, spatial types and, Using Spatial Types to Prevent Social Bias-Joint responsibility

software, Think Learning Curve, Not Perfection

software development, training data and, History of Development Affects Training Data Too

SoR (system of record), Introduction

- need for, When a system of record is needed-When a system of record is needed
- training data–centric approach, Training data–centric (system of record)-The first steps

spatial bias, avoiding, One way to avoid spatial bias-One way to avoid spatial bias

spatial complexity, Attribute Complexity Exceeds Spatial Complexity-Example of sharing attributes between labels

spatial representations, Schema Deep Dive Introduction, Spatial Representation—Where Is It?

- computer vision
    - bounding box, Box (2D)

    - box (2D), Box (2D)
    - circles, Ellipse and circle
    - complex types, Complex spatial types
    - cuboids, Cuboid
    - custom spatial templates, Custom spatial templates
    - ellipses, Ellipse and circle
    - full image tags, Full image tag
    - keypoint geometry, Keypoint geometry
    - multiple uses, Types with multiple uses
    - polygons, Polygon, Polygons and raster masks
    - raster mask, Raster mask
- social bias, spatial types and, Using Spatial Types to Prevent Social Bias-Joint responsibility
- trade-offs, Trade-Offs with Types-Trade-Offs with Types

spatial templates, custom, Custom spatial templates

spatial type, raw data and, Better align the spatial type to the raw data

staff expectations, New Expectations People Have for the Future of AI

stakeholders, Other stakeholders

standards, Standards

static organization, Folders and static organization

storage

- efficiency, Organizing Data and Useful Storage

- installation, Storage
- organizing, Organizing Data and Useful Storage
- raw data, Raw Data Storage
  - geolocation, Raw Data Storage
  - pass by reference, By Reference or by Value-By Reference or by Value
  - pass by value, By Reference or by Value-By Reference or by Value
  - storage class, Raw Data Storage
  - vendor support, Raw Data Storage
- remote, Remote Storage

storage class, Raw Data Storage

strawberry picking, sample creation, Simple Schema for a Strawberry Picking System-Simple Schema for a Strawberry Picking System

streaming

- benefits, Streaming benefits
- drawbacks, Streaming drawbacks
- ecosystem integration, Integrations with the Ecosystem
- fetch and stream, Example: Fetch and stream
- overview, Overview of Streaming
- queries introduction, Queries Introduction
- versus querying, Data Access

suites, Platform and suite solutions-Platform and suite solutions

supervision

- embedded, Embedded Supervision
- HCS (human computer supervision), Human Computer Supervision

synthetic data, Simulation and Synthetic Data-Simulations Still Need Human Review

system design, data relevancy and, Overall system design

system maturity levels, Levels of System Maturity of Training Data Operations

system of record (SoR), Introduction

system planning, Planning a Great System-Planning a Great System

system usefulness, schema and, A System Is Only as Useful as Its Schema-A System Is Only as Useful as Its Schema

T

tags, Image Classification (Tags)

- (see also image classification)

task management, Task Management

tasks

- assigning, Task Assignment Concepts
- human tasks, Why Are Human Tasks Needed?
  - setup, Basics-Basics
- ML (machine learning) training data types and, Relationship of Tasks to Training Data Types
- review loops

    - consensus, Consensus-Consensus
    - standard, Standard review loop
- structure, Tasks and Project Structure

tasks setup, Tasks Setup

teams, New Expectations People Have for the Future of AI

- staff expectations, New Expectations People Have for the Future of AI

technical definitions, Attributes Introduction

technical example of attributes, Technical example of an attribute

templates

- attributes, Attribute concepts
- spatial templates, custom, Custom spatial templates

text, BLOB (Binary Large Object) data, Text

theories

- assumptions about data usage, Surface Assumptions Around Usage of Your Training Data-Use definitions and processes to protect against assumptions
- data supervision, Who Supervises the Data Matters-Who Supervises the Data Matters
- historical data, Working with Historical Data-Working with Historical Data
- human supervision versus classic datasets, Human Supervision Is Different from Classic Datasets-Discovery
- intentionally chosen data, Intentionally Chosen Data Is Best

- system usefulness, schema and, A System Is Only as Useful as Its Schema-A System Is Only as Useful as Its Schema
- training data, code and, Training Data Is Like Code-Training Data Is Like Code

tool terminology, Introduction

tooling, Tooling Proficiency

tools

- annotation, Tools Overview
- annotators, time using, How Long Will the Average Annotator Be Using It?
- assumptions, Hidden Assumptions-Hidden Assumptions
- dedicated tooling, Realizing the Need for Dedicated Tooling-Advent of new standards
- demand, Growing Selection of Tools
- development system, Development System-Sequentially dependent discoveries
- embedded supervision, Embedded Supervision
- employees and, People, Process, and Data-People, Process, and Data
- HCS (human computer supervision), Human Computer Supervision
- human training, New Training and Knowledge Are Required-Executives
- in-between, Tools in between-Tools in between
- metrics-oriented, Need for Both Qualitative and Quantitative Evaluations

- ML (machine learning), Training Data for Machine Learning
- modern training data tools, Modern Training Data Tools-New Engineering Principles
- off-the-shelf, Off-the-Shelf Dedicated Training Data Tooling on Your Own Hardware
- personas, Many Personas
- separation of end concerns, Separation of End Concerns
- software, Think Learning Curve, Not Perfection
- standards, Standards
- workflow, Tools Overview

tools cataloging, Tools Overview

Train/Val/Test, Train/Val/Test Is the Cherry on Top

training, Training-Training

- Gold Standard training, Gold Standard Training

training data

- as named domain, A New Thing for Data Science-A New Thing for Data Science
- culture of, Culture of Training Data
- description, Training Data Introduction
- difficulties, What Makes Training Data Difficult?-What Makes Training Data Difficult?
- importance of, Why Training Data Matters-The Foundation of Successful AI
- installation, Installation

- installed, Installed Versus Software as a Service
- intents, Training Data Intents-The human role
- ML modeling and, The Boundary Between Modeling and Training Data
- SaaS (software as a service), Installed Versus Software as a Service
- uses, What Can You Do With Training Data?-What Can You Do With Training Data?
- versus data science, Upstream Producers and Downstream Consumers-Classic company modification

Training Data Evangelist, Training Data Evangelist

Training Data Production Manager, Training Data Production Manager(s)

training data–centric approach, Training data–centric (system of record)-The first steps

transfer learning, datasets and, Transfer Learning's Relation to Datasets (Fine-Tuning)-Transfer Learning's Relation to Datasets (Fine-Tuning)

truncation, attributes, Attribute concepts

U

UI (user interface) optimization, User Interface Optimizations

Uniform Resource Locators (URLs) (see URLs (Uniform Resource Locators))

updates, pre-labeling and, Updating Data-Pre-labeling data prep process

URLs (Uniform Resource Locators)

- signed, Signed URLs
    - cloud connections, Cloud connections and signed URLs

use cases

- effects, Conceptual Effects of Use Cases-Ongoing impact of use cases
- rubrics for good, Rubric for Good Use Cases-Evaluation example

user access

- security and, User access

user-defined file types, User-Defined Types (Compound Files)

users

- annotators, Annotators
- application engineers, Application engineers
- data scientists, Data scientists
- end users, New Types of Users
- ML programs, ML programs
- roles, User Roles
- stakeholders, Other stakeholders

V

vector methods, semantic segmentation, Semantic Segmentation

vendor support

- raw data and, Raw Data Storage

versioning, Versioning

- per file, Per file and per set
- per set, Per file and per set
- per-export snapshot, Per-export snapshots

- per-instance history, Per-instance history

versioning resolution, Data configuration

video, Video

- BLOB (Binary Large Object) data, Video-Video
- common issues, Common issues in video annotation
- events, Video events
- motion, Motion
- persistent objects, Persistent objects: football example
- sequence errors, Detecting sequence errors
- series example, Series example
- static objects, Static objects
- tracking objects through time, Examples of tracking objects through time (time series)

W

what

- attributes and, Attributes Introduction
- labels and, What Do We Care About?

when

- relationships, When
- sequences, When

wizards, Ingest Wizards

workflow

- analytics
  - annotation metrics, Annotation Metrics Examples
  - data exploration, Data Exploration-Similar image reduction
- annotation
  - direct, Direct Annotation-Common issues in video annotation
- characteristics, Glue Between Tech and People
- data errors, common causes, Common Causes of Training Data Errors
- dataflow
  - data organization, Data Organization
  - pipelines, Pipelines and Processes-Direct Annotation
  - processes, Pipelines and Processes-Direct Annotation
  - streaming, Overview of Streaming
- datasets, Distinctions Between a Dataset, Model, and Model Run
- ML models, debugging humans, Using the Model to Debug the Humans-Using the Model to Debug the Humans
- model run, Distinctions Between a Dataset, Model, and Model Run
- models, Distinctions Between a Dataset, Model, and Model Run
- quality assurance
  - annotator trust, Annotator Trust
  - annotators as partners, Annotators Are Partners-Annotator needs
- schema, persistence, Schemas' Staying Power
- tasks, Glue Between Tech and People
  - assigning, Task Assignment Concepts

    - human tasks, Why Are Human Tasks Needed?, Basics-Basics
    - review loops, Standard review loop-Consensus
- tools, Tools Overview
- training, Training-Training
    - Gold Standard training, Gold Standard Training
- user roles, User Roles

workflow setup, Workflow Setup

OceanofPDF.com

About the Author

Anthony Sarkis is the lead engineer on Diffgram Training Data Management software and founder of Diffgram Inc. Prior to that he was a software engineer at Skidmore, Owings & Merrill and cofounded DriveCarma.ca.

OceanofPDF.com

Colophon

The animals on the cover of *Training Data for Machine Learning* are black-tailed prairie dogs (*Cynomys ludovicianus*). While they are actually a type of ground squirrel, they received the name prairie dog because of the habitats they live in and because the sound of their warning calls are similar to a dog's bark.

Black-tailed prairie dogs are small rodents that weigh between 2 and 3 pounds and grow between 14 and 17 inches long. They have mostly tan fur that is lighter on their bellies and their namesake black tail tip. They have short, round ears, and eyes that are relatively large in comparison to the size of their bodies. Their feet have long claws, which are ideal for digging burrows into the ground.

True to their name, black-tailed prairie dogs live in a variety of grasslands and prairie in the Great Plains of North America. Their habitat usually consists of flat, dry, sparsely vegetated land, such as short grass prairie, mixed-grass prairie, sagebrush, and desert grasslands. Their expansive range is east of the Rocky Mountains in the United States and Canada to the border of Mexico.

Black-tailed prairie dogs may not be considered endangered, but they are a keystone species. They impact the diversity of vegetation, vertebrates, and invertebrates because of their foraging habits and presence as potential prey. It has been shown that grasslands inhabited by them have a higher degree of biodiversity than grasslands not inhabited by them. Prior to a large amount of habitat destruction, they used to be the most abundant species of prairie dog in North America. Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover illustration is by Karen Montgomery, based on a black and white engraving from *The Natural History of Mammals*. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.

OceanofPDF.com